# Livirooms Complete Documentation

<!-- Source: README.md -->

# Rental Apartemen (Livirooms) — Dokumentasi Sistem

Platform penyewaan properti multi-tipe berbasis subscription dan reservasi hotel OTA yang mendukung penyewaan harian, mingguan, bulanan, dan tahunan. Melayani empat aktor utama: **Konsumen (Guest)**, **Host/Mitra**, **Admin**, dan **Corporate**.

> 📍 **Navigasi Lengkap:** Buka **INDEX.md (INDEX.md)** untuk daftar rute topik dokumen dan implementasi kode.

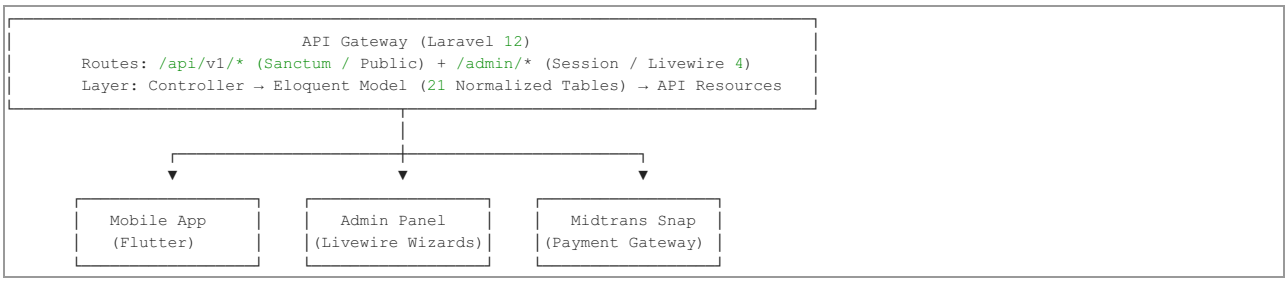
---

## 1. Ringkasan Proyek

| Aspek | Detail Teknis |
|---|---|
| **Nama Proyek** | Rental Apartemen / Livirooms Multi-Accommodation Platform |
| **Tipe Akomodasi** | Apartemen, Rumah, Villa, Hotel Residence, Guest House, Kost Eksklusif, **Hotel (OTA Mode)** |
| **Periode Sewa** | Harian · Mingguan · Bulanan · Tahunan |
| **Platform** | Mobile App (Flutter) + Admin Panel (Laravel 12 / Livewire 4) + Corporate Web |
| **Basis Data** | PostgreSQL (21 tabel ternormalisasi 3NF untuk entitas produk) |
| **Payment Gateway** | Midtrans Snap (VA, QRIS, E-Wallet, Kartu Kredit) dengan Webhook S2S terverifikasi |
| **Autentikasi** | Laravel Sanctum (API Mobile) + Session & Spatie Permission (Admin Panel) |

---

## 2. Arsitektur Sistem




---

## 3. Stack Teknologi Terpasang

### Backend (Laravel 12)

- **Framework:** `laravel/framework ^12.0`
- **Autentikasi API:** `laravel/sanctum ^4.3`
- **UI Admin Panel:** `livewire/livewire ^4.3` (Livewire 4)
- **Tabel Server-side:** `yajra/laravel-datatables-* ^12`
- **Otorisasi:** `spatie/laravel-permission ^6.9`
- **Payment Gateway:** `midtrans/midtrans-php ^2.6`
- **PDF Generation:** `barryvdh/laravel-dompdf`
- **Basis Data:** PostgreSQL (Production)

### Mobile (Flutter)

- **Framework:** Flutter SDK `>=3.19.0`
- **State Management:** `provider ^6.1.2`
- **HTTP Client:** `http ^1.2.1`
- **In-App Payment WebView:** `webview_flutter ^4.13.0`
- **Deep Link Handler:** `app_links ^6.3.2` (`livirooms://payment-result`)

---

## 4. Struktur Database & Model (21 Tabel Ternormalisasi)

Data modeling akomodasi menggunakan arsitektur 3NF terpadu berbasis 1 tabel akar `accommodations` dan 20 tabel satelit:

1. `accommodations` (Entitas akar, diskriminator `booking_mode`: `room` vs `unit`)
2. `building_lists` (Master gedung/tower)
3. `unit_types` (Sumber tunggal harga, kapasitas, dan tipe kamar)
4. `unit_type_prices` (Tarif harian/mingguan/bulanan)
5. `unit_type_images` (Foto tipe kamar)
6. `facility_groups` (Grup kategori fasilitas)
7. `facility_items` (Item fasilitas dengan `context`: `facility`, `furniture`, `spec`)
8. `facility_tags` (Tag fasilitas ringkas)
9. `galleries` (Galeri foto resmi dan foto tamu)
10. `reviews` & `review_photos` (Ulasan seragam skala 1-10)
11. `accommodation_policies` (Kebijakan umum, check-in, dan deposit)

12. `accommodation_questions` (Q&A manager & user)
13. `nearby_points` (Titik minat & transportasi sekitar)
14. `room_furniture_groups` & `room_furniture_items` (Furnitur per ruangan & bed config)
15. `bed_facilities` (Ringkasan tipe ranjang)
16. `accommodation_description_paragraphs` (Paragraf tambahan)
17. `accommodation_details`, `accommodation_detail_groups`, `accommodation_detail_lines` (Rincian spesifikasi & biaya kompleks)

---

### 5. Administrasi & Livewire Form Wizards

Pendaftaran dan pengeditan akomodasi dilakukan melalui form wizard multi-step yang intuitif:

- **Accommodation Wizard (Unified):** [`App\Livewire\Accommodations\WizardForm`] (file:///d:/Project/Fastwork/rental.apartemen/laravel/app/Livewire/Accommodations/WizardForm.php)

Setiap wizard mengelola state transaksional yang aman (`DB::transaction`), upload multi-file foto, validasi per-tab, dan otorisasi role Spatie.

<!-- Source: INDEX.md -->

# Indeks Dokumentasi — Livirooms Multi-Accommodation Architecture

Peta rute dokumentasi proyek: **Topik** → **File Dokumentasi** → **Implementasi Kode**.

> ### Titik Masuk AI: `ai/00-orientation.md`
>
> Paket **`ai/` (ai/)** adalah representasi arsitektur `laravel/` + `flutter/` terpadu:
>
> | File | Isi Pokok |
> |---|---|
> | ai/00-orientation.md (ai/00-orientation.md) | **Titik masuk utama.** Routing direktori, aturan verifikasi inventaris, dan negative constraints. |
> | ai/01-backend-map.md (ai/01-backend-map.md) | Laravel: 21 model ternormalisasi, layer data flow, state machine, Midtrans payments, Livewire wizard. |
> | ai/02-frontend-map.md (ai/02-frontend-map.md) | Flutter: 45 layar, 45 route, 19 service, 44 model, fixture dummy contract. |
> | ai/03-api-contract.md (ai/03-api-contract.md) | 85 endpoint → `Controller::method` → Dart service → payload fixture. |
> | ai/04-drift-register.md (ai/04-drift-register.md) | Daftar drift terverifikasi & status perbaikan data model 3NF. |
>
> Untuk arsitektur data Hotel vs Property, dokumen master adalah:
> **domain/hotel-vs-property.md (domain/hotel-vs-property.md)**

---

## Navigasi Dokumen Berdasarkan Topik

| Kebutuhan Anda | File Rujukan Utama |
|---|---|
| **Memahami Model Database 21 Tabel (3NF)** | **domain/hotel-vs-property.md (domain/hotel-vs-property.md)** |
| **Arsitektur Global & Stack Teknologi** | README.md (README.md) |
| **Konvensi & Panduan Backend Laravel 12** | `/laravel/CLAUDE.md` + ai/01-backend-map.md (ai/01-backend-map.md) |
| **Konvensi & Arsitektur Mobile Flutter** | `/flutter/CLAUDE.md` + ai/02-frontend-map.md (ai/02-frontend-map.md) |
| **Livewire Form Wizards (Add/Edit Property & Hotel)** | configuration/admin-panel.md (configuration/admin-panel.md) + configuration/ui-patterns.md (configuration/ui-patterns.md) |
| **Sistem Pembayaran Midtrans Snap & Webhook** | payments/system.md (payments/system.md) + features/payments.md (features/payments.md) |
| **Sistem Referral & Reward** | features/referral.md (features/referral.md) |
| **Sampel Response API & Master Contract** | response/detail-property.json (response/detail-property.json), response/hotels.json (response/hotels.json), response/home.json (response/home.json) |
| **Alur Eksekusi API & Workflows** | workflows/run_api_sequence.xml (workflows/run_api_sequence.xml) |

---

## Ringkasan Struktur Modul Backend

### 1. Entitas Akomodasi Ternormalisasi (21 Tabel)

- **Akar:** `accommodations` (`booking_mode`: `room` vs `unit`).
- **Unit & Tarif:** `unit_types`, `unit_type_prices`, `unit_type_images`.
- **Fasilitas:** `facility_groups`, `facility_items` (`context`: `facility`/`furniture`/`spec`), `facility_tags`.
- **Media & Ulasan:** `galleries`, `reviews`, `review_photos`.
- **Kebijakan & Q&A:** `accommodation_policies` (`general`/`check_in`/`deposit`), `accommodation_questions`, `nearby_points`.
- **Detail Ruangan:** `room_furniture_groups`, `room_furniture_items`, `bed_facilities`.
- **Detail Kompleks:** `accommodation_description_paragraphs`, `accommodation_details`, `accommodation_detail_groups`, `accommodation_detail_lines`.

### 2. Livewire 4 Multi-Step Wizard

- **Accommodation Wizard (Unified):** `App\Livewire\Accommodations\WizardForm` (`livewire.accommodations.wizard_form`).

### 3. API Resources Terpadu

- **`AccommodationResource`**: Mengembalikan payload `hotelShape()` untuk hotel dan `propertyShape()` untuk properti, 100% presisi dengan contract JSON Flutter.

<!-- Source: ai/00-orientation.md -->

# AI Orientation — Baca Ini Dulu, Selalu

> **Fungsi file ini.** Titik masuk tunggal untuk AI yang menerima prompt baru di repo ini. Tujuannya satu: **kamu tidak perlu membaca ulang seluruh `laravel/` dan `flutter/`.** Paket di `docs/ai/` sudah memuat hasil pembacaan itu.
>
> Alurnya: baca file ini → pilih pack dari tabel dispatch → buka **hanya** file kode yang disebut pack itu → kerjakan.

---

## 1. Ringkasan Proyek

**Livirooms** — marketplace penyewaan properti multi-tipe dan hotel OTA. Monorepo dengan dua deliverable:

| Bagian | Isi & Arsitektur |
|---|---|
| `laravel/` | Laravel 12. **Dua muka:** REST API (`api/v1`, Sanctum) untuk app mobile + Admin Panel (Metronic blade + Livewire 4 + Yajra DataTables) untuk admin & mitra. DB PostgreSQL dengan 21 model ternormalisasi (3NF). |
| `flutter/` | Satu app, **dua peran**: Konsumen (`MainShell`) & Mitra/Host (`MitraShell`). State: Provider. HTTP: package `http`. |

Stok akomodasi dikelola lewat entitas akar terpadu `accommodations` dengan diskriminator `booking_mode`:

- `'unit'`: Sewa unit utuh (apartemen, villa, kost). Kapasitas & harga diatur via 1 baris internal `unit_types` + `unit_type_prices`.
- `'room'`: Sewa kamar hotel per malam. Kapasitas, tipe kamar, & inventori diatur via multiple `unit_types`.

---

## 2. Tabel Dispatch — Prompt Seperti Apa → Baca Apa

| Prompt kamu tentang… | Baca | Jangan buka |
|---|---|---|
| **Properti, hotel, kamar, stok, ketersediaan, pencarian listing, harga** | 🟢 `docs/domain/accommodations.md` **wajib lebih dulu**, lalu `01-backend-map.md` §Domain | seluruh `app/Models/` |
| Endpoint API — ada/tidak, request/response, siapa yang memanggil | `03-api-contract.md` (tabel endpoint lengkap) | `routes/api_mobile.php` |
| Bug/fitur di sisi mobile — layar, navigasi, service | `02-frontend-map.md` | seluruh `lib/` |
| Booking → order → payment → webhook → deep link | `01-backend-map.md` §Pembayaran + `docs/payments/system.md` + `03-api-contract.md` §Alur bayar | — |
| Modul admin baru / DataTable / Livewire Wizard | `01-backend-map.md` §Admin Panel + `docs/configuration/admin-panel.md` + `docs/configuration/ui-patterns.md` | `routes/admin_generated.php` |
| Status booking/order/payment, transisi state | `01-backend-map.md` §State machine | migration enum |
| Permission, role, guard admin | `01-backend-map.md` §Auth & permission | — |
| "Kenapa docs bilang X tapi kode Y?" | `04-drift-register.md` | — |
| Referral & Reward | `docs/features/referral.md` | — |

---

## 3. Aturan Keras Saat Mengerjakan Repo Ini

1. **NO TESTING / NO MIGRATING:** Jangan jalankan test, `php artisan test`, atau migration/seeder. Dilarang di semua sesi di repo ini.
2. **NO TOUCHING FLUTTER:** Jangan ubah kode Flutter atau JSON fixture terlindungi saat mengerjakan tugas backend.
3. **SINKRONKAN DOCS (Zero-Omission):** Setiap perubahan kode di `laravel/` wajib diikuti update file docs terkait di `docs/`.
4. **Jangan edit `laravel/routes/admin_generated.php`.** File hasil scaffold.
5. **Jangan commit `laravel/.env`** — berisi kredensial DB nyata.

---

## 4. Peta Dokumen Lain

| File | Deskripsi |
|---|---|
| `docs/domain/hotel-vs-property.md` | ☑ **Master domain properti/hotel & 21 tabel 3NF.** |
| `docs/INDEX.md` | ☑ Indeks navigasi global |
| `docs/README.md` | ☑ Ringkasan sistem & stack |
| `docs/configuration/admin-panel.md` | ☑ Pola modul admin & Livewire Form Wizards |
| `docs/configuration/ui-patterns.md` | ☑ Template Blade & Livewire UI |
| `docs/features/payments.md` | ☑ Sistem pembayaran Midtrans |
| `docs/features/referral.md` | ☑ Sistem referral |
| `docs/payments/system.md` | ☑ Spesifikasi pembayaran, webhook, dan deep link |

<!-- Source: ai/01-backend-map.md -->

# Backend Map — Laravel 12 (Livirooms)

> **Fungsi file ini.** Peta lengkap `laravel/` untuk AI: layer, inventaris, domain, state machine, pembayaran, admin panel, job, permission. Dibaca sebagai ganti menjelajah `app/` secara buta.
>
> Untuk arsitektur data Hotel vs Property, dokumen master adalah: 🟢 **`docs/domain/accommodations.md`**.

---

## 1. Dua Muka, Satu Aplikasi

| Muka | Entry Route | Auth | Response Handler | |
|---|---|---|---|---|
| **REST API Mobile** | `routes/api_mobile.php`, prefix `api/v1` | `auth:sanctum` (Bearer token) | JSON amplop `ApiResponse` | |
| **Admin Panel** | `routes/web.php` → `routes/admin_generated.php`, prefix `admin` | session + `role:admin\` | host` | Blade (Metronic) + Livewire 4 + Yajra DataTables |
| **Website Publik** | `routes/web.php` → `Front/FrontController` | — | Blade | |
| **Callback Midtrans** | `routes/web.php` → `Payment/{Finish,Unfinish,Error}Controller` | — | redirect ke deep link `livirooms://` | |

`routes/api.php` **hanya** me-require `api_mobile.php`.

---

## 2. Stack Terpasang

```
laravel/framework          ^12.0
livewire/livewire          ^4.3      (Livewire 4)
laravel/sanctum            ^4.3
spatie/laravel-permission  ^6.9
yajra/laravel-datatables-* ^12.0
midtrans/midtrans-php      ^2.6
barryvdh/laravel-dompdf    *
```

Basis Data: **PostgreSQL** (`pgsql`) dengan 21 tabel ternormalisasi (3NF) untuk entitas akomodasi.

---

## 3. Inventaris Komponen Backend

| Direktori | Deskripsi & Isi |
|---|---|
| `app/Models/` | 21 model ternormalisasi produk (`Accommodation`, `UnitType`, `UnitTypePrice`, `FacilityItem`, `Gallery`, dll) + model auth, billing, cms |
| `app/Http/Resources/` | 17 resource produksi terpadu (`AccommodationResource`, `UnitTypeResource`, `FacilityItemResource`, dll) |
| `app/Http/Controllers/Api/` | 21 controller API mobile (`AccommodationController`, `PropertyController`, `HotelApiController`, `BookingController`, `PaymentController`, dll) |
| `app/Http/Controllers/Admin/` | 34 controller admin (thin, render index DataTable) |
| `app/Livewire/` | 31 folder komponen admin panel, termasuk `Properties/WizardForm` dan `Hotels/WizardForm` |
| `app/DataTables/` | 28 Yajra DataTables |
| `app/Support/` | `ApiResponse`, `MidtransSnap`, `MidtransGateway`, `PaymentDeepLink`, `HotelNormalizer` |
| `app/Jobs/` | `CancelExpiredBookingsJob`, `ExtendHotelInventoriesJob`, `IssueInsurancePolicyJob`, `SyncInsuranceStatusJob` |

---

## 4. Layer & Aliran Kontrol

```
API mobile:
  routes/api_mobile.php
    → app/Http/Controllers/Api/*Controller
      → Eloquent Model (21 Normalized Models)
      → app/Http/Resources/*Resource (e.g. AccommodationResource)
      → app/Support/ApiResponse::success() (JSON envelope)

Admin panel:
  routes/admin_generated.php & web.php
    → app/Http/Controllers/Admin/*Controller (index → DataTable)
    → app/Livewire/Properties/WizardForm & Hotels/WizardForm (Multi-Step Add/Edit)
      → Eloquent Model (DB::transaction)
```

---

## 5. Amplop API Standard (`ApiResponse`)

Setiap endpoint mengembalikan amplop terstandar:

```
{
  "status": true,
  "code": 200,
  "message": "Detail akomodasi berhasil dimuat.",
  "data": { ... },
  "meta": null,
  "errors": null
}
```

---

## 6. Domain Akomodasi: Single Root `accommodations`

Satu entitas akar `accommodations` dengan diskriminator `booking_mode`:

- **`'unit'` (Property):** Satu unit utuh (apartemen, villa, kost). Menggunakan 1 baris `unit_types` internal untuk kapasitas (`max_guests`, `bedrooms`, `bathrooms`) dan tarif (`unit_type_prices`).
- **`'room'` (Hotel):** Dijual per tipe kamar via multiple `unit_types` dan `unit_type_images`.

Transformasi output ditangani oleh **`AccommodationResource`**:

- Menghasilkan `hotelShape()` untuk hotel (persis `GET_hotels_id_new.json`).
- Menghasilkan `propertyShape()` untuk properti (persis `GET_properties_id_new.json`).

---

## 7. State Machine Booking & Payment

### `bookings.status` (9 Nilai)

`waiting_payment` → `waiting_confirmation` → `confirmed` → `paid` → `waiting_checkin` → `staying` → `completed` (atau `cancelled` / `expired`)

### `orders.status` (6 Nilai)

`pending` → `processing` → `confirmed` → `completed` (atau `cancelled` / `refunded`)

### `payments.status` (5 Nilai)

`pending` → `success` / `settlement` (atau `failed` / `expired` / `refunded`)

---

## 8. Pembayaran Midtrans Snap & Webhook

- **Midtrans S2S Webhook** adalah satu-satunya sumber perubahan status pembayaran ke `success`/`settlement`.
- Callback browser (`Payment/{Finish,Unfinish,Error}Controller`) **TIDAK BOLEH** mengubah status; hanya mengarahkan browser ke deep link `livirooms://payment-result`.
- Signature SHA512 dan verifikasi nominal `gross_amount` wajib diverifikasi sebelum status booking diperbarui.

---

## 9. Livewire Multi-Step Wizards

1. **`App\Livewire\Properties\WizardForm`:**
    - 4 tab: Info Dasar, Kapasitas & Kamar, Harga & Biaya, Galeri & Konten.
    - Mengelola relasi `accommodations` (`booking_mode='unit'`), `unit_types`, `unit_type_prices`, `facility_items` (`context='facility'`), dan `galleries`.
2. **`App\Livewire\Hotels\WizardForm`:**
    - 4 tab: Info Hotel & Bintang, Tipe Kamar (Repeater), Fasilitas & Aturan, Transit & FAQ.
    - Mengelola relasi `accommodations` (`booking_mode='room'`), multiple `unit_types`, `accommodation_policies` (`section='general'`), `nearby_points`, `accommodation_questions`, dan `galleries`.

<!-- Source: ai/02-frontend-map.md -->

# Frontend Map — Flutter (Livirooms)

> **Fungsi file ini.** Peta lengkap `flutter/` untuk AI: struktur, 47 route,
> 110 layar/widget, 20 service, 46 model, lapisan API, deep link, dan fixture mapping.
>
> **Catatan arsitektur:** `Property.php` dan `Hotel.php` telah dihapus. Semua relasi terkonsolidasi di `Accommodation.php` dengan diskriminator `booking_mode`.
>
> **Master JSON Contracts:** [`GET_hotels_id_new.json`](file:///d:/Project/Fastwork/rental.apartemen/flutter/assets/json/GET_hotels_id_new.json) & [`GET_properties_id_new.json`](file:///d:/Project/Fastwork/rental.apartemen/flutter/assets/json/GET_properties_id_new.json).
>
> **Backend model contract:** `App\Models\Accommodation` adalah single source of truth. `Property.php` dan `Hotel.php` telah dihapus. Semua relasi terkonsolidasi di `Accommodation.php` dengan diskriminator `booking_mode` (`room` = Hotel, `unit` = Property).

---

## 1. Bentuk Aplikasi

**Satu aplikasi Flutter, dua peran** yang dibedakan oleh shell:

| Peran | Shell | Entry Route |
|---|---|---|
| Konsumen | `lib/screens/main_shell.dart` | `Routes.home` |
| Mitra / Host | `lib/screens/mitra_shell.dart` | `Routes.mitraHome` |

### Stack Terpasang

- State management: **`provider ^6.1.2`**
- HTTP Client: **`http ^1.2.1`**
- WebView Snap: **`webview_flutter ^4.13.0`**
- Deep Link: **`app_links ^6.3.2`** (`livirooms://payment-result`)
- Formatting: **`intl ^0.20.3`** (`id_ID`)

---

## 2. Struktur `lib/`

```
lib/
├── main.dart              MaterialApp, MultiProvider, DeepLinkHandler, navigatorObservers [routeObserver]
├── router.dart            Routes (47 route) + AppRouter.onGenerateRoute + global routeObserver
├── config/                AppConfig (baseUrl, dummy toggle), theme
├── screens/               110 *_screen.dart + 2 shell (MainShell, MitraShell)
│   ├── mitra/             PropertyManagementScreen, AccommodationFormScreen (Unified Wizard), etc.
│   │   └── sheets/        11 modal bottom sheets (room_spec, bed_facility, price_scheme, operational_rule, checkin_checkout_note,
policy_item, etc.)
│   ├── home/              MainShell, FilterScreen, SearchResultsScreen
│   ├── property/          PropertyDetailScreen, RincianUnitScreen, Hotel screens
│   ├── booking/           BookingDetailScreen, BookingListScreen, etc.
│   ├── payment/           PaymentMethodScreen, PaymentResultScreen, etc.
│   ├── account/           AccountMenuScreen, EditProfileScreen, etc.
│   ├── auth/              LoginScreen, RegisterScreen, SplashScreen, etc.
│   ├── chat/              ChatListScreen, ChatRoomScreen
│   └── settings/          HelpSupportScreen, NotificationSettingsScreen, etc.
├── services/              20 service (Http, Auth, Property, Hotel, Payment, SearchLog, dll)
├── models/                46 model (Accommodation, UnitType, Booking, dll)
├── providers/             AuthProvider (registered), HotelSearchProvider
└── utils/                 Formatters, icons, promo helpers
```

---

## 3. Integrasi API & Dummy Mode

Aplikasi mendukung mode live API dan fallback fixture JSON terstandar di `assets/json/`:

- **Hotel Detail Screen (`/hotel/detail`):** Mengonsumsi `GET /api/v1/hotels/{id}` atau fixture `GET_hotels_id_new.json`.
- **Property Detail Screen (`/property/detail`):** Mengonsumsi `GET /api/v1/properties/{id}` atau fixture `GET_properties_id_new.json`.
- **Unified API:** Backend Laravel merespons kedua payload tersebut melalui `App\Http\Resources\AccommodationResource` yang 100% kompatibel tanpa perlu adaptasi di Flutter.

---

## 4. Accommodation Creation Flow (Flutter → Laravel)

### Alur Lengkap

```
Flutter AccommodationFormScreen
  ├── Mode Selection (room / unit)
  ├── Accommodation Wizard (7 tabs)
  │   ├── Tab 1: Info Dasar → name, property_type, address, city, building_list, status
  │   ├── Tab 2: Kapasitas & Kamar (unit) / Tipe Kamar (room) → capacity, bedrooms, bathrooms, room_types[]
  │   ├── Tab 3: Harga → base_price, price_original, prices[]
  │   ├── Tab 4: Fasilitas → facility_groups[], facility_tags[], bed_facilities[]
  │   ├── Tab 5: Konten → gallery[], intro, description_extra
  │   ├── Tab 6: Kebijakan → cancellation_policy, check_in/out_time, deposit_amount, policies[]
  │   ├── Tab 7: Lokasi & FAQ → latitude, longitude, map_preview, faqs[], nearby_points[]
  └── Submit → HostService.createProperty() / HotelService.createHotelForProperty()
      ├── POST host/properties (propertyPayload) — untuk unit mode
      └── POST host/hotels (hotelPayload) + POST host/hotels/{id}/room-types — untuk room mode
```

### Service Methods

| Flutter Service | Laravel Endpoint | Method |
|---|---|---|
| `HotelService.createHotelForProperty()` | `POST host/hotels` + `POST host/hotels/{id}/room-types` | Hotel creation |
| `HotelService.updateHotelForProperty()` | `PUT host/hotels/{id}` + `PUT/POST host/room-types/{id}` | Hotel update |
| `HotelService.getHostHotelDetail()` | `GET host/hotels/{id}` | Host-side detail |
| `HostService.createProperty()` | `POST host/properties` | Property creation |
| `HostService.updateProperty()` | `PUT host/properties/{id}` | Property update |
| `HostService.propertyDetail()` | `GET host/properties/{id}` | Property detail |

---

## 5. Model Mapping

| Flutter Model | Laravel Resource | Purpose |
|---|---|---|
| `Hotel` | `AccommodationResource` (hotelShape) | Hotel detail screen |
| `AccommodationHotel` | `AccommodationHotelListResource` | Hotel search/listing card |
| `Property` | `AccommodationResource` (propertyShape) | Property detail screen |
| `HotelRoomItem` | `UnitTypeResource` | Room type card |
| `HotelFacilityGroup` | `FacilityGroupResource` | Facility group |
| `HotelPolicy` | `AccommodationPolicyResource` | Policy block |
| `AccommodationFaqItem` | `AccommodationQuestionResource` | FAQ item |
| `NearbyPlace` | `NearbyPointResource` | Transport/nearby point |
| `Review` | `ReviewResource` | Guest review after completed booking |
| `ReviewPhoto` | `ReviewPhotoResource` | Photo attachment for guest review |
| `AccommodationDetailRoom` (JSON dummy) | `AccommodatonDetailRoomResource` | **Baru** — Detail & Booking mode `room` (`lib/screens/accommodation/accommodation_detail_room_screen.dart`) |
| `AccommodationDetailUnit` (JSON dummy) | `AccommodationDetailUnitResource` | **Baru** — Detail & Booking mode `unit` (`lib/screens/accommodation/accommodation_detail_unit_screen.dart`) |

> **Flutter dummy assets (struktur `policies` per section):** `assets/json/accommodation_room_id.json` (mode `room`) & `assets/json/accommodation_unit_id.json` (mode `unit`). Kedua file mereplikasi persis output API Resource, dengan blok `policies` dikelompokkan berdasarkan `section` (`general`, `check_in`, `check_out`, `cancellation`, `smoking_policy`, `extra_bed_policy`, `deposit`, `room_policy`).

> **Seeder aman (chunking):** `AccommodationSeeder` memproses seluruh pekerjaan seed dalam chunk eksplisit (maksimal 100 chunk) dengan jeda `sleep(2)` di akhir setiap chunk untuk mencegah beban puncak CPU/memori saat `migrate:fresh --seed`.

---

## 6. Lifecycle & Navigation Auto-Refresh

- **`BookingListScreen` (`/bookings` & Tab 1 MainShell):**
  - Mengimplementasikan `RouteAware` dengan mendaftarkan diri ke `routeObserver` di `didChangeDependencies()`. Ketika user kembali dari halaman detail booking atau pembayaran (`didPopNext()`), data booking otomatis di-fetch ulang (`_load()`).
  - Mendukung `didUpdateWidget(oldWidget)` dengan parameter `isActive` yang di-pass dari `MainShell` (`isActive: _index == 1`). Pergantian tab di dalam `IndexedStack` otomatis memicu reload tanpa perlu me-recreate state.
  - Menggunakan `WidgetsBindingObserver` (`didChangeAppLifecycleState`) untuk mendeteksi status `AppLifecycleState.resumed` saat user kembali ke aplikasi setelah menyelesaikan pembayaran di luar aplikasi (misalnya browser Midtrans Snap / e-wallet).
  - Pada `_BookingTab._openDetail()`, `Navigator.pushNamed(...)` di-`await`, dan pemanggilan `onRefresh()` langsung dieksekusi begitu navigasi ditutup (`pop`).
  - Mengawasi perubahan user pada `AuthProvider` di dalam `didChangeDependencies()` agar data otomatis ter-refresh saat status autentikasi berubah.

---

## 7. Booking Status Display & Action Button Invariants

Penyajian status pemesanan dan tombol aksi mengikuti aturan:

- **`completed` (Pesanan Selesai):** Menampilkan penjelasan eksplisit di layar bahwa masa sewa/inap telah selesai (`Pesanan Selesai`, ikon centang hijau `AppColors.success`). Tombol **Bayar** dan **Batalkan Pemesanan** tidak ditampilkan.
- **`cancelled` & `expired` (Dibatalkan / Kedaluwarsa):** Menampilkan penjelasan status pembatalan atau kedaluwarsa dengan warna `AppColors.danger`. Tombol **Bayar** dan **Batalkan Pemesanan** tidak ditampilkan sama sekali.
- **`waiting_payment` (Menunggu Pembayaran):** Menampilkan countdown timer serta tombol aktif **Bayar** (dan tombol **Batalkan Pemesanan** jika diizinkan kebijakan pembatalan).
- **Layar & Komponen Terkait:**
  - `lib/screens/booking/booking_detail_screen.dart` (Property Booking Detail)
  - `lib/screens/booking/booking_detail_hotel_screen.dart` (Hotel Booking Detail)
  - `lib/screens/booking/widgets/booking_widgets_bottom_bar.dart`
  - `lib/screens/booking/widgets/booking_widgets_segment_status.dart`

---

## 8. Authentication & Role Routing (Mitra vs Consumer)

- **Routing Shell:**
  - `User` dengan status host (`auth.isHost == true`) otomatis diarahkan ke `Routes.mitraHome` (`MitraShell`) baik saat selesai login di `LoginScreen` maupun saat auto-login di `SplashScreen`.
  - `User` dengan status konsumen biasa diarahkan ke `Routes.home` (`MainShell`).
- **Resolusi Status Host (`User.isHost`):**
  - **Flutter (`User.isHost`):** Mengevaluasi `type == 'host'`, array `roles.contains('host')`, serta fallback pola email host (`.host@` / `host@` seperti `budi.host@example.com`).
  - **Laravel (`AuthController` & `ProfileController`):** Memvalidasi `$user->hasRole('host')`, kepemilikan properti/hotel (`Accommodation::where('host_id', $user->id)->exists()`), atau pola email host. Otomatis memperbarui `UserProfile.type = 'host'` dan menyertakan `roles` dalam response JSON payload `auth/login`, `auth/me`, dan `profile`.

<!-- Source: ai/03-api-contract.md -->

# API Contract — Jembatan Laravel ↔ Flutter

> **Fungsi file ini.** Satu tabel untuk seluruh endpoint `api/v1`:
> method · path · `Controller::method` · service Dart pemanggil · fixture dummy.
>
> **Status:** Tersinkronisasi dengan 21 model ternormalisasi dan `AccommodationResource`.

---

## 1. Aturan Dasar

- Prefix penuh: `{host}/api/v1/...`. Sisi Dart mengirim path **tanpa** `api/v1` dan tanpa leading slash.
- Amplop response: `ApiResponse` standar (`status`, `code`, `message`, `data`, `meta`, `errors`).
- Kontrak Output: `AccommodationResource` secara otomatis memetakan `hotelShape()` atau `propertyShape()` sesuai discriminator `booking_mode`.

---

## 2. Publik — 23 Endpoint (Tanpa Sanctum)

| # | Method + Path | Controller::method | Dart Service | Dummy Fixture | Keterangan |
|---|---|---|---|---|---|
| 1 | `POST auth/register` | `AuthController::register` | `auth_service.dart:22` | ☑ | Registrasi user |

| | | | | | |
|---|---|---|---|---|---|
| 2 | `POST auth/login` | `AuthController::login` | `auth_service.dart:36` | ☑ | Login user & return token |
| 3 | `GET remote-config` | `RemoteConfigApiController::fetch` | `remote_config_service.dart:35` | ☑ | Parameter remote config |
| 4 | `GET remote-config/{key}` | `RemoteConfigApiController::fetchOne` | `remote_config_service.dart:55` | ☑ | Nilai tunggal key |
| 5 | `GET home` | `HomeController::index` | `home_service.dart:35` | ☑ | Banners + recommended listings (menerima `?city_id={id}` untuk memfilter banner & rekomendasi) |
| 6 | `GET cities` | `CityController::index` | `city_service.dart:18` | ☑ | Daftar kota |
| 7 | `GET cities/{city}` | `CityController::show` | `city_service.dart:23` | ☑ | Detail kota |
| 8 | `GET properties` | `PropertyController::index` | `property_service.dart:24` | ☑ | Pencarian & filter properti |
| 9 | `GET properties/{property}` | `PropertyController::show` | `property_service.dart:39` | ☑ | Detail properti (`propertyShape`) |
| 10 | `GET properties/{property}/pricing` | `PropertyController::pricing` | `property_service.dart:46` | ☑ | Kalkulator harga sewa |
| 11 | `GET properties/{property}/reviews` | `ReviewController::index` | `property_service.dart:55` | ☑ | Ulasan properti |
| 12 | `GET properties/{property}/availability` | `PropertyController::availability` | `booking_service.dart:49` | ☑ | Kalender ketersediaan unit |
| 13 | `GET properties/{property}/host` | `PropertyController::host` | `property_service.dart:79` | ☑ | Profil host/mitra |
| 14 | `GET insurance/plans` | `InsuranceApiController::plans` | `hotel_service.dart:92` | ☑ | Pilihan asuransi |
| 15 | `GET vouchers` | `VoucherController::index` | `voucher_service.dart:41` | ☑ | Kupon promo aktif |
| 16 | `GET hotels` | `HotelApiController::index` | `hotel_service.dart:20` | ☑ | Pencarian & filter hotel OTA |
| 17 | `GET hotels/{hotel}` | `HotelApiController::show` | `hotel_service.dart:26` | ☑ | Detail hotel 8-tab (`hotelShape`) |
| 18 | `GET hotels/{hotel}/room-types` | `HotelApiController::roomTypes` | — | ☐ | Allotment tipe kamar |
| 19 | **`GET accommodations/{id}`** | **`AccommodationController::show`** | `property_service.dart:62` | ☑ | **Unified endpoint (Auto-branching Hotel & Property)** |
| 19a | **`GET accommodations/{id}/edit`** | **`AccommodationController::edit`** | — | ☑ | **Form prefill — mengembalikan detail dengan `policies` terkelompok per section** |
| 19b | **`POST accommodations`** | **`AccommodationController::store`** | — | ☑ | **Buat akomodasi (race-condition safe + `DB::transaction`)** |
| 19c | **`PUT accommodations/{id}`** | **`AccommodationController::update`** | — | ☑ | **Update akomodasi (`lockForUpdate` + `DB::transaction`)** |
| 19d | **`GET accommodations/{property}/pricing`** | **`PropertyController::pricing`** | `property_service.dart:66` | ☑ | **Kalkulator harga sewa akomodasi / unit (UnitTypeResource & UnitTypePriceResource typed)** |
| 19e | **`GET accommodations/{property}/reviews`** | **`ReviewController::index`** | `property_service.dart:77` | ☑ | **Ulasan akomodasi** |
| 19f | **`GET accommodations/{property}/availability`** | **`PropertyController::availability`** | `property_service.dart:86` | ☑ | **Kalender ketersediaan akomodasi** |
| 19g | **`GET accommodations/{property}/host`** | **`PropertyController::host`** | `property_service.dart:101` | ☑ | **Profil host akomodasi** |

**Resource baru (detail per `booking_mode`):**

- `App\Http\Resources\AccommodatonDetailRoomResource` — mode `room` (Hotel). Output identik `hotelShape` + blok `policies` terstruktur per `section` (`general`, `check_in`, `check_out`, `cancellation`, `smoking_policy`, `extra_bed_policy`, `deposit`, `room_policy`).
- `App\Http\Resources\AccommodationDetailUnitResource` — mode `unit` (Apartemen/Vila/Kost). Output identik `propertyShape` + blok `policies` terstruktur per `section`.

**Kontrak `policies` (stabil, zero-error):** selalu `Map<section, List<{section, icon, tone, title, paragraphs[]}>>`; setiap section selalu hadir sebagai array (bisa kosong). JSON dummy Flutter: `assets/json/accommodation_room_id.json` & `accommodation_unit_id.json`.

| 20 | `GET cms/{slug}` | `HomeController::cmsPage` | `content_service.dart:100` | ☑ | Privacy Policy, T&C, About |
|---|---|---|---|---|---|
| 21 | `GET faqs` | `HomeController::faqs` | `content_service.dart:85` | ☑ | FAQ Publik |
| 22 | `POST payments/webhook` | `PaymentController::webhook` | — | 🔓 | **Midtrans Server-to-Server Webhook** |
| 23 | **`POST search-logs`** | **`SearchLogController::store`** | **`search_log_service.dart`** | ☐ | **Analytics pencarian — fire-and-forget dari Flutter** |

---

## 3. Terautentikasi — 63 Endpoint (`auth:sanctum`)

### Auth & Profil

- `POST auth/logout` (`AuthController::logout`)
- `GET auth/me` (`AuthController::me`)
- `GET profile` (`ProfileController::show`) — Mengambil detail profil user (`User` & `UserProfile`)
- `PUT profile` / `POST profile` (`ProfileController::update`) — Memperbarui data pengguna (nama, telepon, alamat, gender, tanggal lahir, dll.)
- `POST profile/avatar` (`ProfileController::uploadAvatar`) — Upload foto profil user (`multipart/form-data`: `avatar`, validasi image/max 5MB, disimpan ke storage public `avatars/`, mengembalikan `avatar` & `avatar_url`)
- `POST profile/become-host` (`ProfileController::becomeHost`) — Mengubah user menjadi host (validasi identitas wajib terpenuhi)
- `GET user/validation` (`UserValidationController::index`)
- `POST user/validation` (`UserValidationController::store`)

### Chat & Percakapan

- `GET conversations` (`ChatController::index`) — `chat_service.dart:10` — Daftar percakapan user (mengembalikan `type` ('booking' | 'accommodation'),

accommodation_id, accommodation:id,name, booking_id, booking:id,booking_code,status,check_in,check_out, other_user, last_message, last_message_at)

- POST conversations (ChatController::store) — chat_service.dart:15 — Buka percakapan baru dengan recipient_id, accommodation_id, dan booking_id (auto-resolve host/guest jika dibuka dari booking atau listing akomodasi)
- GET conversations/{conversation}/messages (ChatController::messages) — chat_service.dart:23 — Riwayat pesan dalam percakapan (paginated)
- POST conversations/{conversation}/messages (ChatController::sendMessage) — chat_service.dart:30 — Kirim pesan baru dalam percakapan *Catatan Relasi Model `Conversation`:* Menggunakan relasi accommodation() (belongsTo(Accommodation::class)) dan booking() (belongsTo(Booking::class)). Kolom booking_id ditambahkan melalui migrasi 2026_09_05_000004_add_booking_id_to_conversations_table.php. Relasi inverse conversations() terdaftar pada model Accommodation dan Booking.

### Booking & Checkout

- GET bookings (BookingController::index)
- POST booking-property (BookingController::storeProperty) — Checkout sewa unit utuh
- POST booking-hotels (BookingController::storeHotel) — Checkout sewa per kamar hotel
- POST bookings (Legacy) (BookingController::store) — Dispatched by booking_type
- GET bookings/{booking} (BookingController::show)
- POST bookings/{booking}/cancel (BookingController::cancel)
- POST bookings/{booking}/confirm (BookingController::confirm)
- POST bookings/{booking}/reject (BookingController::reject)
- POST bookings/{booking}/checkin (BookingController::checkin) — Check-in tamu (update status ke staying)
- POST bookings/{booking}/checkout (BookingController::checkout) — Check-out tamu (update status ke completed & order completed)
- POST bookings/{booking}/confirm-checkin (BookingController::confirmCheckin)
- POST bookings/{booking}/review (ReviewController::store) — **Buat ulasan hanya untuk booking completed**. Setelah ulasan dibuat, AccommodationScoreComponent dengan component = 'basic' otomatis diperbarui dengan rata-rata rating baru. Event model Review juga menangani update/delete untuk menjaga sinkronisasi.
- GET properties/{property}/reviews (ReviewController::index) — Daftar ulasan properti

### Orders & Payment Snap

- GET orders (OrderController::index) — booking_service.dart:219 — User's order list (nested booking)
- GET orders/{order} (OrderController::show) — booking_service.dart:224 — Order detail
- POST orders/{order}/pay (PaymentController::pay) — payment_service.dart — Create Midtrans Snap token & redirect URL
- POST orders/{order}/pay/retry (PaymentController::payUrl) — payment_service.dart — Retry or continue payment
- GET payments/check/{order} (PaymentController::check) — payment_service.dart — Poll payment status
- GET payments/methods (PaymentController::methods) — payment_service.dart — Enabled payment methods
- GET payments/{payment} (PaymentController::show) — payment_service.dart — Payment detail
- POST payments/{payment}/reconcile (PaymentController::reconcile) — payment_service.dart — Sync status with Midtrans
- POST bookings/{booking}/cancel (BookingController::cancel) — booking_service.dart — Cancel booking
- POST bookings/{booking}/review (ReviewController::store) — booking_service.dart — Create review
- POST payments/webhook (PaymentController::webhook) — Midtrans S2S webhook (signature SHA-512 verified, not Sanctum)

### Host Management (Mitra)

- GET host/accommodations (PropertyController::hostAccommodations) — host_service.dart:18 — **FIXED & AUDITED:** Unified listing of host accommodations (both unit and room/hotel modes) via AccommodationResource. Returns status, host_id, numeric deposit_amount, and supports page, per_page, status, booking_mode, and q search without filtering out unpriced/draft listings.
- POST host/accommodations (PropertyController::store) — host_service.dart:23 — Buat listing akomodasi mitra baru (Mendukung intro, description_extra_heading, description_extra array paragraf, otomatis membuat relasi accommodation_description_paragraphs)
- GET host/accommodations/{property} (PropertyController::hostShow) — host_service.dart:116 — Detail akomodasi host (Eager-load descriptionParagraphs + fallback kolom JSONB)
- PUT host/accommodations/{property} (PropertyController::update) — host_service.dart:28 — Update akomodasi host (Sinkronisasi penuh intro, description_extra_heading, dan description_extra)
- DELETE host/accommodations/{property} (PropertyController::destroy) — host_service.dart:121 — Hapus akomodasi host
- GET host/orders (OrderController::forHost) — host_service.dart:33 — **FIXED:** Orders with nested booking object; query supports page, status, booking_status. Returns Paginated<Order>.
- GET host/orders/{order} (OrderController::showForHost) — host_service.dart:41 — Host order detail
- POST bookings/{booking}/confirm (BookingController::confirm) — host_service.dart:46 — Confirm booking (Accept order)
- POST bookings/{booking}/reject (BookingController::reject) — host_service.dart:50 — Reject booking (Tolak order)
- POST bookings/{booking}/checkin (BookingController::checkin) — host_service.dart:checkin — Check-in tamu (status -> staying)
- POST bookings/{booking}/checkout (BookingController::checkout) — host_service.dart:checkout — Check-out tamu (status -> completed)
- POST bookings/{booking}/confirm-checkin (BookingController::confirmCheckin) — host_service.dart:confirmCheckin — Confirm check-in
- GET host/properties (PropertyController::hostIndex)
- POST host/properties (PropertyController::store)
- GET host/properties/{property} (PropertyController::hostShow)
- PUT host/properties/{property} (PropertyController::update)
- DELETE host/properties/{property} (PropertyController::destroy)
- GET host/hotels (HostHotelController::index) — hotel_service.dart — Daftar hotel mitra
- POST host/hotels (HostHotelController::store) — hotel_service.dart:createHotelForProperty — Registrasi hotel + facility_groups + policies + faqs + transport_points + gallery + intro, description_extra_heading, description_extra
- GET host/hotels/{hotel} (HostHotelController::show) — hotel_service.dart:getHostHotelDetail — Detail hotel mitra (hotelShape sekarang menyertakan intro, description_extra_heading, dan description_extra)
- PUT host/hotels/{hotel} (HostHotelController::update) — hotel_service.dart:updateHotelForProperty — Update metadata hotel + room types + intro, description_extra_heading, description_extra
- DELETE host/hotels/{hotel} (HostHotelController::destroy) — hotel_service.dart:deleteHotel — Hapus hotel
- POST host/hotels/{hotel}/room-types (HostHotelController::storeRoomType) — hotel_service.dart (via createHotelForProperty) — Tambah tipe kamar
- PUT host/room-types/{roomType} (HostHotelController::updateRoomType) — hotel_service.dart — Edit tipe kamar
- DELETE host/room-types/{roomType} (HostHotelController::destroyRoomType) — hotel_service.dart:deleteRoomType — Hapus tipe kamar
- GET host/wallet (HostWalletController::index)
- POST host/wallet/withdraw (HostWalletController::withdraw)
- GET host/unit-type-prices/{id} (UnitTypePriceController::show) — host_service.dart — Detail data tarif UnitTypePrice
- PUT host/unit-type-prices/{id} (UnitTypePriceController::update) — host_service.dart:updateUnitTypePrice — **Strictly Update-Only (No Create):** Quick update ketersediaan & tarif (amount, cleaning_fee, deposit, service_fee, price_type, status). 404 jika data tidak ditemukan, 422 jika validasi gagal, sinkronisasi otomatis unit_types.base_price.

- `PUT unit-types/{unitType}/prices/{price}` (`UnitTypePriceController::updateForUnitType`) — Nested route update tarif UnitTypePrice.

<!-- Source: ai/04-drift-register.md -->

# Drift Register — Docs & Klaim vs Kode

> **Fungsi file ini.** Daftar status selisih terverifikasi, perbaikan arsitektur data modeling, dan catatan integrasi sistem.
>
> **Status per 2026-08-26:** Flutter→Laravel accommodation creation flow audit selesai. Drift kritis ditemukan dan direkonsiliasi.
> **Update per 2026-08-27:** Relasi `Accommodation::bookings()` ditambahkan untuk menghilangkan `BadMethodCallException` di `ReportController::propertyPerformanceReport()`.
> **Update per 2026-08-27:** `Property.php` dan `Hotel.php` dihapus. Semua relasi terkonsolidasi di `Accommodation.php`. `HotelFaq` diubah menjadi `AccommodationFaq` (tabel `accommodation_faqs`). `HotelScoreComponent` diubah menjadi `AccommodationScoreComponent` (tabel `accommodation_score_components`). `AccommodationReviewQuote` dihapus; review quotes tidak lagi disimpan secara terpisah.
> **Update per 2026-08-27:** Resource audit selesai. `PresentsProperty` trait dibersihkan dari referensi `PropertyOther` yang sudah dihapus. `AccommodationQuestionResource::toPropertyArray()` diselaraskan dengan Flutter `HotelFaqItem`. `NearbyPointResource::toPropertyArray()` diselaraskan dengan Flutter `TransportPoint`.
> **Update per 2026-08-27:** `Accommodation::updateScoreComponents()` ditambahkan untuk men-sinkronkan rata-rata rating review ke tabel `accommodation_score_components` (component `basic`). Event `created`, `updated` (hanya jika `rating` berubah), dan `deleted` pada model `Review` memicu penyaluran otomatis. `ReviewController::store()` juga memanggilnya secara eksplisit sebagai safety net.
> **Update per 2026-08-27:** Audit `Accommodations\WizardForm` + blade selesai. Accessor rekursif `getCheckInTimeAttribute()`/`getCheckOutTimeAttribute()` dihapus. Validasi `prices` kini kondisional via `required_if:booking_mode,unit`. `has_breakfast` dipindah ke `unit_types`, tidak lagi disimpan ke tabel `accommodations`. Default transit icon diubah ke `map-pin`. Semua 23 test Livewire wizard lolos.
> **Update per 2026-08-30:** `AndroidManifest.xml` diperbaiki: referensi `@mipmap/launcher_icon` diganti `@mipmap/ic_launcher` dan meta-data Firebase `com.google.firebase.messaging.default_notification_icon` (menunjuk ke `@drawable/logo_launcher` yang tidak ada) dihapus karena Firebase tidak digunakan. Build debug Android kini berhasil link resource.
> **Update per 2026-08-29:** WizardForm icon inputs diganti dari free-text menjadi `<select>` dropdown dengan opsi Flutter-mapped yang valid. `HotelNormalizer` diintegrasikan ke pipeline `save()` untuk sanitasi `facility_groups`, `transport_points`, `faqs`, `questions`, dan `room_images`. Autocomplete suggestions disinkronkan dari master data `AccommodationSeeder` dan model `FacilityItem`/`NearbyPoint`. Dropdown `booking_mode` ditambahkan di Tab 1.
> **Update per 2026-09-03:** `HotelApiController::getFilterMetadata` diubah dari `->pluck('star_rating')` menjadi `->get()->pluck('star_rating')` karena `star_rating` bukan kolom fisik di tabel `accommodations`, melainkan accessor komputasi yang dihasilkan dari `selectRaw`. PostgreSQL gagal menguraikan alias `star_rating` pada query builder `pluck` tanpa `get()` terlebih dahulu.
> **Update per 2026-09-04:** Sinkronisasi endpoint rincian harga sewa (`GET accommodations/{id}/pricing` dan `properties/{id}/pricing`). `PropertyController::pricing` diperbarui dengan `UnitTypeResource` & `UnitTypePriceResource` untuk memastikan seluruh nilai nominal bertipe integer murni (menghindari type cast exception pada Dart). Model Dart `UnitPricing` & child models dilengkapi parser angka defensif `_toNum`/`_toInt`. Resolver rental period pada `RincianUnitScreen` diselaraskan agar memilih periode valid yang tersedia di database.
> **Update per 2026-09-04:** Perbaikan order management mitra. Tiga fixture dummy (`GET_host_orders.json`, `GET_orders.json`, `GET_host_orders_id.json`) merepresentasikan objek `Booking` yang di-flatten ke level atas, bukan objek `Order` dengan nested `booking`. Akibatnya: `Order.fromJson` menghasilkan `bookingStatus = null`, sehingga order tidak muncul di tab manapun ("Baru"/"Dikonfirmasi"/"Selesai"). Fixture juga tidak memiliki field Order (`order_code`, `booking_id`, `subtotal`, `discount`, `total`) dan `meta` pakai `page` instead of `current_page`. Ketiga fixture sudah diperbaiki untuk mencerminkan format API nyata (Order dengan nested `booking`). `Order.fromJson` juga dibuat defensif — pakai `Formatters.asInt` untuk `id`/`bookingId` dan `Formatters.asString` untuk `status`/`bookingStatus`, menghindari `TypeError` bila backend mengembalikan nilai integer sebagai string.

---

## 1. Status Resolusi Arsitektur (Selesai Direkonsiliasi)

| Topik | Isu Lama | Status & Solusi Baru |
|---|---|---|
| **Data Modeling Hotel vs Property** | Tabel paralel tidak konsisten & JSON blob `display_meta` tak terstruktur | ☑ **Terselesaikan:** 21 tabel ternormalisasi 3NF dengan single root `accommodations` & `booking_mode`. |
| **Harga & Kapasitas** | Duplikasi harga di properti dan room type | ☑ **Terselesaikan:** Single source of truth pada `unit_types` + `unit_type_prices`. Relasi `cheapestUnitType()` (`HasOne::ofMany`). |
| **Transformasi Output API** | Resource monolitik campur aduk | ☑ **Terselesaikan:** `AccommodationResource` auto-branching `hotelShape()` & `propertyShape()`, 100% cocok dengan master fixture JSON. |
| **Admin Form Wizards** | Form Livewire belum mendukung relasi ternormalisasi | ☑ **Terselesaikan:** `Properties\WizardForm` & `Hotels\WizardForm` mendukung 4-step wizard transaksional (`DB::transaction`). |
| **Flutter→Laravel Creation Drift** | Flutter `AccommodationFormScreen` mengirim 15+ field (facility_groups, policies, faqs, transport_points, gallery, star_rating) tapi Laravel `HostHotelController::store()` hanya menerima 11 field dasar | ☑ **Terselesaikan:** Controller `store()` kini menerima dan memproses semua field yang dikirim Flutter form. |
| **Order Management Fixture Drift** | `GET_host_orders.json` / `GET_orders.json` / `GET_host_orders_id.json` berisi objek `Booking` yang di-flatten, bukan `Order` dengan nested `booking`; `GET_orders.json` berisi data Property. Akibat: order tidak muncul di tab manapun | ☑ **Terselaiperbaiki:** Ketiga fixture diselaraskan ke format API nyata (Order + nested `booking`). `Order.fromJson` dibuat defensif dengan `Formatters.asInt`/`asString`. Debug `print` di `api_client.dart _uri()` dihapus. |

---

## 2. Flutter→Laravel Accommodation Creation Flow

**Alur Lengkap (Flutter Form → Laravel API)**

```
Flutter AccommodationFormScreen
  ├── _submitHotel()
  │    ├── hotelPayload → POST host/hotels
  │    │    ├── name, star_rating, open_year, check_in_time, check_out_time
  │    │    ├── description, address, latitude, longitude
  │    │    ├── facility_groups[] → facility_groups table
  │    │    ├── policies{} → accommodation_policies table
  │    │    ├── deposit_amount, cancellation_policy
  │    │    ├── faqs[], transport_points[]
  │    │    └── gallery[] → galleries table
  │    └── roomTypesPayload[] → POST host/hotels/{id}/room-types
  │         ├── name, description, capacity_adults, capacity_children
  │         ├── max_guests, room_size_sqm, bed_configuration
  │         ├── base_price, price_original, breakfast_included
  │         ├── has_window, no_smoking, free_wifi, total_inventory
  │         └── facilities[], images[]
  └── _submitUnit()
       └── buildPropertyPayload() → POST host/properties
```

**Field Mapping: Flutter → Laravel → DB**

| Flutter Field | Laravel Controller | DB Table | Status |
|---|---|---|---|
| name, address, description | `store()` validated | `accommodations` | ☑ |
| star_rating | `store()` validated | computed from reviews | ☑ |
| open_year, check_in_time, check_out_time | `store()` validated | `accommodations` | ☑ |
| cancellation_policy | `store()` validated | `accommodation_policies` | ☑ |
| deposit_amount | `store()` validated | `accommodations` | ☑ |
| facility_groups | `store()` processed | `facility_groups` + `facility_items` | ☑ |
| policies (child, extra_bed, smoking) | `store()` processed | `accommodation_policies` | ☑ |
| faqs | `store()` processed | `accommodation_questions` | ☑ |
| transport_points | `store()` processed | `nearby_points` | ☑ |
| gallery | `store()` processed | `galleries` | ☑ |
| room_types[] | `storeRoomType()` | `unit_types` | ☑ |
| room_type.prices | via unit type | `unit_type_prices` | ☑ |

**Perbandingan dengan Seeder**

| Entitas | Seeder | Flutter Form | Catatan |
|---|---|---|---|
| Accommodation | ☑ Full fields | ☑ Full fields | Konsisten |
| UnitType (room) | ☑ 2 types (Merlynn) | ☑ Dynamic | Flutter dinamis |
| UnitTypePrice | ☑ daily price | Via room type | Konsisten |
| Gallery | ☑ 5 photos | ☑ Dynamic upload | Konsisten |
| Policy | ☑ cancellation + check_in | ☑ cancellation + general | Konsisten |
| FacilityGroup | ☑ 1 group + 6 items | ☑ Dynamic | Konsisten |
| FacilityTag | ☑ 4 tags | Via facility_tags | Konsisten |
| NearbyPoint | ☑ 3 points | ☑ Dynamic | Konsisten |
| Question (FAQ) | ☑ 2 FAQs | ☑ Dynamic | Konsisten |
| Review | ☑ 1 review + photo | ✗ Not in form | Reviews come from guests |

---

## 3. Catatan Integrasi Aktif

### 1. Model `Room` (Fisik)

Stok kamar hotel sepenuhnya menggunakan counter `unit_types.remaining` (real-time) dan bukan penomoran kamar fisik. Tabel `room_inventories` (daily calendar) sudah dihapus di refactoring terakhir.

### 2. Push Notification & Maps

- Parameter `AppConfig.googleMapsApiKey` dan endpoint token `POST devices/fcm-token` tersedia di backend untuk ekspansi masa depan tanpa memblokir runtime Flutter.

### 3. Known Resource Issues (Fixed)

- `AccommodationResource`: N+1 risk pada `question_count` → fixed dengan `whenLoaded()`
- `AccommodationHotelListResource`: null-cast pada `rating_label` → fixed
- `AccommodationListResource`: N+1 risk pada `city`, `rating` accessors → fixed

<!-- Source: ai/05-wizard-form-ui.md -->

# Wizard Form UI — PMS 7-Step Architecture & Input Upgrades

> **Last updated:** 2026-08-29
> **Scope:** `app/Livewire/Accommodations/WizardForm.php` + `resources/views/livewire/accommodations/wizard_form.blade.php`

---

## 1. Blueprint 7-Step PMS Flow

Alur wizard diselaraskan penuh dengan standar industri PMS (*Property Management System*), menciptakan keselarasan logika antara mode **Hotel** (`booking_mode = 'room'`) dan **Properti Sewa Unit** (`booking_mode = 'unit'`):

| Step | Tab Name | Mode Room (Hotel) | Mode Unit (Apartemen/Villa/Kost) |
|---|---|---|---|
| Tab 1 | **1. Identitas & Lokasi** | Host, Tipe Akomodasi, Jenis Properti, Nama Hotel, Gedung, Kota, Alamat, Lat/Long, Status, Star Rating, Tahun Buka. | Host, Tipe Akomodasi, Jenis Properti, Nama Properti, Gedung/Tower, Kota, Alamat, Lat/Long, Status, Tahun Buka. |
| Tab 2 | **2. Inventaris & Ruang** | Manajemen Tipe Kamar (`room_types`): Nama, Bed, Luas m², Tamu Dws/Anak, Stok Kamar, Fitur Kamar, Foto Kamar, Tag Fasilitas Kamar. | Spesifikasi Ruang (Tamu, Bed/Bath, Luas m²), Fasilitas Kasur (`bed_facilities`), Furnitur per Ruangan (`room_furniture_groups`). |
| Tab 3 | **3. Skema Harga** | Matrix Tarif Kamar: Tarif per malam, Harga coret diskon, Termasuk sarapan (`has_breakfast`), stok allotment. | Skema Sewa Multi-Durasi (`prices`: daily, weekly, monthly, yearly), Biaya Kebersihan, Deposit, Service Fee, Harga Coret, Caption. |
| Tab 4 | **4. Fasilitas Properti** | Grup & Item Fasilitas Hotel (`facility_groups` + `facility_items` context=`facility`) dengan badge & status ketersediaan. | Fasilitas Gedung/Kompleks (`facilities`) & Spesifikasi Properti Tambahan (`specs`). |
| Tab 5 | **5. Media & Cerita** | Foto Cover & Galeri Foto (`galleries`), Intro Singkat, Deskripsi Lengkap, Paragraf Sorotan, Rekomendasi & Flash Sale. | Foto Cover & Galeri Foto (`galleries`), Intro Singkat, Deskripsi Lengkap, Paragraf Sorotan, Rekomendasi & Flash Sale. |
| Tab 6 | **6. Aturan & Kebijakan** | Jam Check-in/out, Resepsionis 24 Jam & Jam Operasional, Usia Min, Aturan Anak/Extra Bed/Rokok, Daftar Kebijakan Pembatalan Dinamis (`cancellation_policies`), Catatan Check-in & Deposit. | Jam Check-in/out, Deposit Properti, Daftar Kebijakan Pembatalan Dinamis (`cancellation_policies`), Catatan Check-in & Deposit. |
| Tab 7 | **7. Sekitar & FAQ** | Preview Peta (`map_preview`), Titik POI (`nearby_points`), Titik Transit (`transport_points`), Tanya Jawab Pengelola (`faqs`). | Preview Peta (`map_preview`), Titik POI (`nearby_points`), Tanya Jawab (`questions`), Detail Properti Tambahan (`unit_detail`). |

---

## 2. Icon Rendering System

Semua icon picker dan rendering status menggunakan **Material Design SVG paths** murni (1:1 dengan Flutter) — tanpa dependensi icon font Metronic `ki-outline`:

### Blade Components

| Component | Path | Purpose |
|---|---|---|
| `flutter-icon` | `resources/views/components/flutter-icon.blade.php` | Server-side inline SVG untuk badge & sel tabel |
| `flutter-icon-registry` | `resources/views/components/flutter-icon-registry.blade.php` | Script global penyuplai `window._flutterIconPaths` untuk Alpine.js |
| `icon-picker` | `resources/views/components/icon-picker.blade.php` | Dropdown Alpine.js dengan preview SVG interaktif terikat Livewire |

### Penggunaan

```
{{-- Static icon rendering --}}
<x-flutter-icon name="pool" size="16" />

{{-- Interactive icon picker bound to Livewire property --}}
<x-icon-picker
    wire="new_facility_group_icon"
    :icons="['star' => 'Star', 'pool' => 'Kolam Renang']"
/>
```

---

## 3. Suggestion Pools & Autocomplete Reaktif

Semua input teks pendukung dilengkapi dropdown autocomplete reaktif berbasis Alpine.js (`wire:model.live` + getter `get filtered()`):

| Pool Property | Sumber Data | Digunakan Pada |
|---|---|---|
| `$facilitySuggestions` | `FacilityItem` (context=facility) DB | Tab 2/4 Autocomplete Nama Fasilitas |
| `$specSuggestions` | `FacilityItem` (context=spec) DB | Tab 4 Autocomplete Spesifikasi |
| `$nearbySuggestions` | `NearbyPoint` DB | Tab 7 Autocomplete Nama Lokasi POI |
| `$buildingListSuggestions` | `BuildingList` DB | Tab 1 Autocomplete Nama Gedung/Tower |
| `$roomNameSuggestions` | Presets Preset | Tab 2 Autocomplete Nama Tipe Kamar |
| `$bedSuggestions` | Presets Preset | Tab 2 Autocomplete Konfigurasi Bed |
| `$facilityCategorySuggestions` | Presets Preset | Tab 4 Autocomplete Kategori Fasilitas |
| `$nearbyTypeSuggestions` | Presets Preset | Tab 7 Autocomplete Tipe Lokasi Sekitar |
| `$nearbyDistanceSuggestions` | Presets Preset | Tab 7 Autocomplete Jarak |
| `$nearbyDurationSuggestions` | Presets Preset | Tab 7 Autocomplete Durasi |
| `$badgeSuggestions` | Presets Preset | Tab 4 Shortcut Badge Fasilitas |
| `$roomPolicyPresets` | Presets Preset | Tab 6 Shortcut & Presets Kebijakan Kamar |
| `$cancellationPolicyPresets` | Presets Preset | Tab 6 Shortcut Kebijakan Pembatalan |
| `$faqQuestionSuggestions` | Presets Preset | Tab 7 Autocomplete Pertanyaan FAQ |
| `$detailSuggestions` | Presets Preset | Tab 7 Autocomplete Label Detail |

---

## 4. Model `AccommodationPolicy` Multi-Section Architecture

Seluruh kebijakan properti dan kamar disimpan secara modular pada tabel `accommodation_policies`:

- `section = 'general'` — Pengaturan umum: jam operasional resepsionis, usia min tamu, boolean flags (resepsionis 24 jam, allow children, extra bed).
- `section = 'room_policy'` — Multi-item dinamis kebijakan kamar hotel (icon, tone, title, paragraphs/text).
- `section = 'cancellation'` — Butir-butir kebijakan pembatalan dinamis.
- `section = 'check_in'` — Catatan instruksi check-in khusus (icon, tone, title, paragraphs/text).
- `section = 'deposit'` — Catatan uang jaminan / deposit.

---

### 5. Single Root DOM & State Invariants

1. **Single Root Element:** File `wizard_form.blade.php` diawali dengan modal root tunggal `<div class="modal fade" ...>` dan `<x-flutter-icon-registry />` diletakkan di dalam root untuk mencegah kerusakan morphing Livewire.
2. **Navigasi Dinamis:** Method `setTab(n)`, `nextTab()`, dan `prevTab()` mengelola state `$currentTab` dengan aman.
3. **Validasi Kondisional:** Method `rules()` secara cerdas memvalidasi `guest_capacity`, `bedrooms`, dan `prices` hanya saat `booking_mode = 'unit'`, serta memvalidasi `room_types` hanya saat `booking_mode = 'room'`.

<!-- Source: domain/hotel-vs-property.md -->

# Master Context — Accommodations (Hotel vs Property)

> **PENGALIHAN DOKUMEN (DOCUMENT REDIRECT):**
> Dokumen ini telah diperbarui dan digantikan oleh dokumen master yang lebih komprehensif:
>
> 👉 **[docs/domain/accommodations.md](file:///d:/Project/Fastwork/rental.apartemen/docs/domain/accommodations.md)**
>
> Silakan rujuk ke `accommodations.md` untuk referensi lengkap mengenai:
>
> - Skema 21 tabel 3NF PostgreSQL & Model Eloquent.
> - Mekanika Single Source of Truth (SSoT) `unit_types` & `cheapestUnitType()`.
> - Transformasi auto-branching `AccommodationResource` (Hotel vs Property).
> - Komponen Livewire 4 Multi-Step Form Wizards.
> - Integrasi Flutter Mobile (Model Dart, Layar UI, Provider, Midtrans Snap Flow & Deep Link).
> - 9 Invarian utama sistem.

<!-- Source: domain/accommodations.md -->

# Master Context — Accommodations: Backend & Frontend Architecture (Laravel 12 & Flutter)

> **Fungsi file ini.** Master reference komprehensif arsitektur data PostgreSQL, Laravel 12 Backend, API Resources, dan integrasi Flutter Frontend untuk seluruh entitas akomodasi (Hotel & Property) di ekosistem **Livirooms**.
>
> **Status:** 100% konsisten dengan skema 3NF PostgreSQL (21 tabel), Laravel 12, dan payload kontrak master [`GET_hotels_id.json`](file:///d:/Project/Fastwork/rental.apartemen/flutter/assets/json/GET_hotels_id.json) serta [`GET_properties_id.json`](file:///d:/Project/Fastwork/rental.apartemen/flutter/assets/json/GET_properties_id.json).

---

# BAGIAN I: ARSITEKTUR BACKEND (LARAVEL 12)

### 1. Arsitektur Terpadu: Satu Entitas Akar `accommodations`

Hotel dan Property disatukan dalam satu entitas akar `accommodations` dengan diskriminator `booking_mode`:

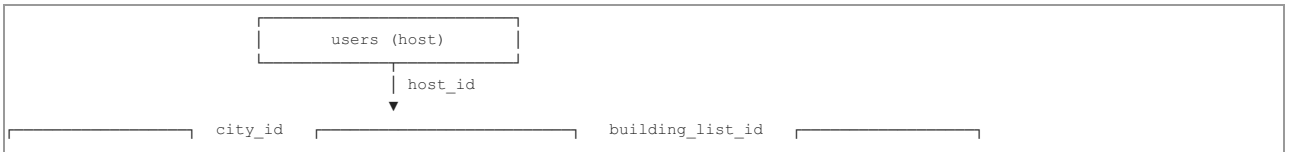
- **`booking_mode = 'room'` (Hotel OTA):**
  - Dijual per tipe kamar (`unit_types`).
  - Mendukung banyak tipe kamar (mis. Deluxe, Executive Suite, Family Suite).
  - Stok dikelola per allotment harian (`total_inventory` / `remaining`).
  - Harga listing (`price_from`) dan kapasitas diturunkan otomatis dari kamar termurah aktif melalui relasi `cheapestUnitType()`.
- **`booking_mode = 'unit'` (Property Rental):**
  - Dijual per unit utuh (Apartemen, Villa, Rumah, Kost Eksklusif).
  - Memiliki tepat **1 baris `unit_types`** internal yang dibuat otomatis sebagai Single Source of Truth (SSoT) spesifikasi fisik (`max_guests`, `bedrooms`, `bathrooms`, `room_size_sqm`) dan harga multi-durasi (`unit_type_prices`: harian, mingguan, bulanan, tahunan).

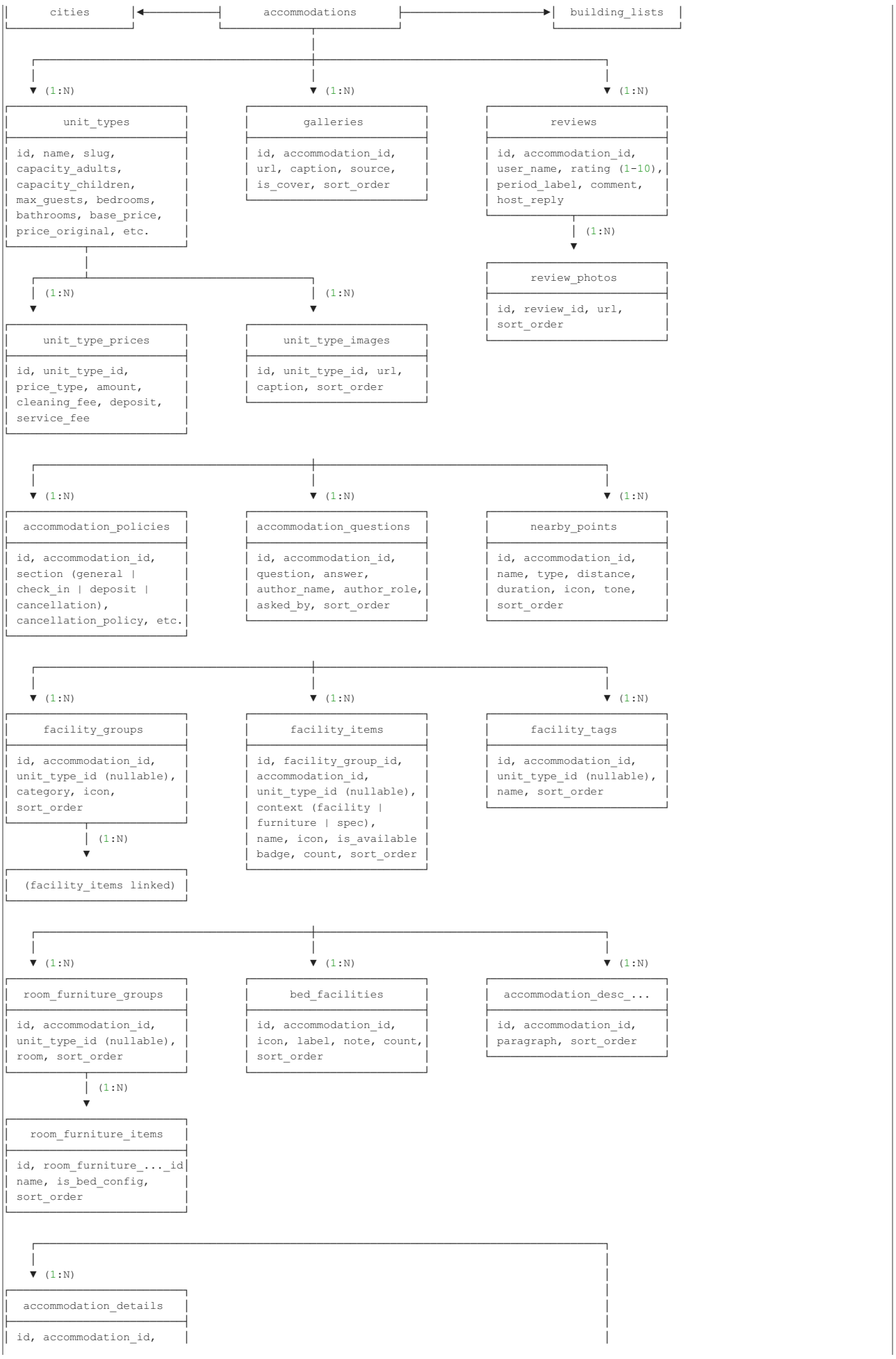
**Tiga Pilar Arsitektur 3NF:**

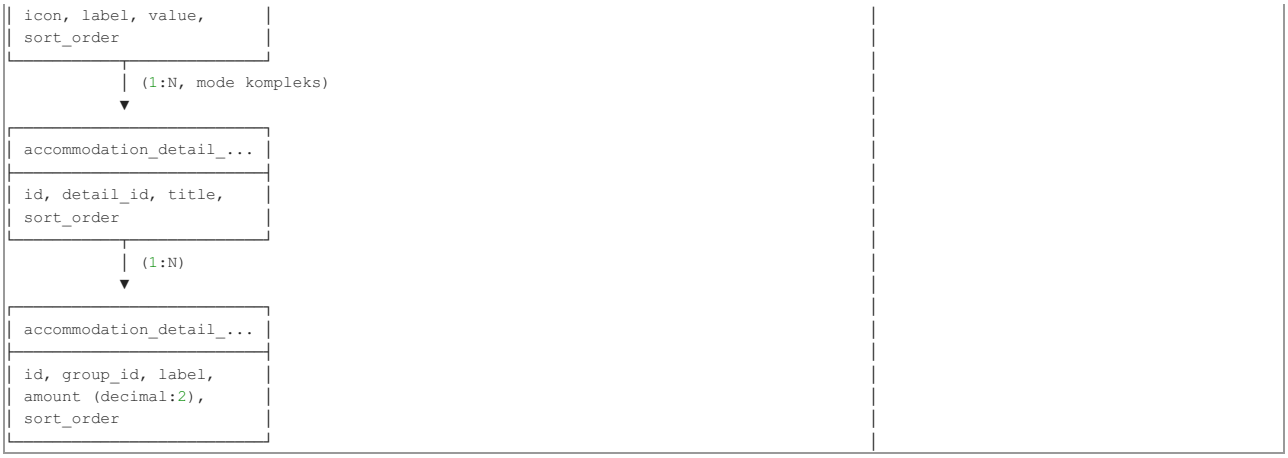
1. **Zero JSON Blob:** Semua data tampilan terstruktur ke kolom skema asli (`intro`, `map_preview`, `price_caption`, `is_flash_sale`), accessor turunan (`type_label`, `area_label`, `extra_media_count`, `cancellation_policy`), atau tabel relasional satelit (`accommodation_description_paragraphs`, `accommodation_details`).
2. **Single Source of Truth (SSoT) untuk Harga & Kapasitas:** Tidak ada duplikasi kolom `guest_capacity`, `bedrooms`, `bathrooms`, `has_breakfast`, `price_from`, `price_original` di tabel `accommodations`. Semua dihitung on-the-fly dari `unit_types` via `cheapestUnitType()` (`HasOne::ofMany`).
3. **Penyatuan Satelit Multi-Konteks:** Tabel `facility_items` melayani 3 kebutuhan (`context = 'facility' | 'furniture' | 'spec'`) untuk efisiensi skema tanpa redundansi struktur.

---

### 2. Diagram Relasi Lengkap 21 Tabel Produk (PostgreSQL)




cities
accommodations
building_lists
(1:N)
unit_types
id, name, slug, capacity_adults, capacity_children, max_guests, bedrooms, bathrooms, base_price, price_original, etc.
galleries
id, accommodation_id, url, caption, source, is_cover, sort_order
reviews
id, accommodation_id, user_name, rating (1-10), period_label, comment, host_reply
review_photos
id, review_id, url, sort_order
unit_type_prices
id, unit_type_id, price_type, amount, cleaning_fee, deposit, service_fee
unit_type_images
id, unit_type_id, url, caption, sort_order
accommodation_policies
id, accommodation_id, section (general | check_in | deposit | cancellation), cancellation_policy, etc.
accommodation_questions
id, accommodation_id, question, answer, author_name, author_role, asked_by, sort_order
nearby_points
id, accommodation_id, name, type, distance, duration, icon, tone, sort_order
facility_groups
id, accommodation_id, unit_type_id (nullable), category, icon, sort_order
(facility_items linked)
facility_items
id, facility_group_id, accommodation_id, unit_type_id (nullable), context (facility | furniture | spec), name, icon, is_available badge, count, sort_order
facility_tags
id, accommodation_id, unit_type_id (nullable), name, sort_order
room_furniture_groups
id, accommodation_id, unit_type_id (nullable), room, sort_order
room_furniture_items
id, room_furniture_..._id name, is_bed_config, sort_order
bed_facilities
id, accommodation_id, icon, label, note, count, sort_order
accommodation_desc_...
id, accommodation_id, paragraph, sort_order
accommodation_details
id, accommodation_id,




## 3. Rincian 21 Model Eloquent & Tabel Database

| # | Model Eloquent | Tabel Database | Peran Arsi |
|---|---|---|---|
| 1 | [Accommodation](file:///d:/Project/Fastwork/rental.apartemen/laravel/app/Models/Accommodation.php) | accommodations | Root Entity (Ho Property). Men identitas inti, k jam check-in/o terhitung. |
| 2 | [BuildingList](file:///d:/Project/Fastwork/rental.apartemen/laravel/app/Models/BuildingList.php) | building_lists | Master tower/g apartemen (building_li building_un |
| 3 | [UnitType](file:///d:/Project/Fastwork/rental.apartemen/laravel/app/Models/UnitType.php) | unit_types | **Single Source** harga, kapasita (max_guests, bedrooms, ba base_price). |
| 4 | [UnitTypePrice](file:///d:/Project/Fastwork/rental.apartemen/laravel/app/Models/UnitTypePrice.php) | unit_type_prices | Harga periodik (daily, weekl monthly, yea deposit & clea serta status op (active, inac sold_out, maintenance |
| 5 | [UnitTypeImage](file:///d:/Project/Fastwork/rental.apartemen/laravel/app/Models/UnitTypeImage.php) | unit_type_images | Foto galeri spe kamar hotel. |
| 6 | [FacilityGroup](file:///d:/Project/Fastwork/rental.apartemen/laravel/app/Models/FacilityGroup.php) | facility_groups | Kategori fasilit bertingkat ber- (Hotel). |
| 7 | [FacilityItem](file:///d:/Project/Fastwork/rental.apartemen/laravel/app/Models/FacilityItem.php) | facility_items | Item fasilitas n konteks (`conte facility |
| 8 | [FacilityTag](file:///d:/Project/Fastwork/rental.apartemen/laravel/app/Models/FacilityTag.php) | facility_tags | Tag/chip fasilit ringkas pada k listing. |
| 9 | [Gallery](file:///d:/Project/Fastwork/rental.apartemen/laravel/app/Models/Gallery.php) | galleries | Media galeri u (source = ga dan foto tamu = guest_med |
| 10 | [Review](file:///d:/Project/Fastwork/rental.apartemen/laravel/app/Models/Review.php) | reviews | Ulasan seragai 1.0 - 10.0 deng rating agregat otomatis. |
| 11 | [ReviewPhoto](file:///d:/Project/Fastwork/rental.apartemen/laravel/app/Models/ReviewPhoto.php) | review_photos | Foto lampiran ulasan dari tar |
| 12 | [AccommodationPolicy] (file:///d:/Project/Fastwork/rental.apartemen/laravel/app/Models/AccommodationPolicy.php) | accommodation_policies | Kebijakan ters general, che deposit, dan cancellatio |
| 13 | [AccommodationQuestion] (file:///d:/Project/Fastwork/rental.apartemen/laravel/app/Models/AccommodationQuestion.php) | accommodation_questions | Tanya jawab manajemen h (author_role FAQ properti (asked_by). |
| 14 | [NearbyPoint](file:///d:/Project/Fastwork/rental.apartemen/laravel/app/Models/NearbyPoint.php) | nearby_points | POI & transit te (stasiun, halte, bandara, mall, durasi). |
| 15 | [RoomFurnitureGroup] (file:///d:/Project/Fastwork/rental.apartemen/laravel/app/Models/RoomFurnitureGroup.php) | room_furniture_groups | Grup ruangan (mis. Master B Living Room). |

| | | | |
|---|---|---|---|
| 16 | [RoomFurnitureItem](file:///d:/Project/Fastwork/rental.apartemen/laravel/app/Models/RoomFurnitureItem.php) | `room_furniture_items` | Item perabot + `is_bed_conf` konfigurasi ran |
| 17 | [BedFacility](file:///d:/Project/Fastwork/rental.apartemen/laravel/app/Models/BedFacility.php) | `bed_facilities` | Kartu ringkasa (King Bed, Sin ukuran & jumla |
| 18 | [AccommodationDescriptionParagraph](file:///d:/Project/Fastwork/rental.apartemen/laravel/app/Models/AccommodationDescriptionParagraph.php) | `accommodation_description_paragraphs` | Paragraf long- tambahan (`descriptior` |
| 19 | [AccommodationDetail](file:///d:/Project/Fastwork/rental.apartemen/laravel/app/Models/AccommodationDetail.php) | `accommodation_details` | Rincian detail (mode sederha `value` atau ko `groups`). |
| 20 | [AccommodationDetailGroup](file:///d:/Project/Fastwork/rental.apartemen/laravel/app/Models/AccommodationDetailGroup.php) | `accommodation_detail_groups` | Sub-grup deta kompleks (mis. Mobil", "Parkir |
| 21 | [AccommodationDetailLine](file:///d:/Project/Fastwork/rental.apartemen/laravel/app/Models/AccommodationDetailLine.php) | `accommodation_detail_lines` | Baris tarif dala grup (label + a |

### Relasi Transaksional Tambahan

`Accommodation` juga memiliki relasi `bookings()` ke model [`Booking`](file:///d:/Project/Fastwork/rental.apartemen/laravel/app/Models/Booking.php) (tabel `bookings`) melalui foreign key legacy `property_id`. Relasi ini digunakan untuk laporan admin seperti `ReportController::propertyPerformanceReport()`.

---

## 4. API Resources & Eager Loading Contract (Zero N+1)

Transformasi data dilakukan oleh [`AccommodationResource`](file:///d:/Project/Fastwork/rental.apartemen/laravel/app/Http/Resources/AccommodationResource.php) yang secara otomatis mencabangkan output JSON sesuai `booking_mode`:

- **`booking_mode === 'room'`**: memanggil `hotelShape()` → menghasilkan JSON identik dengan [`GET_hotels_id.json`](file:///d:/Project/Fastwork/rental.apartemen/flutter/assets/json/GET_hotels_id.json).
- **`booking_mode === 'unit'`**: memanggil `propertyShape()` → menghasilkan JSON identik dengan [`GET_properties_id.json`](file:///d:/Project/Fastwork/rental.apartemen/flutter/assets/json/GET_properties_id.json).

### Kontrak Eager Loading Wajib di Controller:

```php
// Hotel (booking_mode = 'room')
Accommodation::hotels()->with([
    'city', 'buildingList', 'galleries',
    'policies', 'questions', 'nearbyPoints',
    'facilityGroups.items', 'cheapestUnitType',
    'unitTypes.images', 'unitTypes.facilityItems',
])->findOrFail($id);

// Property (booking_mode = 'unit')
Accommodation::properties()->with([
    'city', 'buildingList', 'host', 'galleries',
    'policies', 'questions', 'nearbyPoints',
    'facilityItems', 'facilityTags', 'bedFacilities',
    'roomFurnitureGroups.items', 'roomFurnitureGroups.bedConfig',
    'reviews.photos',
    'cheapestUnitType.prices',
    'descriptionParagraphs', 'details.groups.lines',
])->findOrFail($id);
```

---

## 5. Livewire 4 Multi-Step Form Wizards (Admin Panel)

1. **[`App\Livewire\Accommodations\WizardForm`](file:///d:/Project/Fastwork/rental.apartemen/laravel/app/Livewire/Accommodations/WizardForm.php) (Unified 7-Step PMS Form):**
    - **Tab 1 (Identitas & Lokasi):** Host, Mode (`room`/`unit`), Jenis Properti, Nama, Gedung/Tower (autocomplete), Kota, Alamat, Lat/Long, Status, Star Rating, Tahun Buka.
    - **Tab 2 (Inventaris & Ruang):**
        - *Mode Room*: Manajemen Tipe Kamar (`room_types`): Nama, Bed, Luas m², Stok Allotment, Fitur Kamar, Foto Kamar, Tag Fasilitas Kamar.
        - *Mode Unit:* Spesifikasi Fisik (Tamu, Bed/Bath, Luas m²), Fasilitas Tempat Tidur (`bed_facilities`), Furnitur per Ruangan (`room_furniture_groups`).
    - **Tab 3 (Skema Harga):**
        - *Mode Room*: Matrix Tarif Kamar (Tarif dasar/malam, harga coret, switch sarapan).
        - *Mode Unit:* Skema Sewa Multi-Durasi (`prices`: daily, weekly, monthly, yearly), Biaya Kebersihan, Deposit, Service Fee, Harga Coret, Caption.
    - **Tab 4 (Fasilitas Properti):**
        - *Mode Room*: Grup & Item Fasilitas Hotel (`facility_groups` + `facility_items` context=`facility`).
        - *Mode Unit:* Fasilitas Gedung/Kompleks (`facilities`) & Spesifikasi Properti Tambahan (`specs`).
    - **Tab 5 (Media & Cerita):** Foto Cover & Galeri Utama (`galleries`), Intro Singkat, Deskripsi Lengkap, Paragraf Sorotan, Rekomendasi Utama & Flash Sale.
    - **Tab 6 (Aturan & Kebijakan):** Jam Check-in/out, Ketentuan Resepsionis & Usia Min, Kebijakan Anak/Extra Bed/Rokok, Daftar Kebijakan Pembatalan Dinamis (`cancellation_policies`), Catatan Check-in & Deposit.
    - **Tab 7 (Sekitar & FAQ):** Preview Peta (`map_preview`), Titik Lokasi Sekitar POI (`nearby_points`), Titik Transit (`transport_points`), Tanya Jawab FAQ (`accommodation_questions`), Detail Properti Tambahan (`accommodation_details`).
    - **Persistence:** Menyimpan seluruh relasi 3NF PostgreSQL dalam satu `DB::transaction()` atomik.

2. **`App\Livewire\Hotels\WizardForm`**(file:///d:/Project/Fastwork/rental.apartemen/laravel/app/Livewire/Hotels/WizardForm.php):
    - Wizard khusus hotel (Mode Room).
3. **`App\Livewire\Properties\WizardForm`**(file:///d:/Project/Fastwork/rental.apartemen/laravel/app/Livewire/Properties/WizardForm.php):
    - Wizard khusus rental properti (Mode Unit).

---

# BAGIAN II: INTEGRASI FLUTTER FRONTEND

## 1. Aliran Konsumsi Data & Model Dart

Aplikasi Flutter Livirooms mengonsumsi data akomodasi melalui REST API dengan arsitektur **Provider** dan package `http`:

```
Backend API (JSON Envelope)
  ├── GET /api/v1/hotels/{id}        ──► HotelDetailResponse.fromJson() ──► Hotel Model
  ├── GET /api/v1/properties/{id}    ──► PropertyDetailResponse.fromJson() ──► PropertyDetail Model
  └── GET /api/v1/accommodations/{id}──► Auto-detected via booking_mode
```

### Pemetaan Field Payload Master ke Model Dart:

| Field JSON Payload | Tipe Data | Model Dart Target | Komponen UI Flutter |
|---|---|---|---|
| `id` / `property_id` | `int` | `id` | Primary Key & Identitas Navigasi |
| `name` | `String` | `name` | Judul AppBar & Header Layar |
| `booking_mode` | `String` | `bookingMode` | Pengarah Rendering UI (`'room'` vs `'unit'`) |
| `rating` | `double` | `rating` | Badge Skor Rating (Skala 1.0 - 10.0) |
| `rating_label` | `String` | `ratingLabel` | Label Kualitatif ("Istimewa", "Sangat Baik") |
| `review_count` | `int` | `reviewCount` | Teks Jumlah Ulasan "(428 ulasan)" |
| `price_from` | `int` | `priceFrom` | Teks Harga Kartu "Mulai dari Rp 685.000 /malam" |
| `price_original` | `int?` | `priceOriginal` | Teks Harga Coret Diskon |
| `gallery` | `List<String>` | `gallery` | `CarouselSlider` / PageView Header Foto |
| `cover_image` | `String?` | `coverImage` | Thumbnail Listing & Hero Animation |
| `building_list` / `building_units` | `Map` | `BuildingList` | Kartu Info Kawasan / Gedung Hunian |
| `facility_tags` | `List<String>` | `facilityTags` | `Wrap` Chip Fasilitas Ringkas |
| `facilities` | `List<FacilityGroup>` | `facilities` | Grid / List Fasilitas Hotel Berkelompok |
| `policy_notes` | `Map` | `PolicyNotes` | Accordion / Card Aturan Check-in & Jam Resepsionis |
| `cancellation_policy` | `String?` | `cancellationPolicy` | Badge / Accordion Kebijakan Pembatalan |
| `questions` | `List<Question>` | `questions` | Accordion FAQ Tanya Jawab Manajer |
| `nearby_points` | `List<NearbyPoint>` | `nearbyPoints` | List Akses Transportasi & POI Terdekat |
| `unit_types` | `List<RoomType>` | `roomTypes` | List Kartu Pilihan Kamar Hotel (Pemesanan Kamar) |
| `specs` | `List<PropertySpec>` | `specs` | Grid Spesifikasi Properti (Luas, Kamar, Furnished) |
| `room_furniture_groups` | `List<FurnitureGroup>` | `furnitureGroups` | List Detail Perabot & Bed Config per Ruangan |
| `unit_detail` | `List<UnitDetail>` | `unitDetails` | List Rincian Biaya & Spesifikasi Unit Lengkap |
| `reviews` | `List<Review>` | `reviews` | List Card Ulasan Tamu + Galeri Foto Tamu |

---

## 2. Navigasi & Tampilan Layar (Screens)

### 1. Layar Hotel Detail (`HotelDetailScreen`)

- **Header Foto:** Carousel foto galeri hotel dengan counter indeks dan tombol favorit/share.
- **Header Info:** Nama hotel, bintang (1-5 bintang), rating badge (1-10), lokasi kota.
- **Blok Pilihan Kamar (`unit_types`):**
    - Render kartu per tipe kamar (Deluxe, Suite, dll).
    - Menampilkan foto kamar (`images`), konfigurasi ranjang (`bed_configuration`), luas m², sisa stok (`remaining`), sarapan (`has_breakfast`), dan harga dasar (`base_price`).
    - Tombol "Pilih Kamar" mengarahkan ke form pemilihan jumlah kamar & checkout.
- **Blok Fasilitas (`facilities`):** Tab / Grid kategori fasilitas (Fasilitas Utama, Restoran, Spa, Transportasi).
- **Blok Akses & Lokasi (`nearby_points`):** Menampilkan stasiun, halte, bandara dengan durasi dan jarak.
- **Blok Kebijakan & FAQ (`policy_notes, questions`):** Jam check-in/out, deposit jaminan, dan FAQ manajer.

### 2. Layar Property Detail (`PropertyDetailScreen`)

- **Header Banner & Info:** Cover image, nama unit/apartemen, tipe hunian, harga sewa per bulan/hari.
- **Spesifikasi Cepat (`specs`):** Ikon kapasitas tamu, jumlah kamar tidur, kamar mandi, dan luas unit.
- **Konfigurasi Ranjang (`bed_facilities`):** Kartu visual jenis dan ukuran kasur per kamar.
- **Perabot & Ruangan (`room_furniture_groups`):** Rincian perlengkapan Master Bedroom, Dapur, Living Room.
- **Rincian Biaya Unit (`unit_detail`):** Detail biaya sewa, biaya parkir motor/mobil bulanan, biaya kartu akses.
- **Kebijakan Pembatalan & Catatan Check-in (`check_in_notes`, `cancellation_policy`).
- **Bottom Bar Sewa:** Menampilkan pilihan opsi sewa (Harian / Bulanan / Tahunan) dan tombol "Sewa Sekarang" yang mengarahkan ke `RincianUnitScreen`.

### 2.1 Alur Layar Checkout: Hotel vs Property

1. **Hotel OTA (`booking_mode = 'room'`):**
    - Layar: `HotelBookingScreen` (`flutter/lib/screens/property/hotel/hotel_booking_screen.dart`).
    - Endpoint: `POST /api/v1/booking-hotels`.
    - Payload: Menerima array `guests[]` (dengan `room_number`, `room_name`, `first_name`, `last_name`), `rooms_count`, `adults`, `children`, `check_in`, `check_out`, `contact_email`, `contact_phone`.
    - Voucher: Diambil dari `showPromoCodeSheet`, diverifikasi via `VoucherService`.

2. **Property Rental (`booking_mode = 'unit'`):**
    - Layar: `RincianUnitScreen` (`flutter/lib/screens/property/rincian_unit_screen.dart`).
    - Endpoint: `POST /api/v1/booking-property`.
    - Payload: `accommodation_id`, `check_in`, `check_out`, `duration_type` (`daily` | `monthly` | `yearly`), `adults` (minimal 1, default dari `bedrooms` atau 1 untuk tipe studio), `add_on_ids`, `plan_id` (opsional untuk cicilan tahunan), `promo_code` (opsional), `payable_amount`.
    - Validasi Voucher: Diverifikasi dinamis melalui `VoucherController::resolveDiscount`. Jika voucher tidak valid / kedaluwarsa, kode otomatis dibatalkan dari state dan pesan spesifik ditampilkan ke pengguna.
    - Pilihan Tanggal: Mendukung pemilihan rentang tanggal stay harian (`showDateRangePicker`) dan tanggal mulai bulanan/tahunan (`showDatePicker`) dengan proteksi tanggal lampau (`after_or_equal:today`).
    - State Tombol: `BrandButton` mendukung `isLoading` dengan `_isSubmitting` guard untuk mencegah duplikasi order.

---

### 3. Alur Transaksi & Pembayaran (Midtrans Snap Flow)

```
[Flutter App] ── 1. POST /api/v1/bookings ──► [Laravel Backend]
                                                      │
                                              2. Create Booking (status: pending)
                                              3. Request Snap Token ke Midtrans API
                                                      │
[Flutter App] ◄── 4. Return snap_token & redirect_url ┘
     │
5. Buka Midtrans Snap WebView
     │
6. Tamu Selesaikan Pembayaran di Payment Gateway
     │
     ├──► 7. Midtrans Server-to-Server (S2S) Webhook ──► [Laravel Webhook Handler]
     │                                                          │
     │                                                  8. DB Transaction:
     │                                                     - Mutasi Status Booking -> 'confirmed'
     │                                                     - Catat Settlement di payments
     │                                                     - Kirim Notifikasi / Push Notification
     │
     └──► 9. Redirect Browser / WebView ke Deep Link:
             livirooms://payment-result?order_id={id}&status={status}
                     │
             10. Ditangkap oleh Flutter DeepLinkHandler
                     │
             11. Arahkan ke Layar BookingSuccessScreen / BookingFailedScreen
```

---

### 4. Sembilan Invarian Utama Sistem (System Invariants)

1. **`booking_mode` adalah satu-satunya diskriminator stok.** `'room'` untuk multi-room hotel OTA, `'unit'` untuk single-unit rental properti.
2. **`unit_types` adalah satu-satunya sumber kebenaran (SSoT) harga & kapasitas.** Tidak ada kolom kapasitas atau tarif sewa di tabel `accommodations`.
3. **Seluruh field antarmuka memiliki entitas relasional yang valid.** Dilarang menambahkan kolom JSON blob generik (`display_meta`).
4. **Relasi `cheapestUnitType()` wajib di-eager-load.** Accessor `price_from`, `bedrooms`, `bathrooms`, `has_breakfast` bergantung pada relasi ini.
5. **Kebijakan pembatalan dikelola di `accommodation_policies`.** Disimpan pada baris dengan `section = 'cancellation'`.
6. **Satu-satunya sumber perubahan status pembayaran adalah Midtrans S2S Webhook.** Callback browser atau deep link frontend tidak boleh mengubah status pembayaran.
7. **Skala rating seragam 1.0 - 10.0.** Seluruh rating dihitung dari rata-rata tabel `reviews`.
8. **Eager loading bebas N+1 wajib diterapkan pada seluruh endpoint API.**
9. **Dokumentasi `docs/` wajib selalu sinkron dengan implementasi kode aktif.**

<!-- Source: domain/property-hotel-form-sync.md -->

# Sinkronisasi Form Wizard Properti & Hotel — Flutter ↔ Laravel Livewire

Dokumen ini mendokumentasikan implementasi terpadu dari sistem pendaftaran dan pengelolaan akomodasi Mitra (Host) di Livirooms, mencakup frontend Flutter (7 tab wizard) dan backend Laravel (Livewire WizardForm & API `host/accommodations`).

---

### 1. Arsitektur Single Source of Truth (SSoT)

Sistem akomodasi Livirooms membedakan dua mode pemesanan (`booking_mode`):

1. **`unit` (Unit Stay)**: Penyewaan seluruh unit (Apartemen, Rumah Tapak, Villa, Kost Eksklusif). Disewakan secara harian, mingguan, bulanan, atau tahunan.
2. **`room` (Room Stay / Hotel OTA)**: Penyewaan per kamar dengan inventaris multi-room (Hotel, Guest House, Hostel, Hotel Residence). Disewakan per malam per tipe kamar.

#### Hubungan Entitas Database:

- **`accommodations`**: Root entity yang menyimpan data identitas, lokasi, kota, koordinat, rating bintang, status, dan field tambahan dalam kolom JSON `other`.
- **`unit_types`**: SSoT untuk spesifikasi fisik unit/kamar (kapasitas tamu, kamar tidur, kamar mandi, luas meter persegi, bed configuration, base pricing, flag sarapan/wifi/window).
    - Pada mode `unit`: Terdapat tepat 1 record `unit_types` default yang merepresentasikan spesifikasi fisik unit tersebut.
    - Pada mode `room`: Terdapat 1 atau lebih record `unit_types` untuk masing-masing tipe kamar (e.g. Deluxe Room, Superior Room, Suite).
- **`unit_type_prices`**: SSoT untuk skema harga durasi rental pada mode `unit` (`daily`, `weekly`, `monthly`, `yearly`), mencakup biaya kebersihan (`cleaning_fee`), deposit, biaya layanan (`service_fee`), serta status ketersediaan & operasional (`status`: `active`, `inactive`, `sold_out`, `maintenance`).

---

## 2. Struktur 7 Tab Wizard (Flutter ↔ Livewire Parity)

Kedua layar formulir di Flutter (`PropertyFormScreen` untuk unit dan `HotelFormScreen` untuk room) telah disinkronkan 100% dengan struktur form wizard Laravel Livewire:

| Tab | Form Properti (`unit`) | Form Hotel (`room`) |
|---|---|---|
| **Tab 1: Identitas & Lokasi** | Jenis properti (`apartment`, `house`, `villa`, `exclusive_kost`), Nama, Gedung, Kota, Status, Alamat, Koordinat Lat/Long, Tahun Buka, Rating Bintang (1-5), Jam Check-in/Check-out. | Jenis properti (`hotel`, `guest_house`, `hostel`, `hotel_residence`), Nama Hotel, Gedung, Kota, Status, Alamat, Koordinat Lat/Long, Tahun Buka, Rating Bintang (1-5), Jam Check-in/Check-out. |
| **Tab 2: Inventaris & Ruang** | Kapasitas tamu, Kamar tidur, Kamar mandi, Luas m², Konfigurasi tempat tidur (`bed_facilities[]`), Furniture kamar (`room_furniture[]`), Grup furnitur (`room_furniture_groups[]` dikelola via modal bottom sheet interaktif). | Daftar tipe kamar (`room_types[]`): Nama kamar, Bed config, Harga dasar, Harga coret, Total kuota/inventaris, Kapasitas dewasa/anak, Luas m², Flag: Breakfast, Window, Non-smoking, Free WiFi. |
| **Tab 3: Harga & Pajak** | Daftar skema sewa (`prices[]`: daily/weekly/monthly/yearly), Biaya kebersihan, Deposit, Service fee, Harga coret, Caption harga listing, Persentase pajak, Kalkulasi pajak (`inclusive`/`exclusive`). | Matrix tarif kamar interaktif (`room_types[]` diedit via `showPriceSchemeSheet()`): Tarif dasar, Harga + Pajak, Stok, Status badge, Fees subtitle, Sarapan. Caption harga listing, Persentase pajak, Kalkulasi pajak (`inclusive`/`exclusive`). |
| **Tab 4: Fasilitas** | Fasilitas gedung/properti (`facilities[]`), Spesifikasi unit (`specs[]`), Tag fasilitas. | Grup fasilitas bertingkat (`facility_groups[]`): Kategori (Populer, Kamar, Kamar Mandi, F&B), Icon, Item fasilitas dengan badge status. |
| **Tab 5: Media & Galeri** | Galeri foto utama, Deskripsi pengantar (`intro`), Deskripsi lengkap, Judul tambahan, Paragraf tambahan. | Galeri foto hotel, Deskripsi pengantar (`intro`), Deskripsi lengkap, Judul tambahan, Paragraf tambahan, Foto spesifik tiap tipe kamar. |
| **Tab 6: Aturan & Kebijakan** | Jam check-in/out, Resepsionis 24 jam, Kebijakan anak & extra bed, Kebijakan merokok, Catatan check-in/check-out dengan tone & icon, Kebijakan pembatalan (`cancellation_policies[]`) dengan tombol preset cepat. | Jam operasional resepsionis + preset, Resepsionis 24 jam, Usia minimum, Kebijakan anak, Extra bed, Merokok, Sarapan, Kebijakan spesifik kamar (`room_policies[]`), Catatan check-in/out + preset. |
| **Tab 7: Sekitar & FAQ** | Titik penting sekitar (`nearby_points[]`: nama, tipe, jarak, durasi, icon), Rincian spesifik unit (`unit_detail[]`), Tanya jawab (`questions[]`). | Transportasi & akses publik (`transport_points[]`: stasiun, bandara, terminal, jarak, durasi), Tanya jawab hotel (`faqs[]`) dengan profil author. |

---

## 3. Presets & Nilai Cepat (Quick Chips)

Formulir menyediakan preset teks standar yang identik dengan `WizardForm.php` backend:

- **Child Policy**: Gratis 0-5 thn, Semua usia, Tidak terima anak, Dewasa 12+.
- **Extra Bed Policy**: Berbayar 250rb, Bervariasi per kamar, Tidak tersedia, Diskon 50% anak.
- **Smoking Policy**: Non-Smoking seluruh area, Dilarang merokok, Balkon only, Smoking room +Rp 100rb.
- **Breakfast Policy**: Terdaftar pada voucher, Tambahan bayar di resto hotel.
- **Cancellation Policy**: Fleksibel (24 jam), Sedang (7 hari), Ketat (Non-refundable), Gratis kapan saja.
- **Reception Hours**: 24 Jam setiap hari, 07:00 - 22:00, 08:00 - 20:00, Keyless entry 24 jam.
- **Check-in / Check-out Notes**: Format terstruktur `{label, icon, tone, text}` dengan preset Mandiri, Wajib ID resmi, Resepsionis lobi, Standar 12:00, Late check-out, Express drop-box.

---

## 4. Spesifikasi Mobile API (`host/accommodations`)

Semua endpoint pendaftaran dan pengelolaan listing Mitra menggunakan rute RESTful terpadu di bawah middleware `auth:sanctum`:

### Daftar Endpoint:

| Metode | Endpoint | Handler | Deskripsi |
|---|---|---|---|
| `GET` | `host/accommodations` | `PropertyController@hostIndex` | Mengambil daftar akomodasi host dengan filter status & paginasi |
| `POST` | `host/accommodations` | `PropertyController@store` | Menyimpan akomodasi baru (otomatis membuat SSoT `unit_types` dan `prices`) |
| `GET` | `host/accommodations/{id}` | `PropertyController@hostShow` | Mengambil detail akomodasi milik host untuk populasi formulir edit |
| `PUT` | `host/accommodations/{id}` | `PropertyController@update` | Memperbarui akomodasi, tipe kamar, durasi harga, fasilitas, dan relasi |
| `DELETE` | `host/accommodations/{id}` | `PropertyController@destroy` | Menghapus listing akomodasi host |
| `POST` | `host/accommodations/gallery/upload` | `PropertyController@uploadGallery` | Unggah file foto galeri via multipart |
| `DELETE` | `host/accommodations/{id}/gallery` | `PropertyController@deleteGallery` | Menghapus foto dari galeri akomodasi tersimpan |
| `DELETE` | `host/accommodations/gallery` | `PropertyController@deleteGalleryTemp` | Menghapus file foto sementara yang belum disimpan |

> **Catatan Kompatibilitas:** Endpoint lama `host/properties` dan `host/hotels` tetap dialihkan ke handler yang sama untuk menjamin backward-compatibility.

---

## 5. Sinkronisasi Flutter Routing

- **Route Constant**: `Routes.mitraAccommodationForm = '/mitra/accommodations/form'`.
- **Dispatcher**: `router.dart` membaca argumen rute:
  - Jika `booking_mode == 'room'` → Merender `HotelFormScreen(hotelId, accommodationId, initialData)`.
  - Jika `booking_mode == 'unit'` → Merender `PropertyFormScreen(property, accommodationId, initialData)`.
  - Jika mode belum ditentukan (`null`) → Merender `AccommodationFormScreen()` sebagai mode selector dialog.
- **Mitra Property Management**: Tombol Floating Action Button (`+ Tambah`) memanggil `_showAddOptionSheet` yang memungkinkan Mitra memilih antara unit apartemen/villa atau hotel OTA.

---

## 6. Arsitektur Modal Bottom Sheets Formulir (`lib/screens/mitra/sheets/`)

Untuk menyederhanakan ukuran file formulir yang kompleks (`property_form_screen.dart` dan `hotel_form_screen.dart`), seluruh formulir sub-dialog dan builder entitas modular telah dipisahkan ke direktori `lib/screens/mitra/sheets/` dengan barrel export `sheets.dart`:

| Sheet File | Fungsi / Modal Builder | Target Form | Deskripsi & Payload Model |
|---|---|---|---|
| `room_spec_sheet.dart` | `showRoomSpecSheet()` | Unit (`PropertyFormScreen`) | Kapasitas tamu, kamar tidur, kamar mandi, luas m² via steppers interaktif dengan validasi minimum. Mengembalikan `Map<String, num>`. |
| `bed_facility_sheet.dart` | `showBedFacilitySheet()` | Unit (`PropertyFormScreen`) | Form tambah/edit konfigurasi tempat tidur (`label`, `note`, `count`, `icon_name`). Mengembalikan `Map<String, dynamic>`. |
| `room_furniture_group_sheet.dart` | `showRoomFurnitureGroupSheet()` | Unit (`PropertyFormScreen`) | Modal builder nama ruangan, daftar item furnitur per ruangan, dan konfigurasi bed per grup. |
| `price_scheme_sheet.dart` | `showPriceSchemeSheet()` | Unit (`PropertyFormScreen`) & Hotel (`HotelFormScreen` Tab 3) | Form tambah/edit skema sewa (`daily`, `weekly`, `monthly`, `yearly`), nominal harga (`amount` / `base_price`), cleaning fee, deposit, service fee, dan status operasional (`active`, `inactive`, `sold_out`, `maintenance`). Jika dibuka dari `booking_type = 'room'` / `booking_mode = 'room'`, dropdown "Tipe Sewa" otomatis disembunyikan dan dikunci ke `daily`. Sinkron penuh dengan penyimpanan room types & `unit_type_prices` di Laravel `PropertyController@update`. |
| `building_facility_sheet.dart` | `showBuildingFacilitySheet()` | Unit (`PropertyFormScreen`) | Form tambah/edit fasilitas gedung dengan 11 icon picker interaktif (`facility_name`, `icon`). Mengembalikan `Map<String, dynamic>`. |
| `property_spec_sheet.dart` | `showPropertySpecSheet()` | Unit (`PropertyFormScreen`) | Form tambah/edit spesifikasi properti dengan 5 kategori icon (`spec_name`, `icon`). Mengembalikan `Map<String, dynamic>`. |
| `hotel_room_type_sheet.dart` | `showHotelRoomTypeSheet()` | Hotel (`HotelFormScreen`) | Full builder tipe kamar hotel: Nama kamar, bed configuration, base price, original price, allotment kuota inventaris, kapasitas dewasa/anak, luas m², deskripsi, dan flags (breakfast, window, non-smoking, free WiFi). Mengembalikan `Map<String, dynamic>`. |
| `hotel_facility_group_sheet.dart` | `showHotelFacilityGroupSheet()` | Hotel (`HotelFormScreen`) | Form tambah/edit grup fasilitas bertingkat: Kategori/group selector (dengan quick chips grup yang sudah ada), icon picker, item name, badge selector (Gratis/Tersedia/Populer/etc), dan toggle ketersediaan. Mengembalikan `Map<String, dynamic>`. |
| `operational_rule_sheet.dart` | `showOperationalRuleSheet()` | Unit & Hotel | Modal konfigurasi aturan operasional: jam check-in, jam check-out, nominal deposit, usia minimum tamu, jam operasional resepsionis (+ presets), dan toggle switch 24 jam resepsionis, tamu anak, serta extra bed. Mengembalikan `Map<String, dynamic>`. |
| `checkin_checkout_note_sheet.dart` | `showCheckInOutNoteSheet()` | Unit & Hotel | Form tambah/edit catatan operasional check-in dan check-out dengan preset cepat, pemilih tone notifikasi (`info`, `warning`, `success`, `danger`), dan icon picker (`clock`, `info`, `building`, `key`, `door`, dll). Mengembalikan `Map<String, dynamic>`. |
| `policy_item_sheet.dart` | `showPolicyItemSheet()` / `PolicyArraySection` | Unit & Hotel | Modal tambah/edit butir kebijakan custom & komponen visual `PolicyArraySection` untuk kebijakan anak, extra bed, merokok, sarapan, pembatalan, dan kamar hotel dengan preset dinamis serta aksi edit/hapus. Mengembalikan `String`. |
| `nearby_point_sheet.dart` | `showNearbyPointSheet()` | Unit & Hotel | Form modal tambah/edit titik lokasi penting sekitar (POI) dan titik transit transportasi hotel dengan saran kategori (`Stasiun`, `Bandara`, `Terminal`, dll), jarak, estimasi durasi, dan pemilih ikon (`train`, `airport`, `bus`, `mall`, dll). Mengembalikan `Map<String, dynamic>`. |
| `faq_item_sheet.dart` | `showFaqItemSheet()` | Unit & Hotel | Form modal tambah/edit tanya jawab (FAQ) akomodasi: pertanyaan tamu, jawaban pengelola/staff, nama penjawab, dan peran/jabatan penjawab. Mengembalikan `Map<String, dynamic>`. |
| `unit_detail_sheet.dart` | `showUnitDetailSheet()` | Unit (`PropertyFormScreen`) | Form modal tambah/edit rincian spesifik unit (Detail Properti Tambahan) dengan preset nilai cepat (`Parkir`, `Daya Listrik`, `Lantai`, `Pemandangan`, dll) dan pemilih ikon. Mengembalikan `Map<String, dynamic>`. |
| `building_form_sheet.dart` | `showBuildingFormSheet()` | Unit & Hotel | Form modal tambah gedung/tower apartemen baru langsung dengan upload foto gedung, pilihan kota, dan nama gedung. Mengembalikan `BuildingList`. |
| `pricing_tax_sheet.dart` | `showPricingTaxSheet()` | Unit (`PropertyFormScreen`) & Hotel (`HotelFormScreen` Tab 3) | Form modal pengaturan harga coret/normal (`price_original`), caption harga listing (`price_caption`), persentase pajak (`tax_percentage`), dan kalkulasi pajak (`exclusive`/`inclusive`). Mengembalikan `Map<String, dynamic>`. |
| `sheets.dart` | *Barrel Export* | Unit & Hotel | Meng-export seluruh enam belas (16) sheet di atas untuk kemudahan import satu pintu (`import 'sheets/sheets.dart';`). |

---

## 7. Mekanisme Sinkronisasi Tarif Kamar & Skema Harga (`unit_types` & `unit_type_prices`)

Pada mode hotel (`room`), setiap tipe kamar memiliki konfigurasi tarif dasar dan biaya-biaya pendukung:

1. **Struktur Penyimpanan Backend**:
    - `unit_types`: Menyimpan metadata kamar fisik (`name`, `bed_configuration`, `base_price`, `price_original`, `total_inventory`, `room_size_sqm`, `has_breakfast`, `description`, `is_active`).
    - `unit_type_prices`: Menyimpan rincian harga per durasi (`price_type` default `daily`), `amount` (nominal tarif kamar), `cleaning_fee`, `deposit`, `service_fee`, dan ketersediaan `status` (`active`, `inactive`, `sold_out`, `maintenance`).
2. **Serialisasi API Resource**:
    - `UnitTypeResource`: Membuka field flat pada objek root (`price`, `amount`, `price_type`, `cleaning_fee`, `deposit`, `service_fee`, `status`) dari skema harga harian (`dailyPrice`), selain menyertakan relasi nested `prices`.

- `AccommodationResource`: Menyediakan detail hotel dengan eager loading `unitTypes.prices` sehingga tarif kamar dan biaya pendukung selalu tersedia saat form dibuka kembali.

3. **Penyimpanan Controller**:
    - `PropertyController@update` & `HostHotelController@update`: Membaca array `room_types` dari payload, memperbarui tabel `unit_types`, dan melakukan upsert (`updateOrCreate`) ke `unit_type_prices` baik dari field flat maupun array nested `prices`.
4. **Alur Frontend Flutter**:
    - `HotelRoomItem`: Menyimpan dan mem-parse `priceType`, `cleaningFee`, `deposit`, `serviceFee`, `status`, `isActive`, dan `prices`.
    - `hotel_room_type_sheet.dart`: Menggunakan spread operator `{ if (init != null) ...init, ... }` saat submit sehingga field tarif dan biaya tidak terhapus ketika user mengedit spesifikasi fisik kamar di Tab 2.
    - `hotel_form_screen.dart` (Tab 3): Memanggil `showPriceSchemeSheet(bookingType: 'room')`, memperbarui `_roomTypes[idx]` dengan field flat sekaligus nested `'prices'` array, dan langsung merefleksikan perubahan pada kartu ringkasan matrix harga (tarif dasar, harga + pajak, stok, status badge, dan rincian biaya kebersihan/deposit/service fee).

---

## 8. Pembaruan Kilat Ketersediaan & Tarif (Quick Availability & Price Edit)

Untuk mempercepat perubahan ketersediaan dan harga tanpa harus membuka wizard edit penuh 7 tab, telah diimplementasikan alur end-to-end terpadu:

### 1. Backend: Strictly Update-Only API (`UnitTypePriceController`)

- **Rute**:
    - `GET api/v1/host/unit-type-prices/{id}`
    - `PUT api/v1/host/unit-type-prices/{id}` (dan nested `PUT api/v1/unit-types/{unitType}/prices/{price}`)
- **Prinsip Keamanan Strictly Update-Only (No Create)**:
    - Menggunakan `UnitTypePrice::findOrFail($id)` tanpa method `create` atau `insert`.
    - Jika ID tidak ditemukan, mengembalikan `404 Not Found`.
    - Upaya `POST` diblokir oleh rute (`405 Method Not Allowed`).
    - Verifikasi kepemilikan host (`accommodation->host_id === $request->user()->id`), mengembalikan `403 Forbidden` jika bukan pemilik.
    - Validasi payload ketat: `amount` (min: 0), `cleaning_fee` (min: 0), `deposit` (min: 0), `service_fee` (min: 0), `price_type` (in: `daily,weekly,monthly,yearly`), `status` (in: `active,inactive,sold_out,maintenance`).
    - Mencegah duplikasi `price_type` pada `unit_type` yang sama (`422 Unprocessable Entity`).
    - Mengembalikan respon `200 OK` dengan `UnitTypePriceResource` serta menyinkronkan `unit_types.base_price` jika durasi harian diubah.

### 2. Frontend: Modal Bottom Sheet Navigasi (`property_management_screen.dart`)

- Pada saat kartu properti ditekan (`onTap` di baris 319), sistem menampilkan Modal Bottom Sheet dengan 2 opsi jelas:
    1. **Opsi 1 (Full Edit)**: Membuka wizard edit penuh 7 tab via `_openForm(p)` (`Routes.mitraAccommodationForm`).
    2. **Opsi 2 (Available & Price Edit)**: Membuka layar pembaruan kilat via `_openAvailabilityPriceEdit(p)` (`Routes.mitraAvailabilityPriceEdit`).
- Opsi juga dapat diakses langsung melalui menu titik tiga (`PopupMenuButton`) pada setiap kartu.

### 3. Frontend: Layar Pembaruan Cepat (`availability_price_edit_screen.dart`)

- **Kondisi Unit (Apartemen/Villa/Kost)**:
    - Mengadopsi arsitektur dari baris 1876 `property_form_screen.dart`.
    - Menampilkan daftar skema tarif rental (`daily`, `weekly`, `monthly`, `yearly`).
    - Tombol edit memanggil `showPriceSchemeSheet(bookingType: 'unit')`.
    - Nilai yang diperbarui langsung dikirim via `HostService.updateUnitTypePrice(priceId, payload)`.
- **Kondisi Room (Hotel OTA)**:
    - Mengadopsi arsitektur dari baris 1940–2070 `hotel_form_screen.dart`.
    - Menampilkan matrix tipe kamar (nama kamar, kapasitas, harga dasar, simulasi harga + pajak, stok kamar, badge status ketersediaan, rincian biaya).
    - Tombol edit memanggil `showPriceSchemeSheet(bookingType: 'room')`.
    - Nilai yang diperbarui langsung dikirim via `HostService.updateUnitTypePrice(priceId, payload)`.
- **Feedback & State**:
    - Validasi feedback interaktif (SnackBar sukses/gagal, indikator loading per item, dan refresh otomatis daftar saat navigasi kembali).

<!-- Source: features/property-management.md -->

# Fitur Manajemen Properti Mitra (Host Property Management)

Dokumentasi implementasi, arsitektur, dan audit integrasi untuk modul Manajemen Properti Mitra di Flutter dan Laravel.

---

## 1. Arsitektur Layar & Komponen

- **Frontend Screen**: [property_management_screen.dart](file:///d:/Project/Fastwork/rental.apartemen/flutter/lib/screens/mitra/property_management_screen.dart) (Screen 30)
- **Dart Service**: [host_service.dart](file:///d:/Project/Fastwork/rental.apartemen/flutter/lib/services/host_service.dart) (`properties()`)
- **Backend Controller**: [PropertyController.php](file:///d:/Project/Fastwork/rental.apartemen/laravel/app/Http/Controllers/Api/PropertyController.php) (`hostAccommodations()`)
- **Resource Envelope**: [AccommodationResource.php](file:///d:/Project/Fastwork/rental.apartemen/laravel/app/Http/Resources/AccommodationResource.php) (`hotelShape()` & `propertyShape()`)
- **Model Dart**: [property.dart](file:///d:/Project/Fastwork/rental.apartemen/flutter/lib/models/property.dart) (`Property.fromJson`)

---

## 2. Kontrak Endpoint API

**`GET /api/v1/host/accommodations`**

- **Autentikasi**: Sanctum Token (`auth:sanctum`)
- **Query Parameter**:
  - `page`: Nomor halaman paginasi (default: `1`)
  - `per_page`: Jumlah item per halaman (default: `15`)
  - `status`: Filter status (`active`, `inactive`, `maintenance`)
  - `booking_mode`: Filter mode (`unit`, `room`)
  - `q`: Pencarian kata kunci nama/alamat akomodasi
- **Karakteristik Output**:
  - Mengembalikan daftar akomodasi terpadu (baik unit apartemen/villa maupun hotel) milik host yang sedang login.
  - Tidak memfilter dengan `withValidPricing()`, sehingga listing baru yang belum memiliki unit type / harga tetap dapat dilihat dan dikelola oleh host.
  - `hotelShape` menyertakan `status`, `host_id`, dan `deposit_amount` numerik.

---

## 3. Resolusi Masalah (Audit & Fix)

1. **Type Cast Exception di `Property.fromJson`**:
   - `deposit_amount` pada `hotelShape` sebelumnya diformat sebagai string via `number_format` (`"500000.00"`).
   - `property.dart` telah diperbaiki menggunakan parser defensif `_toDoubleOrNull(json['deposit_amount'])` dan pembacaan numerik yang aman (`_toInt`, `_toDouble`) untuk seluruh atribut numerik.
2. **Missing Fields pada `hotelShape`**:
   - Ditambahkan atribut `'status' => $this->status` dan `'host_id' => $this->host_id` pada `AccommodationResource::hotelShape()`.
   - Mengubah `deposit_amount` pada `hotelShape` menjadi nilai numerik `(float) $this->deposit_amount` yang konsisten.
3. **Penyempurnaan Query Host**:
   - Menghapus filter `->withValidPricing()` pada `hostAccommodations` dan `hostIndex` agar listing draf/baru milik host tidak tersembunyi.
4. **Error Handling & Diagnostics**:
   - Menampilkan `ApiException::message` jika request gagal dan mencatat stack trace via `debugPrint` untuk mempermudah diagnosa.

---

## 4. Pengujian Otomatis

- **Laravel Feature Test**: `HostAccommodationsApiTest::test_host_accommodations_lists_all_host_properties_with_status_and_correct_shapes`
- **Flutter Model Test**: `test/models/property_test.dart`
- **Flutter Widget Test**: `test/screens/mitra/property_management_screen_test.dart`

---

## 5. Form Edit Data Binding & Multi-Step Tab Locking

### A. Edit Data Binding Flow

- **Navigasi dari Manajemen Properti**: Menu "Edit" dan "Kelola Hotel & Kamar" pada [property_management_screen.dart] (file:///d:/Project/Fastwork/rental.apartemen/flutter/lib/screens/mitra/property_management_screen.dart) mengirimkan argumen `{'id': p.id, 'booking_mode': p.bookingMode, 'property': p}`.
- **Router Resolution ([router.dart](file:///d:/Project/Fastwork/rental.apartemen/flutter/lib/router.dart))**:
  - `booking_mode == 'room'` mengarahkan ke [HotelFormScreen] (file:///d:/Project/Fastwork/rental.apartemen/flutter/lib/screens/mitra/hotel_form_screen.dart).
  - `booking_mode == 'unit'` mengarahkan ke [PropertyFormScreen] (file:///d:/Project/Fastwork/rental.apartemen/flutter/lib/screens/mitra/property_form_screen.dart).
  - Objek `Property` dan `accommodationId` diinjeksi langsung ke konstruktor layar form.
- **Inisialisasi Sinkron & Background Refresh**:
  - Pada `initState()`, form secara sinkron memetakan seluruh atribut model `Property` ke dalam `TextEditingController`, dropdown, dan list konfigurasi (`_loadFromPropertyModel`).
  - Bila ID tersedia, detail terbaru di-fetch di background (`_loadRemoteDetail`). Method `_loadInitialData` dirancang defensif sehingga field null atau list kosong dari respons API tidak menimpa data yang telah terisi sebelumnya.

### B. Strict Tab Locking untuk Akomodasi Baru

- **Kondisi Kunci**: Pada mode *Create* (`property == null && accommodationId == null`), host wajib melengkapi dan menyimpan identitas dasar (Tab 1) ke database terlebih dahulu.
- **Mekanisme Penguncian**:
  - **Swipe Lock**: `TabBarView.physics` menggunakan `NeverScrollableScrollPhysics()` hingga `_canNavigateTabs == true`.
  - **Tap Lock**: `TabBar.onTap` mencegat klik pada tab indeks > 0, mengembalikan controller ke tab 0, dan memunculkan notifikasi peringatan.
  - **Visual Indicator**: Setiap tab setelah tab 1 menampilkan ikon gembok (`Icons.lock_outline`).
  - **Aksi Simpan Tahap 1**: Tombol dedicated *"Simpan Identitas & Buka Tab Lainnya"* memvalidasi kelengkapan form Tab 1 (nama, alamat, kota), mengeksekusi `createProperty()`, menyimpan `_savedAccommodationId`, dan membuka kunci seluruh tab serta bertransisi ke Tab 2 secara otomatis.

### C. Pengujian Form & Validasi Tab

- **Widget Test Suite**: [form_binding_and_tab_lock_test.dart] (file:///d:/Project/Fastwork/rental.apartemen/flutter/test/screens/mitra/form_binding_and_tab_lock_test.dart)
  - Pengujian pre-populasi data form pada mode Edit unit & hotel.

- Pengujian pencegahan swipe dan tap navigasi tab pada mode Create.
- Pengujian pembukaan kunci tab setelah penyimpanan Tab 1 berhasil.
- Pengujian resolusi argumen rute pada `AppRouter`.

<!-- Source: features/mitra-order-management.md -->

# Manajemen Order Mitra (Host Order Management)

Dokumentasi arsitektur, state mapping, dan alur kerja untuk manajemen order mitra di Flutter dan Laravel.

---

## 1. Ikhtisar Layar & Routing

| Layar | File | Route | Argumen | Deskripsi |
|---|---|---|---|---|
| **OrderManagementScreen** | `lib/screens/mitra/order_management_screen.dart` | `Routes.mitraOrders` (`/mitra/orders`) | None | Tab bar interaktif 3 kolom (Baru, Dikonfirmasi, Selesai) dengan pagination tak terbatas dan aksi konfirmasi/penolakan instan. |
| **OrderDetailScreen** | `lib/screens/mitra/order_detail_screen.dart` | `Routes.mitraOrderDetail` (`/mitra/orders/detail`) | `Order` (atau map/ID fallback) | Detail order vertikal lengkap: header kode order, status timeline (disembunyikan bila status `cancelled`), informasi properti, data tamu, rincian waktu, rincian pembayaran, serta tombol konfirmasi geser modern (`SlidingConfirmationButton` / swipe-to-confirm) yang ditempatkan pada sticky `bottomNavigationBar` di bagian bawah layar. |

---

## 2. Pemetaan Tab & Status Booking

Order pada layar manajemen diklasifikasikan ke dalam 3 tab berdasarkan `effectiveBookingStatus` (`order.bookingStatus ?? order.status`):

| Tab | Status yang Dicakup | Aksi yang Tersedia |
|---|---|---|
| **Baru** | `waiting_confirmation`, `waiting_payment` | **Terima** (`confirmOrder`), **Tolak** (`rejectOrder`) jika belum expired. |
| **Dikonfirmasi** | `confirmed`, `staying` | Tap kartu membuka detail untuk **Check-in** (`confirmed` -> `staying`) atau **Check-out** (`staying` -> `completed`). |
| **Selesai** | `completed`, `cancelled`, `expired` | Review riwayat menginap & histori pembayaran selesai. |

---

## 3. Alur Aksi & Kontrak Endpoint API

```
graph TD
    A[Order Masuk / waiting_confirmation] -->|Host Terima| B[POST /api/v1/bookings/{id}/confirm]
    A -->|Host Tolak| C[POST /api/v1/bookings/{id}/reject]
    B -->|Status: confirmed| D[Menunggu Check-in]
    D -->|Host Check-in| E[POST /api/v1/bookings/{id}/checkin]
    E -->|Status: staying| F[Tamu Sedang Menginap]
    F -->|Host Check-out| G[POST /api/v1/bookings/{id}/checkout]
    G -->|Status: completed| H[Booking & Order Selesai]
```

### Rincian Endpoint:

1. `GET host/orders` (`OrderController::forHost`): Mengambil daftar order yang terkait dengan akomodasi milik mitra (mendukung filter `page`, `status`, `booking_status`).
2. `GET host/orders/{order}` (`OrderController::showForHost`): Mengambil detail order mitra beserta properti, kamar, asuransi, dan pembayaran.
3. `POST bookings/{booking}/confirm` (`BookingController::confirm`): Host menerima pesanan, status booking berubah menjadi `confirmed`.
4. `POST bookings/{booking}/reject` (`BookingController::reject`): Host menolak pesanan, status booking berubah menjadi `rejected` dan order `cancelled`, serta inventaris dikembalikan.
5. `POST bookings/{booking}/checkin` (`BookingController::checkin`): Host mencatat tamu check-in, status booking berubah menjadi `staying`.
6. `POST bookings/{booking}/checkout` (`BookingController::checkout`): Host mencatat tamu check-out, status booking berubah menjadi `completed` dan status order menjadi `completed`.
7. `POST bookings/{booking}/confirm-checkin` (`BookingController::confirmCheckin`): Host mengonfirmasi check-in tamu (status booking menjadi `staying`).

---

## 4. Model DTO (`Order`)

Model `lib/models/order.dart` diperkaya dengan metadata akomodasi dan tamu yang diekstrak dari payload relasi:

- `propertyName`: Nama akomodasi (`booking.property.name` / `json.property_name`).
- `propertyAddress` & `propertyCity`: Alamat dan kota properti.
- `propertyImage`: URL foto sampul properti.
- `bookingCode`: Kode pemesanan (`booking.booking_code`).
- `guestName` & `guestPhone`: Nama dan kontak pemesan (`booking.contact_phone` / `user.name` / `user.phone`).
- `effectiveBookingStatus`: Helper getter `bookingStatus ?? status`.
- `isConfirmationExpired`: Memastikan indikator kedaluwarsa hanya aktif jika status pemesanan secara eksplisit `expired` dari backend.

---

## 5. Pengujian Otomatis

Modul diuji secara otomatis dengan 100% stabilitas:

- `test/screens/mitra/order_management_screen_test.dart`:

    - Pengujian transisi tab (Baru, Dikonfirmasi, Selesai) dan rendering nama properti.
    - Pengujian alur dialog konfirmasi "Terima" beserta pemanggilan service dan SnackBar.
    - Pengujian alur dialog konfirmasi "Tolak" dan verifikasi pembatalan dialog.

- `test/screens/mitra/order_detail_screen_test.dart`:

    - Pengujian rendering detail lengkap (timeline, data tamu, informasi properti).
    - Pengujian tombol aksi geser kontekstual via gesture swipe ("Geser untuk Terima", "Geser untuk Tolak", "Geser jika Tamu Tiba", "Geser Konfirmasi Check-in", "Geser untuk Check-in", "Geser untuk Check-out").
    - Pengujian pencegahan false expired warning pada order yang telah selesai.
    - Pengujian penyembunyian kartu Status Pesanan (timeline) saat status booking bernilai `cancelled`.

- `test/widgets/sliding_confirmation_button_test.dart`:

    - Pengujian rendering label, ikon, dan status loading pada `SlidingConfirmationButton`.
    - Pengujian gesture drag di bawah ambang batas (snaps back tanpa memicu aksi).
    - Pengujian gesture drag melampaui ambang batas (memicu `onConfirmed`).
    - Pengujian status `isLoading: true` yang mematikan drag gesture dan menampilkan indikator loading.

<!-- Source: features/payments.md -->

# Payment Setup Guide

Panduan lengkap mengisi URL di dashboard Midtrans, memahami perbedaan Notification URL dan Finish Redirect URL, serta langkah testing dan produksi.

## Daftar Isi


- Kebijakan Guest Browsing
- Ringkasan URL
- Notification URL vs Finish Redirect URL
- Alur Lengkap
- Langkah Setup
- Testing Lokal
- Naik ke Produksi
- Troubleshooting


## Kebijakan Guest Browsing

Browsing properti bersifat publik, tetapi booking dan pembayaran memerlukan login:

- **Publik:** `GET /api/v1/home`, `/api/v1/properties`, `/api/v1/properties/{id}`, `/api/v1/cities`, `/api/v1/vouchers`
- **Login Required:** `POST /api/v1/bookings`, `/POST /api/v1/orders/{id}/pay`, `/api/v1/wishlists`, `/api/v1/conversations`
- Di Flutter, `SplashScreen` mengarahkan guest ke `Routes.home`. Aksi transaksional menampilkan `Routes.login` jika tidak terautentikasi.

## Ringkasan URL

| Field di Dashboard Midtrans | Isi |
|---|---|
| **Payment Notification URL** | `https://livirooms.com/api/v1/payments/webhook` |
| **Finish Redirect URL** | `https://livirooms.com/payment/finish` |
| **Unfinish Redirect URL** | `https://livirooms.com/payment/unfinish` |
| **Error Redirect URL** | `https://livirooms.com/payment/error` |

## Perbedaan Notification URL dan Finish Redirect URL

### Notification URL — jalur yang dipercaya

```
Server Midtrans  ──HTTP POST──▶  Server Laravel Anda
```

- Dipanggil **server-ke-server**. Browser pengguna tidak terlibat.
- Membawa `signature_key` = `sha512(order_id + status_code + gross_amount + server_key)`.
- Karena `server_key` hanya diketahui Midtrans dan server Anda, tanda tangan ini **membuktikan** pesannya asli.
- **Inilah satu-satunya jalur yang boleh mengubah status pembayaran.**
- Midtrans mengulang kirim sampai dapat HTTP `200`. Handler-nya wajib idempoten — dan di kode ini sudah: `applyTransactionStatus()` keluar lebih awal bila `status === 'paid'` (`Api/PaymentController.php:319`).

### Finish Redirect URL — jalur yang TIDAK dipercaya

```
Browser pengguna  ──redirect──▶  Server Laravel Anda
```

- Yang mengunjungi adalah **browser pengguna** setelah ia menekan tombol selesai di halaman Snap.
- Semua query string-nya dikendalikan pengguna. Siapa pun bisa mengetik `?order_id=ORD-123-45&transaction_status=settlement` di address bar.
- **Tidak boleh menyentuh status pembayaran.** Lihat komentar di `app/Http/Controllers/Payment/FinishController.php:10-20`.
- Tugasnya hanya satu: memantulkan pengguna kembali ke aplikasi Flutter.

> Analogi: Notification URL itu petugas bank yang menelepon lewat nomor resmi. Finish Redirect URL itu orang yang mengetuk pintu Anda dan mengaku dari bank. Yang pertama boleh mengubah saldo, yang kedua hanya boleh dipersilakan masuk.

Kalau Anda hanya mengisi Finish Redirect URL dan mengabaikan Notification URL, **tidak ada satu pun pembayaran yang akan tercatat lunas.** Booking pengguna akan kedaluwarsa oleh `payments:cancel-expired` meskipun uangnya sudah masuk.

## Notification URL — Detail

### Route-nya

`laravel/routes/api_mobile.php:52-53`

```
Route::post('payments/webhook', [PaymentController::class, 'webhook']);
```

Route ini berada di dalam `Route::prefix('v1')`, dan file `routes/api.php` memuatnya di bawah prefix `/api` bawaan Laravel. Jadi path lengkapnya:

```
/api  +  /v1  +  /payments/webhook   →   /api/v1/payments/webhook
```

Perhatikan letaknya: **di luar** grup `auth:sanctum`. Memang harus begitu — Midtrans tidak punya token Sanctum. Pengaman jalur ini adalah tanda tangan SHA512, bukan sesi login.

### Yang diisi di dashboard

| Lingkungan | Isi |
|---|---|
| Sandbox, domain publik | `https://staging.livirooms.com/api/v1/payments/webhook` |
| Produksi | `https://livirooms.com/api/v1/payments/webhook` |
| Lokal (via tunnel) | `https://a1b2c3d4.ngrok-free.app/api/v1/payments/webhook` |

### Syarat yang wajib dipenuhi

1. **HTTPS.** Midtrans menolak `http://` di produksi.
2. **Dapat diakses publik.** `localhost` dan IP privat tidak akan pernah dihubungi Midtrans.
3. **Tanpa auth.** Jangan letakkan di balik Basic Auth, Cloudflare Access, atau halaman "maintenance".
4. **Balas 200 secepatnya.** Kalau server balas `500` atau timeout, Midtrans mengulang beberapa kali lalu menyerah — dan pembayaran akan menggantung.
5. **Bebas dari CSRF.** Sudah aman: route API tidak memakai middleware `web`.

### Yang diisi di dashboard

`Settings` → `Configuration` → kolom **Payment Notification URL**.

Ada juga kolom `Finish/Unfinish/Error Redirect URL` di halaman yang sama — lihat bagian berikutnya.

## Finish Redirect URL — Detail

### Route-nya

`laravel/routes/web.php:35-38`

```
Route::get('/payment/finish',   [FinishController::class, 'finish'])->name('payment.finish');
Route::get('/payment/unfinish', [UnfinishController::class, 'unfinish'])->name('payment.unfinish');
Route::get('/payment/error',    [ErrorController::class, 'error'])->name('payment.error');
```

Ketiganya di `routes/web.php` **tanpa** prefix `admin` dan **tanpa** middleware `auth` — memang harus begitu, karena yang membukanya browser eksternal yang mungkin belum login.

Jadi path-nya persis: `/payment/finish` — tanpa `/api`, tanpa `/v1`.

### Yang diisi di dashboard

| Lingkungan | Finish | Unfinish | Error |
|---|---|---|---|
| Produksi | `https://livirooms.com/payment/finish` | `.../payment/unfinish` | `.../payment/error` |
| Sandbox | `https://staging.livirooms.com/payment/finish` | `.../payment/unfinish` | `.../payment/error` |
| Lokal | `https://a1b2c3d4.ngrok-free.app/payment/finish` | `.../payment/unfinish` | `.../payment/error` |

### Kapan masing-masing dipanggil

| URL | Dipanggil saat |
|---|---|
| **Finish** | Pengguna menyelesaikan alur pembayaran. **Belum tentu lunas** — bisa saja ia baru dapat nomor VA dan belum mentransfer. |
| **Unfinish** | Pengguna menutup/meninggalkan halaman Snap sebelum selesai. |
| **Error** | Terjadi kegagalan di sisi Midtrans. |

Poin di baris "Finish" sering disalahpahami. `finish` **bukan** sinonim "berhasil". Untuk pembayaran VA, pengguna sampai di `finish` begitu nomor VA terbit — pembayaran sesungguhnya baru terjadi berjam-jam kemudian, dan hanya webhook yang tahu.

### Satu catatan penting soal dashboard

Nilai di dashboard sebenarnya adalah **fallback**. Kode ini sudah mengirim `callbacks` di setiap permintaan Snap (`app/Support/MidtransSnap.php:129-133`):

```
'callbacks' => [
    'finish'   => url('/payment/finish'),
    'unfinish' => url('/payment/unfinish'),
    'error'    => url('/payment/error'),
],
```

`url()` menurunkan host dari `APP_URL`. Artinya:

> **`APP_URL` di `.env` wajib benar.** Kalau masih `http://localhost`, Midtrans akan memantulkan pengguna produksi ke `localhost` — dan alur pembayaran putus di situ, tanpa pesan galat yang jelas.

```
APP_URL=https://livirooms.com
```

Tetap isi juga kolom di dashboard sebagai jaring pengaman.

## Alur Lengkap, Ujung ke Ujung




Perhatikan (A) dan (B) berjalan **paralel dan independen**. Aplikasi tidak pernah mempercayai (B) untuk status; ia selalu bertanya ulang ke server.

Deep link yang dipakai (`app/Support/PaymentDeepLink.php:44-52`):

```
sewain://payment-result?paymentId=45&orderId=12
```

Sengaja tidak membawa status — lihat komentar di `PaymentDeepLink.php:7-16`.

## Langkah Setup

### Langkah 1 — Ambil kunci dari dashboard sandbox

1. Masuk ke `https://dashboard.sandbox.midtrans.com`
2. `Settings` → `Access Keys`
3. Salin **Merchant ID**, **Client Key**, dan **Server Key**

Server Key sandbox berawalan `SB-Mid-server-`, Client Key `SB-Mid-client-`. Kunci produksi tanpa awalan `SB-`.

> **Server Key adalah kredensial paling sensitif di sistem ini.** Ia memverifikasi webhook dan mengotorisasi refund. Jangan pernah menaruhnya di kode Flutter, repo publik, atau tiket. Client Key boleh berada di sisi klien; Server Key tidak.

### Langkah 2 — Masukkan kunci ke aplikasi

Ada dua cara, dan **cara pertama yang dianjurkan**:

**Cara A — lewat panel admin (dianjurkan)**

```
Login sebagai admin → menu Pembayaran → Pengaturan Pembayaran
```

Isi Server Key, Client Key, dan sakelar Production Mode, lalu simpan. Nilainya disimpan ke tabel `payment_settings` dan langsung dipakai — `AppServiceProvider` mendorongnya ke `config('services.midtrans.*')` saat boot.

Kelebihannya: rotasi kunci tidak butuh akses SSH dan tidak perlu deploy ulang.

> Halaman ini hanya untuk peran **admin**. Peran `host` (pemilik properti) sengaja tidak diberi akses, karena halaman ini memuat Server Key.

**Cara B — lewat `.env` (hanya untuk instalasi baru)**

```
MIDTRANS_SERVER_KEY=SB-Mid-server-xxxxxxxxxxxxxxxxxxxx
MIDTRANS_CLIENT_KEY=SB-Mid-client-xxxxxxxxxxxxxxxxxxxx
MIDTRANS_IS_PRODUCTION=false
```

`.env` hanya berfungsi sebagai nilai awal. Begitu ada baris di `payment_settings`, baris itulah yang menang (`app/Models/PaymentSetting.php:67-78`).

### Langkah 3 — Pastikan `APP_URL` benar

```
APP_URL=https://livirooms.com
```

Lalu bersihkan cache konfigurasi:

```
php artisan config:clear
```

### Langkah 4 — Isi URL di dashboard

`Settings` → `Configuration`:

| Kolom | Isi |
|---|---|
| Payment Notification URL | `https://livirooms.com/api/v1/payments/webhook` |
| Finish Redirect URL | `https://livirooms.com/payment/finish` |
| Unfinish Redirect URL | `https://livirooms.com/payment/unfinish` |
| Error Redirect URL | `https://livirooms.com/payment/error` |

Tekan **Update**.

### Langkah 5 — Aktifkan metode pembayaran

```
Panel admin → Pembayaran → Metode Pembayaran
```

Kolom `code` berupa dropdown, isinya terbatas pada kode yang bisa dipetakan `MidtransSnap` ke channel Snap:

`virtual_account`, `bank_transfer`, `qris`, `ewallet`, `credit_card`, `paylater`

Pembatasan ini disengaja. Kode di luar daftar akan sampai ke Snap tanpa pemetaan channel, sehingga pengguna melihat **semua** pilihan pembayaran padahal `payments.method` mencatat satu metode tertentu — laporan jadi tidak cocok dengan data Midtrans.

Untuk mempersempit channel lebih jauh, isi kolom Metadata:

```
{"banks": ["bca", "bni"]}
```

atau

```
{"channels": ["gopay", "shopeepay"]}
```

Pemetaannya ada di `app/Support/MidtransSnap.php:35-69`.

### Langkah 6 — Pastikan scheduler berjalan

```
* * * * * cd /var/www/sewain && php artisan schedule:run >> /dev/null 2>&1
```

Tanpa ini, `payments:cancel-expired` tidak pernah jalan dan pembayaran yang telantar akan menahan ketersediaan properti selamanya. Jadwalnya terdaftar di `routes/console.php:14-19` (tiap 10 menit).

## Menguji Secara Lokal (tanpa domain publik)

Midtrans tidak bisa menghubungi `localhost`. Buka terowongan:

```
ngrok http 8000
```

Ambil URL HTTPS yang muncul, misalnya `https://a1b2c3d4.ngrok-free.app`, lalu:

1. Perbarui `.env`:

   ```
   APP_URL=https://a1b2c3d4.ngrok-free.app
   ```

2. `php artisan config:clear`
3. Perbarui keempat URL di dashboard dengan host tersebut.

URL ngrok gratis berubah setiap kali dijalankan ulang, jadi langkah 1–3 harus diulang tiap sesi.

### Menguji webhook tanpa membayar

`Settings` → `Configuration` → tombol **Send test notification**.

Atau kirim manual — perhatikan tanda tangannya harus dihitung dengan Server Key yang sama persis:

```
ORDER_ID="ORD-A1B2C3D4-45"
STATUS_CODE="200"
GROSS_AMOUNT="575000.00"
SERVER_KEY="SB-Mid-server-xxxxxxxxxxxxxxxxxxxx"

SIGNATURE=$(printf '%s%s%s%s' "$ORDER_ID" "$STATUS_CODE" "$GROSS_AMOUNT" "$SERVER_KEY" | sha512sum | cut -d' ' -f1)

curl -X POST https://a1b2c3d4.ngrok-free.app/api/v1/payments/webhook \
  -H "Content-Type: application/json" \
  -d "{
    \"order_id\": \"$ORDER_ID\",
    \"status_code\": \"$STATUS_CODE\",
    \"gross_amount\": \"$GROSS_AMOUNT\",
    \"signature_key\": \"$SIGNATURE\",
    \"transaction_status\": \"settlement\",
    \"fraud_status\": \"accept\",
    \"transaction_id\": \"test-trx-001\"
  }"
```

Dua hal yang mudah keliru:

- `gross_amount` harus **sama persis** dengan yang dikirim Midtrans, termasuk dua angka desimal (`575000.00`, bukan `575000`). Tanda tangan dihitung dari string mentahnya.
- `order_id` harus berakhiran `-{payment_id}` yang benar-benar ada di basis data, karena `PaymentDeepLink::paymentIdFrom()` memotong di tanda hubung terakhir.

### Kartu uji sandbox

| Skenario | Nomor kartu | CVV | Exp | OTP |
|---|---|---|---|---|
| Berhasil | `4811 1111 1111 1114` | `123` | `01/25` | `112233` |
| Ditolak | `4911 1111 1111 1113` | `123` | `01/25` | `112233` |

Daftar lengkap: https://docs.midtrans.com/docs/testing-payment-on-sandbox (https://docs.midtrans.com/docs/testing-payment-on-sandbox)

## Naik ke Produksi

1. **Ganti dashboard** ke `https://dashboard.midtrans.com` (tanpa `sandbox`).
2. **Ambil kunci produksi** — berbeda dari sandbox, tanpa awalan `SB-`.
3. **Perbarui via panel admin**, termasuk menyalakan sakelar **Production Mode**. Sakelar ini yang menentukan `Config::$isProduction` (`app/Support/MidtransSnap.php:77-83`); salah setel berarti transaksi dikirim ke lingkungan yang keliru dan seluruhnya gagal.
4. **Isi ulang keempat URL** di dashboard produksi dengan domain produksi.
5. **Perbarui `APP_URL`** lalu `php artisan config:clear`.
6. **Pastikan sertifikat TLS sah.** Sertifikat kedaluwarsa atau self-signed membuat Midtrans gagal mengirim webhook, dan pembayaran akan menggantung tanpa gejala di sisi pengguna.
7. **Uji satu transaksi kecil sungguhan** sebelum diumumkan.

## Bila Bermasalah

| Gejala | Kemungkinan penyebab | Cara memastikan |
|---|---|---|
| Pembayaran lunas tapi status tetap `pending` | Notification URL salah / tidak terjangkau | `Settings` → `Configuration` → tab riwayat notifikasi; cek `storage/logs/laravel.log` |
| Log berisi `signature mismatch` | Server Key di aplikasi ≠ Server Key dashboard, atau sandbox tercampur produksi | Bandingkan Server Key di panel admin dengan `Access Keys` |
| Log berisi `amount mismatch` | `orders.total` berubah setelah transaksi dibuat | Bandingkan `payments.amount` dengan `gross_amount` di payload |
| Setelah bayar, browser mendarat di `localhost` | `APP_URL` masih nilai lokal | `php artisan tinker` → `config('app.url')` |
| Aplikasi Flutter tidak terbuka setelah bayar | Deep link belum terpasang | Lihat T-12 di `remediation.md` |
| Booking kedaluwarsa padahal sudah dibayar | Scheduler mati **atau** webhook tak pernah sampai | Cek cron; cari `left paid` di log |
| Snap menampilkan semua metode padahal pengguna memilih QRIS | Metadata/kode metode tidak terpetakan | Cek baris di `payment_methods`, cocokkan dengan `MidtransSnap::DEFAULT_CHANNELS` |
| Tombol "Lanjutkan Bayar" tidak muncul di booking detail | `host_confirmed = false` — host belum mengkonfirmasi | Login sebagai host → Manajemen Order → Terima |
| Tombol pembayaran hilang meski status pending | Booking sudah expired (`check_out` terlewat) — tombol disembunyikan otomatis | Cek tanggal `check_out` booking; jika lewat, buat booking baru |
| Booking baru ditolak dengan "Tanggal sudah dipesan" | Ada booking aktif lain yang overlap tanggal yang sama | Cek `bookings` table untuk property_id tersebut |
| Payment expired tapi booking masih waiting_confirmation | `payments:cancel-expired` belum dijalankan | Jalankan `php artisan payments:cancel-expired` |

### Tempat melihat log

```
tail -f storage/logs/laravel.log | grep -i midtrans
```

Yang penting dicari:

- `Midtrans webhook rejected: signature mismatch` → kunci tidak cocok
- `Midtrans webhook rejected: amount mismatch` → nominal berubah
- `Midtrans transaction flagged for manual review` → `fraud_status = challenge`, perlu approve/deny manual di dashboard
- `Midtrans reported failure for an already-paid payment` → notifikasi datang tidak berurutan; sudah ditangani, tapi layak diperiksa

## Rujukan

- Dashboard sandbox — https://dashboard.sandbox.midtrans.com (https://dashboard.sandbox.midtrans.com)
- Dokumentasi Snap — https://docs.midtrans.com/docs/snap-snap-integration-guide (https://docs.midtrans.com/docs/snap-snap-integration-guide)
- HTTP Notification — https://docs.midtrans.com/docs/https-notification-webhooks (https://docs.midtrans.com/docs/https-notification-webhooks)
- SDK PHP — https://github.com/Midtrans/midtrans-php (https://github.com/Midtrans/midtrans-php)

<!-- Source: features/referral.md -->

# Referral System Documentation

Dokumentasi sistem referral pada Livirooms, meliputi database schema, API endpoints, implementasi Flutter, alur kerja, status implementasi, dan rekomendasi teknis.

## Daftar Isi


- Database Schema
- Alur Kerja Referral
- API Endpoints
- Implementasi Flutter
- Status Implementasi
- Rekomendasi Teknis
- Catatan Penting
- Referensi File


## Database Schema

### Tabel `referrals`

Path migration: `laravel/database/migrations/2026_07_14_081700_create_referrals_table.php`

```
Schema::create('referrals', function (Blueprint $table) {
    $table->id();
    $table->foreignId('referrer_id')->constrained('users')->cascadeOnDelete();
    $table->foreignId('referred_id')->constrained('users')->cascadeOnDelete();
    $table->string('referral_code');
    $table->decimal('reward_amount', 15, 2)->nullable();
    $table->enum('status', ['pending','rewarded'])->default('pending');
    $table->timestamps();

    $table->unique(['referrer_id', 'referred_id']);
});
```

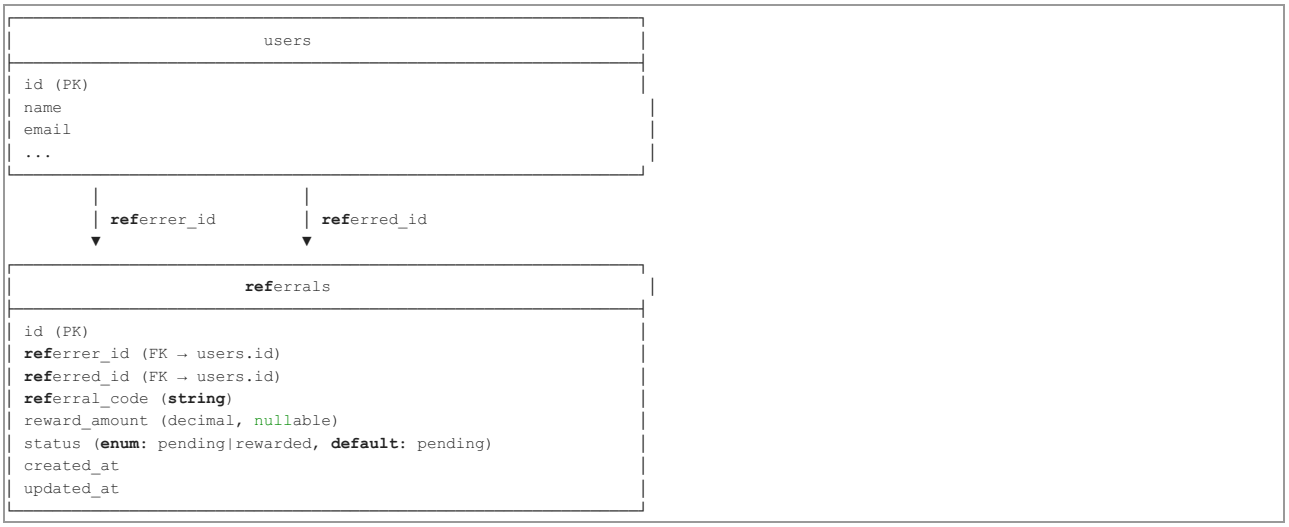
### Relasi Model

```
// App\Models\Referral
class Referral extends Model
{
    protected $fillable = [
        'referrer_id',
        'referred_id',
        'referral_code',
        'reward_amount',
        'status',
    ];

    public function referrer()
    {
        return $this->belongsTo(User::class, 'referrer_id');
    }

    public function referred()
    {
        return $this->belongsTo(User::class, 'referred_id');
    }
}
```

### ER Diagram




---

## Alur Kerja Referral

### Flow 1: Admin Membuat Referral

```
1. Admin login ke panel admin
2. Admin navigasi ke menu "Referral System"
3. Admin klik "Add Referral System"
4. Modal Livewire muncul dengan form:
   - Pilih Pemberi Referral (User A)
   - Pilih Penerima Referral (User B)
   - Input Kode Referral (misal: REF-ANDI-2026)
   - Input Jumlah Reward (misal: Rp50.000)
   - Pilih Status (pending / rewarded)
5. Admin submit form
6. Livewire validasi dan simpan ke database
7. DataTable refresh otomatis
```

### Flow 2: User Membagi Kode Referral

```
1. User A (referrer) login ke aplikasi Flutter
2. User A navigasi ke halaman Referral (Screen 23)
3. Halaman menampilkan:
   - Kode referral milik User A (misal: REF-ANDI-2026)
   - Jumlah referral yang berhasil
   - Total reward yang didapat
4. User A klik "Bagikan Kode"
5. Flutter membuka share dialog (WhatsApp, Telegram, dll) atau copy to clipboard
6. User B menerima kode referral
```

#### Flow 3: User Baru Mendaftar dengan Kode Referral

```
1. User B buka aplikasi Livirooms
2. User B navigasi ke halaman Register
3. User B input data diri (nama, email, password, kode referral opsional)
4. Flutter kirim POST /api/v1/auth/register dengan field referral_code
5. Backend verifikasi:
   - Kode referral ada dan status pending
   - User B bukan user yang membuat kode tersebut
   - User B belum pernah direferensikan sebelumnya
6. Backend buat user baru + otomatis apply referral jika kode valid:
   - referrer_id: id User A
   - referred_id: id User B
   - referral_code: REF-ANDI-2026
   - reward_amount: 50000 (dari admin)
   - status: pending
7. User B berhasil mendaftar dan login
```

#### Flow 4: Reward Diberikan

```
1. User B menyelesaikan booking pertama dan pembayaran berhasil
2. Webhook Midtrans memicu update status booking → confirmed
3. BookingObserver atau Event Listener mendeteksi booking pertama User B
4. Sistem cari referral dengan referred_id = User B dan status = pending
5. Jika ditemukan:
   - Update status → rewarded
   - Update reward_amount sesuai aturan admin
   - Notifikasi dikirim ke User A (push notification / in-app)
   - Saldo/credit User A ditambah (jika ada sistem wallet)
6. Jika tidak ditemukan, tidak ada aksi
```

---

## API Endpoints

### Endpoint Admin

| Method | Endpoint | Fungsi |
|---|---|---|
| GET | `/admin/referrals` | DataTable admin (Yajra) |
| GET | `/admin/referrals/create` | Form create admin |
| GET | `/admin/referrals/{id}/edit` | Form edit admin |
| DELETE | `/admin/referrals/{id}` | Hapus referral |

### Endpoint Mobile API

Path: `laravel/routes/api_mobile.php`

```php
Route::middleware('auth:sanctum')->prefix('referrals')->name('referrals.')->group(function () {
    Route::get('/my-code', [App\Http\Controllers\Api\ReferralController::class, 'myCode'])->name('my-code');
    Route::get('/my-list', [App\Http\Controllers\Api\ReferralController::class, 'myList'])->name('my-list');
    Route::post('/apply', [App\Http\Controllers\Api\ReferralController::class, 'apply'])->name('apply');
    Route::get('/status', [App\Http\Controllers\Api\ReferralController::class, 'status'])->name('status');
});
```

Path: `laravel/app/Http/Controllers/Api/ReferralController.php`

- `GET /api/v1/referrals/my-code` — ambil kode referral user yang login
- `GET /api/v1/referrals/my-list` — ambil daftar referral + statistik
- `POST /api/v1/referrals/apply` — terapkan kode referral
- `GET /api/v1/referrals/status` — cek status referral user yang login

---

## Implementasi Flutter

### Model

Path: `flutter/lib/models/referral.dart`

```dart
class Referral {
  final int id;
  final String referralCode;
  final double? rewardAmount;
  final String status;
  final String? referredName;
  final String? referredEmail;
  final String? createdAt;
```

```
  const Referral({
    required this.id,
    required this.referralCode,
    this.rewardAmount,
    required this.status,
    this.referredName,
    this.referredEmail,
    this.createdAt,
  });

  factory Referral.fromJson(Map<String, dynamic> json) => Referral(
        id: json['id'] as int? ?? 0,
        referralCode: json['referral_code'] as String? ?? '',
        rewardAmount: json['reward_amount'] is num
            ? (json['reward_amount'] as num).toDouble()
            : double.tryParse('${json['reward_amount']}') ?? 0,
        status: json['status'] as String? ?? 'pending',
        referredName: json['referred_name'] as String?,
        referredEmail: json['referred_email'] as String?,
        createdAt: json['created_at'] as String?,
      );
}

class ReferralStats {
  final int totalReferred;
  final int totalRewarded;
  final int totalPending;
  final double totalRewardAmount;

  const ReferralStats({
    required this.totalReferred,
    required this.totalRewarded,
    required this.totalPending,
    required this.totalRewardAmount,
  });

  factory ReferralStats.fromJson(Map<String, dynamic> json) => ReferralStats(
        totalReferred: json['total_referred'] as int? ?? 0,
        totalRewarded: json['total_rewarded'] as int? ?? 0,
        totalPending: json['total_pending'] as int? ?? 0,
        totalRewardAmount: json['total_reward_amount'] is num
            ? (json['total_reward_amount'] as num).toDouble()
            : double.tryParse('${json['total_reward_amount']}') ?? 0,
      );
}

class ReferralCodeResponse {
  final String code;
  final int userId;
  final String name;

  const ReferralCodeResponse({
    required this.code,
    required this.userId,
    required this.name,
  });

  factory ReferralCodeResponse.fromJson(Map<String, dynamic> json) =>
      ReferralCodeResponse(
        code: json['code'] as String? ?? '',
        userId: json['user_id'] as int? ?? 0,
        name: json['name'] as String? ?? '',
      );
}

class ReferralStatusResponse {
  final bool hasReferral;
  final int? referralId;
  final String? referralCode;
  final String? referrerName;
  final double? rewardAmount;
  final String? status;
  final String? createdAt;

  const ReferralStatusResponse({
    required this.hasReferral,
    this.referralId,
    this.referralCode,
    this.referrerName,
    this.rewardAmount,
    this.status,
    this.createdAt,
  });

  factory ReferralStatusResponse.fromJson(Map<String, dynamic> json) =>
      ReferralStatusResponse(
```

```
        hasReferral: json['has_referral'] as bool? ?? false,
        referralId: json['referral_id'] as int?,
        referralCode: json['referral_code'] as String?,
        referrerName: json['referrer_name'] as String?,
        rewardAmount: json['reward_amount'] is num
            ? (json['reward_amount'] as num).toDouble()
            : double.tryParse('${json['reward_amount']}') ?? 0,
        status: json['status'] as String?,
        createdAt: json['created_at'] as String?,
      );
}
```

### Service

Path: `flutter/lib/services/referral_service.dart`

```
import '../models/referral.dart';
import 'api_client.dart';

class ReferralService {
  ReferralService({ApiClient? api}) : _api = api ?? ApiClient();
  final ApiClient _api;

  Future<ReferralCodeResponse> myCode() async {
    final res = await _api.get('referrals/my-code');
    final data = res.dataMap;
    return ReferralCodeResponse.fromJson(data);
  }

  Future<(ReferralStats, List<Referral>)> myList() async {
    final res = await _api.get('referrals/my-list');
    final data = res.dataMap;

    final statsJson = data['stats'] as Map<String, dynamic>? ?? {};
    final stats = ReferralStats.fromJson(statsJson);

    final listJson = data['referrals'] as List<dynamic>? ?? [];
    final referrals = listJson
        .map((e) => Referral.fromJson(e as Map<String, dynamic>))
        .toList();

    return (stats, referrals);
  }

  Future<void> applyCode(String code) async {
    await _api.post('referrals/apply', body: {
      'referral_code': code,
    });
  }

  Future<ReferralStatusResponse> status() async {
    final res = await _api.get('referrals/status');
    final data = res.dataMap;
    return ReferralStatusResponse.fromJson(data);
  }
}
```

### Screen: Referral Screen

Path: `flutter/lib/screens/account/referral_screen.dart`

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'package:share_plus/share_plus.dart';

import '../../config/theme.dart';
import '../../models/referral.dart';
import '../../providers/auth_provider.dart';
import '../../router.dart';
import '../../services/referral_service.dart';

class ReferralScreen extends StatefulWidget {
  const ReferralScreen({super.key});

  @override
  State<ReferralScreen> createState() => _ReferralScreenState();
}

class _ReferralScreenState extends State<ReferralScreen> {
  final _service = ReferralService();
  bool _loading = true;
  String? _myCode;
  ReferralStats? _stats;
  List<Referral> _referrals = [];

  @override
  void initState() {
```

```
    super.initState();
    _loadData();
  }

  Future<void> _loadData() async {
    setState(() => _loading = true);
    try {
      final codeRes = await _service.myCode();
      final (stats, list) = await _service.myList();
      if (!mounted) return;
      setState(() {
        _myCode = codeRes.code;
        _stats = stats;
        _referrals = list;
      });
    } on ApiException catch (e) {
      if (!mounted) return;
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text(e.message)),
      );
    } catch (_) {
      if (!mounted) return;
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('Gagal memuat data referral.')),
      );
    } finally {
      if (mounted) setState(() => _loading = false);
    }
  }

  Future<void> _shareCode() async {
    if (_myCode == null) return;
    final user = context.read<AuthProvider>().user;
    final text =
        'Ayo Livirooms properti Livirooms! Gunakan kode referral saya $_myCode dan dapatkan reward bersama. Download app di
https://livirooms.app';
    await Share.share(text);
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Referral'),
      ),
      body: _loading
          ? const Center(child: CircularProgressIndicator())
          : ListView(
              padding: const EdgeInsets.all(16),
              children: [
                _buildCodeCard(),
                const SizedBox(height: 16),
                _buildStatsCard(),
                const SizedBox(height: 16),
                _buildReferralList(),
              ],
            ),
    );
  }

  Widget _buildCodeCard() {
    return Container(
      padding: const EdgeInsets.all(20),
      decoration: BoxDecoration(
        color: AppColors.navy,
        borderRadius: BorderRadius.circular(16),
      ),
      child: Column(
        children: [
          const Text(
            'Kode Referral Saya',
            style: TextStyle(color: Colors.white70, fontSize: 14),
          ),
          const SizedBox(height: 8),
          Text(
            _myCode ?? '-',
            style: const TextStyle(
              color: AppColors.gold,
              fontSize: 28,
              fontWeight: FontWeight.w800,
              letterSpacing: 2,
            ),
          ),
          const SizedBox(height: 16),
          ElevatedButton.icon(
            onPressed: _shareCode,
```

```
          icon: const Icon(Icons.share_outlined),
          label: const Text('Bagikan Kode'),
          style: ElevatedButton.styleFrom(
            backgroundColor: AppColors.gold,
            foregroundColor: AppColors.navy,
          ),
        ),
      ],
    ),
  );
}

Widget _buildStatsCard() {
  final stats = _stats;
  if (stats == null) return const SizedBox.shrink();

  return Container(
    padding: const EdgeInsets.all(16),
    decoration: BoxDecoration(
      color: AppColors.surface,
      borderRadius: BorderRadius.circular(16),
      border: Border.all(color: AppColors.border),
    ),
    child: Row(
      mainAxisAlignment: MainAxisAlignment.spaceAround,
      children: [
        _statItem('Total Direferensikan', stats.totalReferred.toString()),
        _statItem('Berhasil', stats.totalRewarded.toString()),
        _statItem('Total Reward', Formatters.rupiah(stats.totalRewardAmount)),
      ],
    ),
  );
}

Widget _statItem(String label, String value) {
  return Column(
    children: [
      Text(
        value,
        style: const TextStyle(
          fontSize: 16,
          fontWeight: FontWeight.w800,
          color: AppColors.textPrimary,
        ),
      ),
      const SizedBox(height: 4),
      Text(
        label,
        style: const TextStyle(
          fontSize: 12,
          color: AppColors.textSecondary,
        ),
        textAlign: TextAlign.center,
      ),
    ],
  );
}

Widget _buildReferralList() {
  if (_referrals.isEmpty) {
    return Container(
      padding: const EdgeInsets.all(24),
      decoration: BoxDecoration(
        color: AppColors.surface,
        borderRadius: BorderRadius.circular(16),
        border: Border.all(color: AppColors.border),
      ),
      child: const Center(
        child: Text(
          'Belum ada referral. Bagikan kode referral Anda sekarang!',
          textAlign: TextAlign.center,
          style: TextStyle(color: AppColors.textSecondary),
        ),
      ),
    );
  }

  return Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      const Text(
        'Daftar Referral',
        style: TextStyle(
          fontSize: 16,
          fontWeight: FontWeight.w700,
          color: AppColors.textPrimary,
```

```
          ),
        ),
        const SizedBox(height: 12),
        ..._referrals.map((r) => Container(
              margin: const EdgeInsets.only(bottom: 8),
              padding: const EdgeInsets.all(12),
              decoration: BoxDecoration(
                color: AppColors.surface,
                borderRadius: BorderRadius.circular(12),
                border: Border.all(color: AppColors.border),
              ),
              child: Row(
                children: [
                  Expanded(
                    child: Column(
                      crossAxisAlignment: CrossAxisAlignment.start,
                      children: [
                        Text(
                          r.referredName ?? r.referredEmail ?? 'User #${r.id}',
                          style: const TextStyle(
                            fontWeight: FontWeight.w600,
                            color: AppColors.textPrimary,
                          ),
                        ),
                        const SizedBox(height: 4),
                        Text(
                          r.status == 'rewarded'
                              ? 'Reward: ${Formatters.rupiah(r.rewardAmount ?? 0)}'
                              : 'Menunggu transaksi pertama',
                          style: TextStyle(
                            fontSize: 12,
                            color: r.status == 'rewarded'
                                ? AppColors.success
                                : AppColors.textSecondary,
                          ),
                        ),
                      ],
                    ),
                  ),
                  Container(
                    padding: const EdgeInsets.symmetric(
                      horizontal: 8,
                      vertical: 4,
                    ),
                    decoration: BoxDecoration(
                      color: r.status == 'rewarded'
                          ? AppColors.success.withValues(alpha: 0.1)
                          : AppColors.gold.withValues(alpha: 0.1),
                      borderRadius: BorderRadius.circular(8),
                    ),
                    child: Text(
                      r.status == 'rewarded' ? 'Berhasil' : 'Pending',
                      style: TextStyle(
                        color: r.status == 'rewarded'
                            ? AppColors.success
                            : AppColors.gold,
                        fontSize: 12,
                        fontWeight: FontWeight.w600,
                      ),
                    ),
                  ),
                ],
              ),
            )),
      ],
    );
  }
}
```

### Router

Path: `flutter/lib/router.dart`

**Tambahkan route constant:**

```
static const referral = '/referral';
```

**Tambahkan case di `onGenerateRoute`:**

```
case Routes.referral:
  page = const ReferralScreen();
  break;
```

---

## Status Implementasi

### Yang Sudah Diimplementasikan

| Komponen | File | Deskripsi |
|---|---|---|
| **Model** | `app/Models/Referral.php` | Eloquent model dengan relasi `referrer()` dan `referred()` ke User |
| **Migration** | `database/migrations/2026_07_14_081700_create_referrals_table.php` | Tabel `referrals` dengan foreign key, enum status, nullable reward_amount |
| **Migration (nullable)** | `database/migrations/2026_08_09_171700_make_referred_id_nullable_on_referrals_table.php` | Membuat `referred_id` nullable agar kode referral bisa dibuat sebelum user baru mendaftar |
| **Seeder** | `database/seeders/ReferralSeeder.php` | Sample data: Andi → Dewi, kode `REF-ANDI-2026`, reward Rp50.000, status `rewarded` |
| **DataTable** | `app/DataTables/ReferralDataTable.php` | Yajra DataTable dengan kolom: ID, Referrer, Direferensikan, Kode Referral, Reward, Status, Aksi |
| **Controller (Admin)** | `app/Http/Controllers/Admin/ReferralController.php` | Admin CRUD: `index`, `create`, `edit`, `destroy` |
| **Livewire Form** | `app/Livewire/Referrals/Form.php` | Form modal untuk create/edit referral dengan validasi |
| **Livewire Modal** | `app/Livewire/Referrals/ReferralModal.php` | Modal wrapper untuk form Livewire |
| **Views (Admin)** | `resources/views/admin/referrals/index.blade.php` | Halaman index dengan DataTable + tombol Add |
| **API Controller (Mobile)** | `app/Http/Controllers/Api/ReferralController.php` | 4 endpoint: `myCode`, `myList`, `apply`, `status` |
| **API Routes (Mobile)** | `routes/api_mobile.php` | Route group `referrals` dengan middleware `auth:sanctum` |
| **Flutter Model** | `lib/models/referral.dart` | Model `Referral`, `ReferralStats`, `ReferralCodeResponse`, `ReferralStatusResponse` |
| **Flutter Service** | `lib/services/referral_service.dart` | Service layer untuk API referral |
| **Flutter Screen** | `lib/screens/account/referral_screen.dart` | Halaman referral: kode, stats, daftar referral, copy & share |
| **Flutter Router** | `lib/router.dart` | Route constant `Routes.referral` |
| **Flutter Menu** | `lib/screens/account/account_menu_screen.dart` | Menu item Referral di Account Menu |

### Yang Belum Diimplementasikan

| # | Fitur yang Belum | Keterangan |
|---|---|---|
| 1 | **Notifikasi referral (push/in-app)** | NotificationLog sudah dibuat oleh BookingObserver, tapi belum ada UI Flutter untuk menampilkan notifikasi referral |
| 2 | **Flutter tests untuk referral** | ❌ Belum — tidak ada test file untuk model, service, atau screen referral |

---

## Rekomendasi Teknis

### Prioritas Tinggi

1. **Tambahkan API Endpoints untuk Mobile** — Buat `ReferralController` di `app/Http/Controllers/Api/`, daftarkan route di `routes/api_mobile.php`, semua endpoint wajib pakai `auth:sanctum`.
2. **Auto-generate Referral Code** — Buat observer/listener pada `User` model saat user dibuat. Generate kode unik: `REF-{INITIALS}-{USER_ID}` atau `REF-{RANDOM_6_CHAR}`. Simpan kode di field `referral_code` di tabel `referrals` atau tambahkan kolom `referral_code` di tabel `users`.
3. **Implementasi Flutter Referral Screen** — Buat screen `ReferralScreen` dengan kode, stats, dan daftar referral. Integrasi `share_plus` untuk share kode. Tambahkan menu di `AccountMenuScreen`.

### Prioritas Menengah

4. **Auto-reward Logic** — Buat `BookingObserver` atau event listener. Deteksi booking pertama user (referred_id). Update status referral → `rewarded`. Trigger notifikasi ke referrer.
5. **Validasi Unique Constraint** — Tambahkan unique constraint `unique(['referrer_id', 'referred_id'])` di migration. Validasi di backend dan Flutter agar user tidak bisa referral diri sendiri.
6. **Referral History** — Tampilkan riwayat referral dengan status dan tanggal. Filter by status (pending/rewarded).

### Prioritas Rendah

7. **Notification System** — Push notification saat referral berhasil. In-app notification di Flutter.
8. **Reward Redemption** — Buat endpoint untuk menukarkan reward menjadi saldo/credit. Integrasi dengan sistem wallet.
9. **Analytics** — Track conversion rate (berapa yang daftar vs berhasil bayar). Track top referrers.

---

## Catatan Penting

- **Kode Referral**: Saat ini kode referral di-generate otomatis saat user dibuat via `UserObserver`. Format: `REF-{INITIALS}-{USER_ID}` dengan fallback suffix jika ada duplikat.
- **Reward Logic**: Auto-reward dijalankan oleh `BookingObserver` saat booking status berubah ke `confirmed` atau `completed`. NotificationLog dibuat untuk referrer.
- **Status**: Enum `pending|rewarded` cukup untuk tahap awal. Untuk production, pertimbangkan menambah status `cancelled` atau `expired`.
- **Unique Constraint**: Kolom `referral_code` sudah di-unique-kan via migration terpisah.
- **Security**: Pastikan endpoint API hanya bisa diakses oleh user yang terautentikasi (`auth:sanctum`).
- **Flutter Package**: Untuk share, gunakan package `share_plus`. Tambahkan ke `pubspec.yaml` jika belum ada.
- **Testing**: Laravel tests untuk referral API sudah ada (10 tests).

---

## Referensi File

### Backend (Laravel)

| File | Fungsi |
|---|---|
| app/Models/Referral.php | Model Eloquent Referral |
| database/migrations/2026_07_14_081700_create_referrals_table.php | Migration tabel referrals |
| database/migrations/2026_08_09_171700_make_referred_id_nullable_on_referrals_table.php | Membuat referred_id nullable |
| database/migrations/2026_08_09_165632_add_unique_referral_code_to_referrals_table.php | Unique constraint untuk referral_code |
| database/seeders/ReferralSeeder.php | Seeder sample data referral |
| app/DataTables/ReferralDataTable.php | Yajra DataTable untuk admin |
| app/Http/Controllers/Admin/ReferralController.php | Admin controller (CRUD) |
| app/Http/Controllers/Api/ReferralController.php | API controller untuk mobile (myCode, myList, apply, status) |
| app/Observers/UserObserver.php | Auto-generate kode referral saat user dibuat |
| app/Observers/BookingObserver.php | Auto-reward referral + buat NotificationLog saat booking confirmed/completed |
| app/Providers/AppServiceProvider.php | Registrasi observer User & Booking |
| app/Livewire/Referrals/Form.php | Livewire form modal create/edit |
| app/Livewire/Referrals/ReferralModal.php | Livewire modal wrapper |
| resources/views/admin/referrals/index.blade.php | View admin index |
| resources/views/admin/referrals/create.blade.php | View admin create |
| resources/views/admin/referrals/edit.blade.php | View admin edit |
| resources/views/admin/referrals/columns/_actions.blade.php | Kolom aksi DataTable |
| resources/views/livewire/referrals/form.blade.php | Form modal Livewire |

### Frontend (Flutter)

| File | Fungsi |
|---|---|
| lib/models/referral.dart | Model Referral, ReferralStats, ReferralCodeResponse, ReferralStatusResponse |
| lib/services/referral_service.dart | Service layer untuk API referral |
| lib/screens/account/referral_screen.dart | Screen halaman referral: kode, stats, daftar referral, copy & share |
| lib/router.dart | Route constant Routes.referral |
| test/models/referral_test.dart | Unit test model referral |
| test/services/referral_service_test.dart | Unit test service referral |

<!-- Source: features/account-menu.md -->

# Dokumentasi Fitur: Account Menu Screen (Screen 14)

**File Path:** `flutter/lib/screens/account/account_menu_screen.dart`
**Route:** `Routes.account` (`/account`) & Embedded Tab di `MainShell` (Tab Index 4)
**Tipe:** Consumer Account & Profile Navigation Hub

---

## 1. Ringkasan Desain & Layout

Account Menu Screen telah diperbarui dengan desain modern bernuansa *Trip.com style* yang bersih, elegan, dan fungsional. Layar ini berfungsi sebagai pusat navigasi profil pengguna, loyalty/rewards, status pemesanan, pengaturan, serta portal pendaftaran dan peralihan ke Mode Mitra (Host).

---

## 2. Struktur Komponen UI

### A. Header Profil & Quick Actions

- **Avatar:** Lingkaran kuning lembut (`#FEF3D6`) dengan ikon siluet pengguna amber (`#F5A623`).
- **Judul:** Teks "`Halo Anggota Trip.com`" (atau nama pengguna aktif).
- **Sub-aksi:** Tombol interaktif "`Kelola Akun Saya >`" yang menavigasi ke `Routes.editProfile` (`EditProfileScreen`).
- **Aksi Cepat Kanan:**
    - **Ikon Pindai (Scan QR):** Menampilkan modal informasi kesiapan pemindaian QR tiket dan check-in mandiri.
    - **Ikon Bantuan (Headset):** Menavigasi ke pusat bantuan `Routes.help` (`HelpSupportScreen`).
    - **Ikon Pengaturan (Gear):** Membuka bottom sheet pengaturan lengkap (Edit Profil, Program Referral, Pengaturan Keamanan, Notifikasi, Bahasa, dan Keluar dari Akun).

### B. Carousel Kartu Keuntungan Anggota (Member Benefits)

Carousel horizontal dengan kartu bergaya minimalis:

1. **Promo Member Hotel:** Ikon hotel dengan badge diskon `%`, menavigasi ke daftar promo/diskon (`Routes.myDiscount`).
2. **Jaminan Pembatalan:** Ikon bagasi perjalanan dengan badge verifikasi jaminan fleksibilitas pembatalan.
3. **Selengkapnya:** Tombol panah melingkar untuk melihat semua keuntungan loyalty anggota.

### C. Kartu Trip Coins & Kode Promo

Kartu kontainer dengan pemisah vertikal:

- **Sisi Kiri (Trip Coins):** Ikon koin emas '`T`', nominal saldo ≈`Rp0 (0 Trip Coins)`, dan panduan penghematan.
- **Sisi Kanan (Kode Promo):** Badge merah muda dengan ikon `%`, jumlah voucher aktif, dan tautan langsung ke `Routes.promo` (`PromoCodeScreen`).

### D. Grup Menu Utama (Travel & Akun)

1. **Semua Pemesanan:** Ikon dokumen/kalender biru pada kontainer biru muda (`#EEF4FF`), mengarah ke `Routes.bookingList`.
2. **Perjalanan Mendatang:** Ikon koper biru muda, mengarah ke filter pesanan aktif di `Routes.bookingList`.
3. **Terakhir Dilihat:** Ikon riwayat produk/properti, mengarah ke `Routes.viewedProducts` (`ViewedProductsScreen`).
4. **Kartu Saya:** Ikon kartu kredit outline, mengarah ke penambahan/pengelolaan kartu pembayaran di `Routes.addCard`.
5. **Traveler & Kontak:** Ikon profil outline, mengarah ke data traveler di `Routes.editProfile`.
6. **Disimpan:** Ikon hati (favorit), mengarah ke wishlist di `Routes.wishlist` (`WishlistScreen`).

#### E. Banner & Pusat Kreator (Trip Moments)

- **Banner Trip Moments:** Desain latar biru muda pastel (`#F1F6FE`) dengan teks ajakan membagikan momen perjalanan untuk reward 25 Trip Coins, dilengkapi tombol aksi "`Mulai`".
- **Menu Pusat Kreator:** Item menu dengan ikon pena/sparkle untuk ulasan dan komunitas kreator.

#### F. Bagian Tentang & Ketentuan

- **Tentang Trip.com:** Dialog informasi aplikasi, versi, dan layanan.
- **Beri nilai Aplikasi:** Dialog penilaian bintang interaktif untuk ulasan pengguna.
- **Syarat & Ketentuan:** Dialog ketentuan hukum, privasi, dan standar operasional.

#### G. Kartu Kemitraan (Partner / Host Card)

- **Status Non-Host:** Menampilkan ajakan "`Bergabunglah bersama Trip.com sebagai mitra...`" dengan tombol outline "`Mulai Sekarang`". Menjalankan fungsi `_becomeHost()` untuk validasi dan onboarding mitra.
- **Status Host:** Menampilkan informasi status aktif mitra dan tombol "`Beralih ke Mode Mitra`" yang mengarahkan ke `Routes.mitraHome` (`MitraShell`).
- **Ilustrasi Kanan:** Grafis melingkar berisi ikon jabat tangan (handshake) dan pin lokasi/globe.

#### H. Footer Kepercayaan (Trust Badges)

- Teks utama: "`Layanan yang bisa Anda andalkan`".
- Poin keunggulan:
    - ✔ `Bantuan global 30 detik`
    - ✔ `Travel Bebas Khawatir`

---

## 3. Integrasi State & Service

| Fitur | Sumber Data / Service | Handler |
|---|---|---|
| Autentikasi Pengguna | `AuthProvider.user` | Menampilkan nama dan avatar foto/ikon |
| Upload Foto Profil | `ProfileService.uploadAvatar()` | `_pickAndUploadAvatar()` via Kamera/Galeri (`image_picker`) |
| Pull-to-Refresh Profil | `ProfileService.profile()` | `_refreshProfile()` memperbarui AuthProvider |
| Status Host / Mitra | `AuthProvider.isHost` | Kondisional kartu mitra / beralih mode |
| Onboarding Mitra | `ProfileService.becomeHost()` | `_becomeHost()` |
| Validasi Mitra | `ProfileService.validationStatus()` | `Routes.validation` jika belum tervalidasi |
| Keluar Akun | `AuthProvider.logout()` | `_logout()` dengan konfirmasi dialog |

---

## 4. Dokumentasi Fitur: Profil Pengguna (Screen 15 & 23)

**File Path:** `flutter/lib/screens/account/edit_profile_screen.dart`
**Route:** `Routes.editProfile` (`/account/edit-profile`)
**Tipe:** Manage User Account & Personal Security Information
**API Endpoint:** `GET /api/v1/profile`, `PUT /api/v1/profile`, `POST /api/v1/profile`, `POST /api/v1/profile/avatar`

#### Struktur Komponen UI:

1. **Avatar Header:** Avatar profil lingkaran dengan foto pengguna (`CachedNetworkImage` dari `user.avatarUrl`) atau fallback ikon siluet amber, dilengkapi badge pensil biru (`#2563EB`). Mengetuk badge pensil memunculkan modal pemilihan sumber foto: **Kamera** atau **Galeri Foto** (`image_picker`). Foto diunggah langsung ke endpoint `POST /api/v1/profile/avatar` via multipart request.
2. **Keamanan Akun:**
    - Email dengan masking keamanan (`mohba*****gorie@gmail.com`).
    - Nomor telepon dengan tombol aksi "`Hubungkan`" / "`Ubah`".
    - Ubah Kata Sandi dengan tombol aksi "`Atur`".
    - Kelola Perangkat & Riwayat Login dengan chevron navigasi.
3. **Pengaturan Pembayaran:**
    - PIN Keamanan status "`Belum diatur >`".
4. **Profil Member (Rewards):**
    - Kartu dengan badge `REWARDS` dan deskripsi pentingnya ID resmi.
    - Box informasi nama resmi dan tanggal lahir dengan tombol aksi "`Edit`".
5. **Info Pribadi:**
    - Header dengan tombol "`Edit`".
    - Data: Nama, Jenis kelamin, Negara atau wilayah, Kota tempat tinggal, dan Kota yang sering dikunjungi.
6. **Aksi Akun & Footer:**
    - Tombol putih lebar "`Keluar`" (Logout).
    - Tautan "`Hapus Akun Saya`" dengan konfirmasi dialog dan peringatan permanen.

<!-- Source: payments/system.md -->

# Payment System Documentation

> **Diverifikasi dari kode pada 2026-08-21.** Setiap baris punya bukti path:baris.

## 1. Ringkasan

Sistem pembayaran menggunakan **Midtrans Snap** sebagai payment gateway. Terdapat 6 endpoint di `PaymentController`:

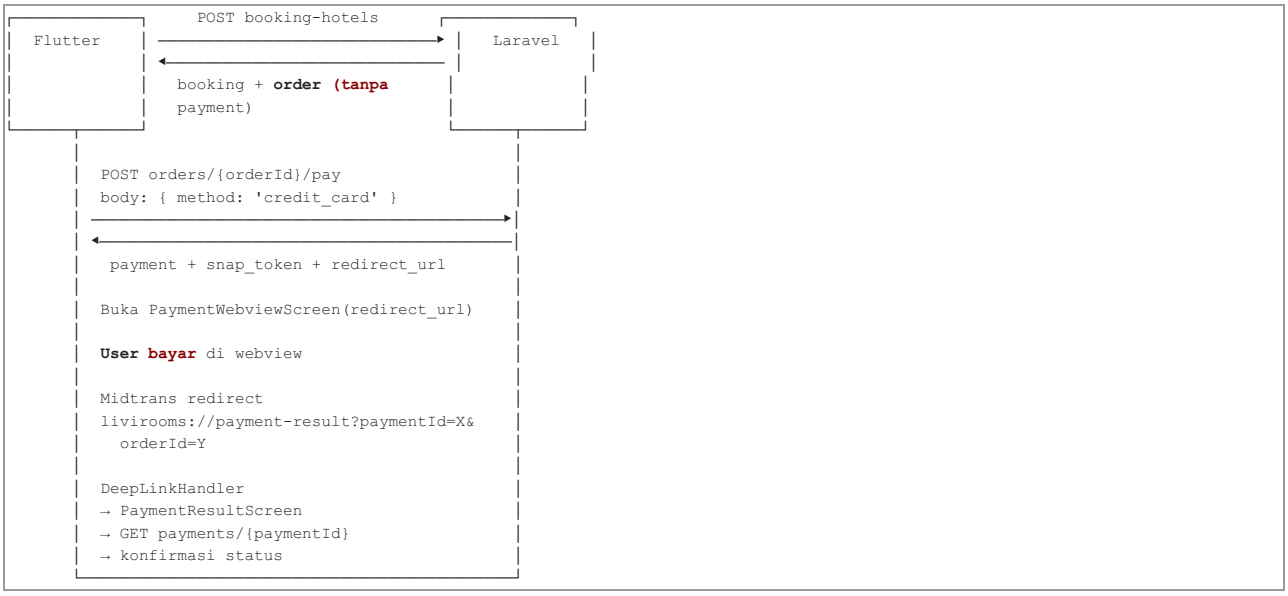
| # | Endpoint | Method | Fungsi |
|---|---|---|---|
| 1 | `GET payments/methods` | Client | List metode pembayaran yang enabled |
| 2 | `POST orders/{order}/pay` | Client | Buat transaksi Midtrans, return `snap_token` + `redirect_url` |
| 3 | `POST orders/{order}/pay/retry` | Client | Retry/continue pembayaran yang sudah ada |
| 4 | `GET payments/{payment}` | Client | Detail pembayaran |
| 5 | `POST payments/{payment}/reconcile` | Client | Sync status pembayaran dengan Midtrans |
| 6 | `POST payments/webhook` | Midtrans S2S | Notifikasi status pembayaran dari Midtrans |

Sistem booking memiliki 2 alur yang terpisah:

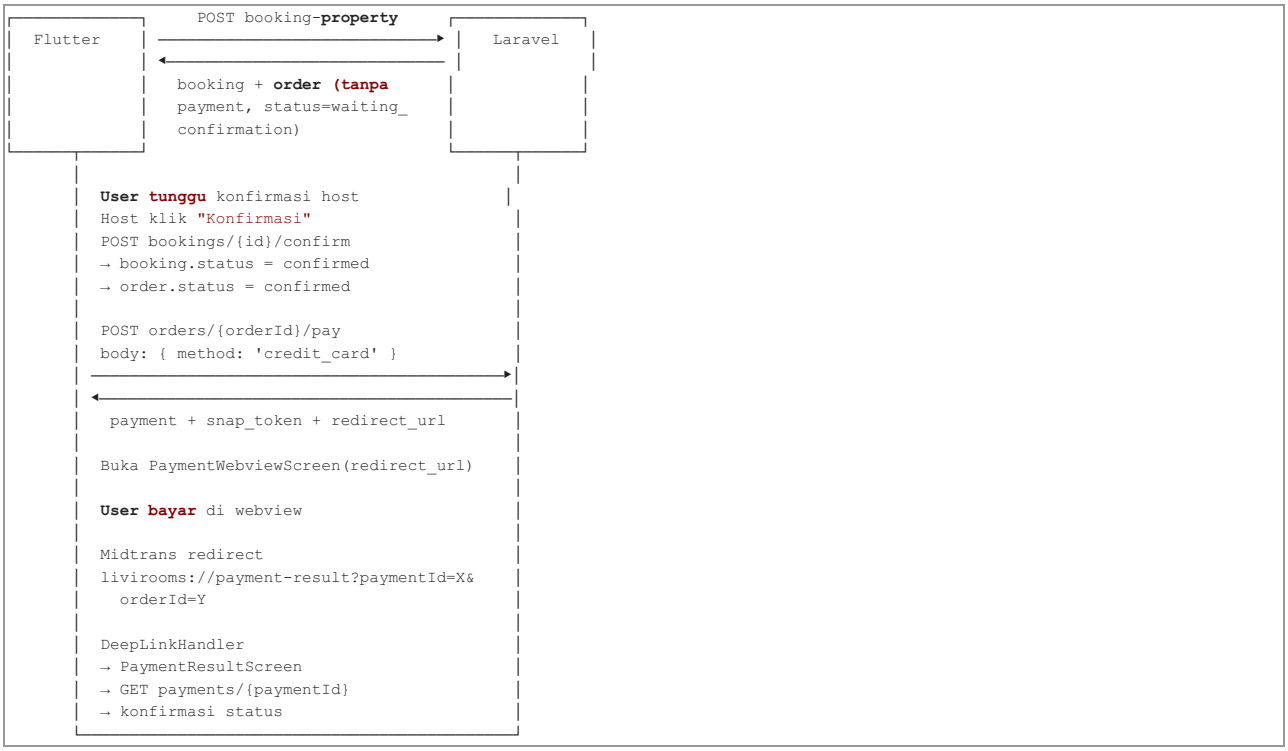
- **Property booking** (`booking_type: unit`) — lewat `POST booking-property`
- **Hotel booking** (`booking_type: room`) — lewat `POST booking-hotels`

Keduanya menghasilkan `Booking` + `Order`, lalu pembayaran dilakukan lewat `POST orders/{order}/pay`.

## 2. Alur Pembayaran Hotel (Booking Mode `room`)




## 3. Alur Pembayaran Property (Booking Mode `unit`)




## 4. Endpoint Detail

### 4.0 Booking Action Endpoints

Semua endpoint aksi dibawah memerlukan auth host (kecuali `cancel` yang dibutuhkan user).

| # | Endpoint | Method | Keterangan |
|---|---|---|---|
| 1 | `POST bookings/{id}/cancel` | User | Batalkan booking (property & hotel) |
| 2 | `POST bookings/{id}/confirm` | Host | Konfirmasi booking property |
| 3 | `POST bookings/{id}/reject` | Host | Tolak booking property |
| 4 | `POST bookings/{id}/checkin` | Host | Catat check-in tamu (status -> `staying`) |
| 5 | `POST bookings/{id}/checkout` | Host | Catat check-out tamu (status -> `completed`) |
| 6 | `POST bookings/{id}/confirm-checkin` | Host | Konfirmasi check-in tamu (property & hotel) |
| 7 | `POST bookings/{id}/review` | User | Beri ulasan setelah booking selesai |

#### 4.0.1 `POST bookings/{id}/cancel`

Membatalkan booking. Hanya bisa dibatalkan jika status masih `waiting_payment`, `waiting_confirmation`, `confirmed`, `paid`, `waiting_checkin`, `staying`, atau `checked_in`. Tidak bisa dibatalkan jika sudah `completed`, `cancelled`, `staying`, `checked_in`, atau `rejected`.

**Response:**

```
{
  "status": true,
  "code": 200,
  "message": "Booking berhasil dibatalkan.",
  "data": {
    "id": 1,
    "status": "cancelled",
    "property": { ... }
  }
}
```

#### 4.0.2 `POST bookings/{id}/confirm`

Host mengkonfirmasi booking property. Mengubah `host_confirmed` menjadi `true` dan `status` menjadi `confirmed`.

**Response:**

```
{
  "status": true,
  "code": 200,
  "message": "Pesanan berhasil dikonfirmasi.",
  "data": {
    "id": 1,
    "status": "confirmed",
    "host_confirmed": true
  }
}
```

#### 4.0.3 `POST bookings/{id}/reject`

Host menolak booking property. Mengubah `status` menjadi `rejected` dan `host_confirmed` menjadi `false`.

**Response:**

```
{
  "status": true,
  "code": 200,
  "message": "Pesanan berhasil ditolak.",
  "data": {
    "id": 1,
    "status": "rejected",
    "host_confirmed": false
  }
}
```

#### 4.0.4 `POST bookings/{id}/checkin`

Host mencatat check-in tamu. Hanya bisa jika status booking adalah `confirmed`, `paid`, atau `waiting_checkin`. Mengubah `status` menjadi `staying`.

**Response:**

```
{
  "status": true,
  "code": 200,
  "message": "Check-in berhasil dicatat.",
  "data": {
    "id": 1,
    "status": "staying"
  }
}
```

#### 4.0.5 `POST bookings/{id}/checkout`

Host mencatat check-out tamu. Hanya bisa jika status booking adalah `staying`. Mengubah status booking menjadi `completed` dan status order menjadi `completed`.

**Response:**

```
{
  "status": true,
  "code": 200,
  "message": "Check-out berhasil dicatat. Status booking telah completed.",
  "data": {
    "id": 1,
    "status": "completed"
  }
}
```

#### 4.0.6 `POST bookings/{id}/confirm-checkin`

Host mengkonfirmasi check-in tamu. Hanya bisa jika status booking adalah `checked_in`. Mengubah `host_confirmed_checkin` menjadi `true`.

**Response:**

```
{
  "status": true,
  "code": 200,
  "message": "Check-in berhasil dikonfirmasi.",
  "data": {
    "id": 1,
    "status": "checked_in",
    "host_confirmed_checkin": true
  }
}
```

#### 4.0.6 `POST bookings/{id}/review`

User memberikan ulasan setelah booking selesai (`status = completed`). Hanya bisa sekali per booking.

**Response:**

```
{
  "status": true,
  "code": 201,
  "message": "Terima kasih, ulasan Anda berhasil dikirim.",
  "data": {
    "id": 1,
    "booking_id": 1,
    "property_id": 1,
    "user_id": 2,
    "rating": 5,
    "comment": "Properti sangat bagus dan bersih!",
    "host_reply": "Terima kasih atas review positifnya!",
    "photos": ["https://example.com/reviews/1/1.jpg"],
    "created_at": "2026-07-20T14:30:00+07:00",
    "user": {
      "id": 2,
      "name": "Siti Nurhaliza"
    }
  }
}
```


### 4.1 `GET payments/methods`

List semua metode pembayaran yang enabled.

**Response:**


```
[
  {
    "code": "credit_card",
    "name": "Kartu Kredit",
    "group": "card",
    "icon": "https://...",
    "min_amount": 10000,
    "max_amount": null,
    "metadata": {}
  }
]
```


### 4.2 `POST orders/{order}/pay`

Buat transaksi Midtrans Snap. Hanya bisa dipanggil sekali per order (kecuali status `failed`/`expired`).

**Request:**


```
{
  "method": "credit_card"
}
```


**Response:**


```
{
  "payment": {
    "id": 1,
    "order_id": 1,
    "method": "credit_card",
    "amount": 500000,
    "status": "pending",
    "snap_token": "abc123...",
    "redirect_url": "https://app.sandbox.midtrans.com/...",
    "paid_at": null,
    "gateway_reference": null,
    "created_at": "2026-08-20T..."
  },
  "snap_token": "abc123...",
  "redirect_url": "https://app.sandbox.midtrans.com/..."
}
```


**Error:**

- `403` — Order bukan milik user
- `422` — Order sudah diproses sebelumnya, atau metode tidak tersedia, atau amount di bawah minimum

### 4.3 `POST orders/{order}/pay/retry`

Retry pembayaran yang sudah ada. Tidak butuh `method` karena method sudah tersimpan di payment row.

**Response:** Sama seperti `POST orders/{order}/pay`.

### 4.4 `GET payments/{payment}`

Detail pembayaran. Include `order.booking`.

**Response:**

```
{
  "id": 1,
  "order_id": 1,
  "method": "credit_card",
  "amount": 500000,
  "status": "pending",
  "snap_token": "abc123...",
  "redirect_url": "https://app.sandbox.midtrans.com/...",
  "paid_at": null,
  "gateway_reference": null,
  "created_at": "2026-08-20T..."
}
```

### 4.5 `POST payments/{payment}/reconcile`

Sync status pembayaran dengan Midtrans. Panggil ini jika `GET payments/{payment}` masih `pending` setelah webview ditutup.

**Response:**

```
{
  "payment_id": 1,
  "status": "paid"
}
```

### 4.6 `POST payments/webhook` — Midtrans S2S Only

**INI ADALAH ENDPOINT UNTUK MIDTRANS SAJA. KLIENT FLUTTER TIDAK BOLEH MEMANGGIL ENDPOINT INI.**

Midtrans mengirim notifikasi S2S setelah transaksi selesai. Endpoint ini:

1. Verifikasi signature SHA-512
2. Cek `gross_amount` vs `payment->amount`
3. Update status payment/order/booking via `applyTransactionStatus()`

**Payload dari Midtrans:**

```
{
  "order_id": "HTL-20260820-A1B2C3D4-1",
  "status_code": "200",
  "gross_amount": "500000.00",
  "signature_key": "sha512_hash...",
  "transaction_status": "settlement",
  "transaction_id": "...",
  "fraud_status": "accept"
}
```

**Catatan keamanan:**

- Signature dihitung: `hash('sha512', order_id + status_code + gross_amount + serverKey)`
- Klien Flutter tidak punya `serverKey`, sehingga tidak dapat membuat signature yang valid
- Mengabaikan verifikasi signature memungkinkan status pembayaran dimanipulasi dari perangkat klien

## 5. Flutter Implementation

### 5.1 Service Methods

`lib/services/booking_service.dart`:

| Method | Endpoint | Return |
|---|---|---|
| `paymentMethods()` | `GET payments/methods` | `List<PaymentMethod>` |
| `pay(orderId, method)` | `POST orders/{orderId}/pay` | `Payment` |
| `retryPayment(orderId)` | `POST orders/{orderId}/pay/retry` | `Payment` |
| `paymentDetail(id)` | `GET payments/{id}` | `Payment` |
| `reconcilePayment(id)` | `POST payments/{id}/reconcile` | `Payment` |
| `paymentWebhook(payload)` | `POST payments/webhook` | `void` |

`lib/services/host_booking_service.dart` (host-side):

| Method | Endpoint | Return |
|---|---|---|
| `confirmBooking(bookingId)` | `POST bookings/{id}/confirm` | `Map<String, dynamic>` |
| `rejectBooking(bookingId)` | `POST bookings/{id}/reject` | `Map<String, dynamic>` |
| `checkinBooking(bookingId)` / `checkin` | `POST bookings/{id}/checkin` | `Map<String, dynamic>` |
| `checkout(bookingId)` | `POST bookings/{id}/checkout` | `Map<String, dynamic>` |
| `confirmCheckin(bookingId)` | `POST bookings/{id}/confirm-checkin` | `Map<String, dynamic>` |

### 5.2 Payment Initiation by Screen

| Screen | Kapan | Method yang Dipanggil |
|---|---|---|
| `hotel_booking_screen.dart` | Setelah booking hotel dibuat | `BookingService.pay(orderId, 'credit_card')` → buka webview |
| `rincian_unit_screen.dart` | Setelah booking unit dibuat | `BookingService.pay(orderId, 'credit_card')` → buka webview |
| `booking_detail_screen.dart` | User klik "Bayar" | `BookingService.retryPayment(orderId)` → buka webview |
| `booking_detail_hotel_screen.dart` | User klik "Bayar" | `BookingService.pay(orderId, firstMethod)` → buka webview |

### 5.3 Host Action Endpoints

Host melakukan aksi pada booking melalui `HostBookingService`:

| Screen | Aksi | Method yang Dipanggil |
|---|---|---|
| `booking_detail_screen.dart` (host) | Konfirmasi booking | `HostBookingService.confirmBooking(id)` |
| `booking_detail_screen.dart` (host) | Tolak booking | `HostBookingService.rejectBooking(id)` |
| `booking_detail_hotel_screen.dart` (host) | Check-in tamu | `HostBookingService.checkinBooking(id)` |
| `booking_detail_hotel_screen.dart` (host) | Konfirmasi check-in | `HostBookingService.confirmCheckin(id)` |

### 5.4 Webview Flow

1. Dapatkan `redirect_url` dari response `pay()` atau `retryPayment()`
2. Buka `PaymentWebviewScreen` dengan `PaymentWebviewArgs(url: redirectUrl, ...)`
3. User selesaikan pembayaran di webview
4. Midtrans redirect ke `livirooms://payment-result?paymentId=X&orderId=Y`
5. `DeepLinkHandler` menangkap deep link
6. `PaymentResultScreen` dibuka
7. Panggil `GET payments/{paymentId}` untuk konfirmasi status
8. Jika masih `pending`, panggil `POST payments/{paymentId}/reconcile`

## 6. Payment Status Machine

### 6.1 Payment Status

```
pending → paid (via webhook/reconcile)
pending → failed (via webhook/reconcile)
pending → expired (via CancelExpiredPayments job)
paid → refunded (admin action)
```

**Sumber kebenaran status:** Webhook S2S Midtrans (`POST payments/webhook`). Status dari webview callback atau parameter URL tidak dipercaya.

### 6.2 Booking Status

Booking memiliki status yang lebih detail untuk mengelola alur:

| Status | Keterangan |
|---|---|
| `waiting_payment` | Booking dibuat, menunggu pembayaran (hotel) |
| `waiting_confirmation` | Booking dibuat, menunggu konfirmasi host (property) |
| `confirmed` | Host mengkonfirmasi booking property |
| `paid` | Pembayaran lunas |
| `waiting_checkin` | Tamu tiba, menunggu konfirmasi check-in |
| `checked_in` | Tamu sudah check-in |
| `staying` | Check-in dikonfirmasi host |
| `completed` | Booking selesai |
| `cancelled` | Booking dibatalkan (user/host) |
| `rejected` | Booking ditolak host |
| `expired` | Booking kedaluwarsa (batas waktu pembayaran) |

## 7. Deep Link

Setelah pembayaran selesai, Midtrans redirect ke:

```
livirooms://payment-result?paymentId={id}&orderId={order_id}
```

**Catatan:** Backend sengaja tidak menyertakan status pembayaran di deep link URL. Flutter app wajib memanggil `GET payments/{paymentId}` untuk mendapatkan status sebenarnya.

## 8. Dummy Mode

Mode dummy mensimulasikan endpoint pembayaran:

| Route | Fixture File |
|---|---|
| `GET payments/methods` | `GET_payments_methods.json` |
| `POST orders/{id}/pay` | `POST_orders_id_pay.json` |
| `POST orders/{id}/pay/retry` | `POST_orders_id_pay_retry.json` |
| `GET payments/{id}` | `GET_payments_id.json` |
| `POST payments/{id}/reconcile` | `POST_payments_id_reconcile.json` |
| `POST payments/webhook` | `POST_payments_webhook.json` |

## 9. Kebijakan Guest Browsing

Browsing properti bersifat publik, tetapi booking dan pembayaran memerlukan login:

- **Publik:** `GET /api/v1/home`, `/api/v1/properties`, `/api/v1/properties/{id}`, `/api/v1/cities`, `/api/v1/vouchers`
- **Login Required:** `POST /api/v1/bookings`, `/POST /api/v1/orders/{id}/pay`, `/api/v1/wishlists`, `/api/v1/conversations`
- Di Flutter, `SplashScreen` mengarahkan guest ke `Routes.home`. Aksi transaksional menampilkan `Routes.login` jika tidak terautentikasi.

## 10. Ringkasan URL

| Field di Dashboard Midtrans | Isi |
|---|---|
| **Payment Notification URL** | `https://livirooms.com/api/v1/payments/webhook` |
| **Finish Redirect URL** | `https://livirooms.com/payment/finish` |
| **Unfinish Redirect URL** | `https://livirooms.com/payment/unfinish` |
| **Error Redirect URL** | `https://livirooms.com/payment/error` |

## 11. Notification URL vs Finish Redirect URL

### Notification URL — jalur yang dipercaya

```
Server Midtrans  ──HTTP POST──▶  Server Laravel Anda
```

- Dipanggil **server-ke-server**. Browser pengguna tidak terlibat.
- Membawa `signature_key` = `sha512(order_id + status_code + gross_amount + server_key)`.
- Karena `server_key` hanya diketahui Midtrans dan server Anda, tanda tangan ini **membuktikan** pesannya asli.
- **Inilah satu-satunya jalur yang boleh mengubah status pembayaran.**
- Midtrans mengulang kirim sampai dapat HTTP `200`. Handler-nya wajib idempoten — dan di kode ini sudah: `applyTransactionStatus()` keluar lebih awal bila `status === 'paid'` (`Api/PaymentController.php:319`).

### Finish Redirect URL — jalur yang TIDAK dipercaya

```
Browser pengguna  ──redirect──▶  Server Laravel Anda
```

- Yang mengunjungi adalah **browser pengguna** setelah ia menekan tombol selesai di halaman Snap.
- Semua query string-nya dikendalikan pengguna. Siapa pun bisa mengetik `?order_id=ORD-123-45&transaction_status=settlement` di address bar.
- **Tidak boleh menyentuh status pembayaran.** Lihat komentar di `app/Http/Controllers/Payment/FinishController.php:10-20`.
- Tugasnya hanya satu: memantulkan pengguna kembali ke aplikasi Flutter.

> Analogi: Notification URL itu petugas bank yang menelepon lewat nomor resmi. Finish Redirect URL itu orang yang mengetuk pintu Anda dan mengaku dari bank. Yang pertama boleh mengubah saldo, yang kedua hanya boleh dipersilakan masuk.

Kalau Anda hanya mengisi Finish Redirect URL dan mengabaikan Notification URL, **tidak ada satu pun pembayaran yang akan tercatat lunas.** Booking pengguna akan kedaluwarsa oleh `payments:cancel-expired` meskipun uangnya sudah masuk.

## 12. Notification URL — Detail

### Route-nya

`laravel/routes/api_mobile.php:52-53`

```
Route::post('payments/webhook', [PaymentController::class, 'webhook']);
```

Route ini berada di dalam `Route::prefix('v1')`, dan file `routes/api.php` memuatnya di bawah prefix `/api` bawaan Laravel. Jadi path lengkapnya:

```
/api  +  /v1  +  /payments/webhook   →   /api/v1/payments/webhook
```

Perhatikan letaknya: **di luar** grup `auth:sanctum`. Memang harus begitu — Midtrans tidak punya token Sanctum. Pengaman jalur ini adalah tanda tangan SHA512, bukan sesi login.

### Yang diisi di dashboard

| Lingkungan | Isi |
|---|---|
| Sandbox, domain publik | `https://staging.livirooms.com/api/v1/payments/webhook` |
| Produksi | `https://livirooms.com/api/v1/payments/webhook` |
| Lokal (via tunnel) | `https://a1b2c3d4.ngrok-free.app/api/v1/payments/webhook` |

### Syarat yang wajib dipenuhi

1. **HTTPS.** Midtrans menolak `http://` di produksi.
2. **Dapat diakses publik.** `localhost` dan IP privat tidak akan pernah dihubungi Midtrans.
3. **Tanpa auth.** Jangan letakkan di balik Basic Auth, Cloudflare Access, atau halaman "maintenance".
4. **Balas 200 secepatnya.** Kalau server balas `500` atau timeout, Midtrans mengulang beberapa kali lalu menyerah — dan pembayaran akan menggantung.
5. **Bebas dari CSRF.** Sudah aman: route API tidak memakai middleware `web`.

## 13. Finish Redirect URL — Detail

### Route-nya

`laravel/routes/web.php:35-38`

```
Route::get('/payment/finish',   [FinishController::class, 'finish'])->name('payment.finish');
Route::get('/payment/unfinish', [UnfinishController::class, 'unfinish'])->name('payment.unfinish');
Route::get('/payment/error',    [ErrorController::class, 'error'])->name('payment.error');
```

Ketiganya di `routes/web.php` **tanpa** prefix `admin` dan **tanpa** middleware `auth` — memang harus begitu, karena yang membukanya browser eksternal yang mungkin belum login.

Jadi path-nya persis: `/payment/finish` — tanpa `/api`, tanpa `/v1`.

### Yang diisi di dashboard

| Lingkungan | Finish | Unfinish | Error |
|---|---|---|---|
| Produksi | `https://livirooms.com/payment/finish` | `.../payment/unfinish` | `.../payment/error` |
| Sandbox | `https://staging.livirooms.com/payment/finish` | `.../payment/unfinish` | `.../payment/error` |
| Lokal | `https://a1b2c3d4.ngrok-free.app/payment/finish` | `.../payment/unfinish` | `.../payment/error` |

### Kapan masing-masing dipanggil

| URL | Dipanggil saat |
|---|---|
| **Finish** | Pengguna menyelesaikan alur pembayaran. **Belum tentu lunas** — bisa saja ia baru dapat nomor VA dan belum mentransfer.<br>Pengguna menutup/meninggalkan halaman Snap sebelum selesai. |

**Unfinish**

**Error** Terjadi kegagalan di sisi Midtrans.

Poin di baris "Finish" sering disalahpahami. `finish` **bukan** sinonim "berhasil". Untuk pembayaran VA, pengguna sampai di `finish` begitu nomor VA terbit — pembayaran sesungguhnya baru terjadi berjam-jam kemudian, dan hanya webhook yang tahu.

### Satu catatan penting soal dashboard

Nilai di dashboard sebenarnya adalah **fallback**. Kode ini sudah mengirim `callbacks` di setiap permintaan Snap (`app/Support/MidtransSnap.php:129-133`):

```
'callbacks' => [
    'finish'   => url('/payment/finish'),
    'unfinish' => url('/payment/unfinish'),
    'error'    => url('/payment/error'),
],
```

`url()` menurunkan host dari `APP_URL`. Artinya:

> **`APP_URL` di `.env` wajib benar.** Kalau masih `http://localhost`, Midtrans akan memantulkan pengguna produksi ke `localhost` — dan alur pembayaran putus di situ, tanpa pesan galat yang jelas.

```
APP_URL=https://livirooms.com
```

Tetap isi juga kolom di dashboard sebagai jaring pengaman.

## 14. Langkah Setup

### Langkah 1 — Ambil kunci dari dashboard sandbox

1. Masuk ke `https://dashboard.sandbox.midtrans.com`
2. `Settings` → `Access Keys`
3. Salin **Merchant ID**, **Client Key**, dan **Server Key**

Server Key sandbox berawalan `SB-Mid-server-`, Client Key `SB-Mid-client-`. Kunci produksi tanpa awalan `SB-`.

> **Server Key adalah kredensial paling sensitif di sistem ini.** Ia memverifikasi webhook dan mengotorisasi refund. Jangan pernah menaruhnya di kode Flutter, repo publik, atau tiket. Client Key boleh berada di sisi klien; Server Key tidak.

### Langkah 2 — Masukkan kunci ke aplikasi

Ada dua cara, dan **cara pertama yang dianjurkan**:

**Cara A — lewat panel admin (dianjurkan)**

```
Login sebagai admin → menu Pembayaran → Pengaturan Pembayaran
```

Isi Server Key, Client Key, dan sakelar Production Mode, lalu simpan. Nilainya disimpan ke tabel `payment_settings` dan langsung dipakai — `AppServiceProvider` mendorongnya ke `config('services.midtrans.*')` saat boot.

Kelebihannya: rotasi kunci tidak butuh akses SSH dan tidak perlu deploy ulang.

> Halaman ini hanya untuk peran **admin**. Peran `host` (pemilik properti) sengaja tidak diberi akses, karena halaman ini memuat Server Key.

**Cara B — lewat `.env` (hanya untuk instalasi baru)**

```
MIDTRANS_SERVER_KEY=SB-Mid-server-xxxxxxxxxxxxxxxxxxxx
MIDTRANS_CLIENT_KEY=SB-Mid-client-xxxxxxxxxxxxxxxxxxxx
MIDTRANS_IS_PRODUCTION=false
```

`.env` hanya berfungsi sebagai nilai awal. Begitu ada baris di `payment_settings`, baris itulah yang menang (`app/Models/PaymentSetting.php:67-78`).

### Langkah 3 — Pastikan `APP_URL` benar

```
APP_URL=https://livirooms.com
```

Lalu bersihkan cache konfigurasi:

```
php artisan config:clear
```

### Langkah 4 — Isi URL di dashboard

`Settings` → `Configuration`:

| Kolom | Isi |
|---|---|
| Payment Notification URL | `https://livirooms.com/api/v1/payments/webhook` |
| Finish Redirect URL | `https://livirooms.com/payment/finish` |
| Unfinish Redirect URL | `https://livirooms.com/payment/unfinish` |
| Error Redirect URL | `https://livirooms.com/payment/error` |

Tekan **Update**.

### Langkah 5 — Aktifkan metode pembayaran

```
Panel admin → Pembayaran → Metode Pembayaran
```

Kolom `code` berupa dropdown, isinya terbatas pada kode yang bisa dipetakan `MidtransSnap` ke channel Snap:

`virtual_account`, `bank_transfer`, `qris`, `ewallet`, `credit_card`, `paylater`

Pembatasan ini disengaja. Kode di luar daftar akan sampai ke Snap tanpa pemetaan channel, sehingga pengguna melihat **semua** pilihan pembayaran padahal `payments.method` mencatat satu metode tertentu — laporan jadi tidak cocok dengan data Midtrans.

Untuk mempersempit channel lebih jauh, isi kolom Metadata:

```
{"banks": ["bca", "bni"]}
```

atau

```
{"channels": ["gopay", "shopeepay"]}
```

Pemetaannya ada di `app/Support/MidtransSnap.php:35-69`.

### Langkah 6 — Pastikan scheduler berjalan

```
* * * * * cd /var/www/livirooms && php artisan schedule:run >> /dev/null 2>&1
```

Tanpa ini, `payments:cancel-expired` tidak pernah jalan dan pembayaran yang telantar akan menahan ketersediaan properti selamanya. Jadwalnya terdaftar di `routes/console.php:14-19` (tiap 10 menit).

## 15. Menguji Secara Lokal (tanpa domain publik)

Midtrans tidak bisa menghubungi `localhost`. Buka terowongan:

```
ngrok http 8000
```

Ambil URL HTTPS yang muncul, misalnya `https://a1b2c3d4.ngrok-free.app`, lalu:

1. Perbarui `.env`:

   ```
   APP_URL=https://a1b2c3d4.ngrok-free.app
   ```

2. `php artisan config:clear`
3. Perbarui keempat URL di dashboard dengan host tersebut.

URL ngrok gratis berubah setiap kali dijalankan ulang, jadi langkah 1–3 harus diulang tiap sesi.

### Menguji webhook tanpa membayar

`Settings` → `Configuration` → tombol **Send test notification**.

Atau kirim manual — perhatikan tanda tangannya harus dihitung dengan Server Key yang sama persis:

```
ORDER_ID="ORD-A1B2C3D4-45"
STATUS_CODE="200"
GROSS_AMOUNT="575000.00"
SERVER_KEY="SB-Mid-server-xxxxxxxxxxxxxxxxxxxx"

SIGNATURE=$(printf '%s%s%s%s' "$ORDER_ID" "$STATUS_CODE" "$GROSS_AMOUNT" "$SERVER_KEY" | sha512sum | cut -d' ' -f1)

curl -X POST https://a1b2c3d4.ngrok-free.app/api/v1/payments/webhook \
  -H "Content-Type: application/json" \
  -d "{
    \"order_id\": \"$ORDER_ID\",
    \"status_code\": \"$STATUS_CODE\",
    \"gross_amount\": \"$GROSS_AMOUNT\",
    \"signature_key\": \"$SIGNATURE\",
    \"transaction_status\": \"settlement\",
    \"fraud_status\": \"accept\",
    \"transaction_id\": \"test-trx-001\"
  }"
```

Dua hal yang mudah keliru:

- `gross_amount` harus **sama persis** dengan yang dikirim Midtrans, termasuk dua angka desimal (`575000.00`, bukan `575000`). Tanda tangan dihitung dari string mentahnya.
- `order_id` harus berakhiran `-{payment_id}` yang benar-benar ada di basis data, karena `PaymentDeepLink::paymentIdFrom()` memotong di tanda hubung terakhir.

### Kartu uji sandbox

| Skenario | Nomor kartu | CVV | Exp | OTP |
|---|---|---|---|---|
| Berhasil | `4811 1111 1111 1114` | `123` | `01/25` | `112233` |
| Ditolak | `4911 1111 1111 1113` | `123` | `01/25` | `112233` |

Daftar lengkap: https://docs.midtrans.com/docs/testing-payment-on-sandbox (https://docs.midtrans.com/docs/testing-payment-on-sandbox)

## 16. Naik ke Produksi

1. **Ganti dashboard** ke `https://dashboard.midtrans.com` (tanpa `sandbox`).
2. **Ambil kunci produksi** — berbeda dari sandbox, tanpa awalan `SB-`.
3. **Perbarui via panel admin**, termasuk menyalakan sakelar **Production Mode**. Sakelar ini yang menentukan `Config::$isProduction` (`app/Support/MidtransSnap.php:77-83`); salah setel berarti transaksi dikirim ke lingkungan yang keliru dan seluruhnya gagal.
4. **Isi ulang keempat URL** di dashboard produksi dengan domain produksi.
5. **Perbarui `APP_URL`** lalu `php artisan config:clear`.
6. **Pastikan sertifikat TLS sah.** Sertifikat kedaluwarsa atau self-signed membuat Midtrans gagal mengirim webhook, dan pembayaran akan menggantung tanpa

gejala di sisi pengguna.
7. **Uji satu transaksi kecil sungguhan** sebelum diumumkan.

## 17. Bila Bermasalah

| Gejala | Kemungkinan penyebab | Cara memastikan |
|---|---|---|
| Pembayaran lunas tapi status tetap `pending` | Notification URL salah / tidak terjangkau | `Settings` → `Configuration` → tab riwayat notifikasi; cek `storage/logs/laravel.log` |
| Log berisi `signature mismatch` | Server Key di aplikasi ≠ Server Key dashboard, atau sandbox tercampur produksi | Bandingkan Server Key di panel admin dengan `Access Keys` |
| Log berisi `amount mismatch` | `orders.total` berubah setelah transaksi dibuat | Bandingkan `payments.amount` dengan `gross_amount` di payload |
| Setelah bayar, browser mendarat di `localhost` | `APP_URL` masih nilai lokal | `php artisan tinker` → `config('app.url')` |
| Aplikasi Flutter tidak terbuka setelah bayar | Deep link belum terpasang | Lihat T-12 di `remediation.md` |
| Booking kedaluwarsa padahal sudah dibayar | Scheduler mati **atau** webhook tak pernah sampai | Cek cron; cari `left paid` di log |
| Snap menampilkan semua metode padahal pengguna memilih QRIS | Metadata/kode metode tidak terpetakan | Cek baris di `payment_methods`, cocokkan dengan `MidtransSnap::DEFAULT_CHANNELS` |
| Tombol "Lanjutkan Bayar" tidak muncul di booking detail | `host_confirmed = false` — host belum mengkonfirmasi | Login sebagai host → Manajemen Order → Terima |
| Tombol pembayaran hilang meski status pending | Booking sudah expired (`check_out` terlewat) — tombol disembunyikan otomatis | Cek tanggal `check_out` booking; jika lewat, buat booking baru |
| Booking baru ditolak dengan "Tanggal sudah dipesan" | Ada booking aktif lain yang overlap tanggal yang sama | Cek `bookings` table untuk property_id tersebut |
| Payment expired tapi booking masih waiting_confirmation | `payments:cancel-expired` belum dijalankan | Jalankan `php artisan payments:cancel-expired` |

### Tempat melihat log

```
tail -f storage/logs/laravel.log | grep -i midtrans
```

Yang penting dicari:

- `Midtrans webhook rejected: signature mismatch` → kunci tidak cocok
- `Midtrans webhook rejected: amount mismatch` → nominal berubah
- `Midtrans transaction flagged for manual review` → `fraud_status = challenge`, perlu approve/deny manual di dashboard
- `Midtrans reported failure for an already-paid payment` → notifikasi datang tidak berurutan; sudah ditangani, tapi layak diperiksa

## 18. Rujukan

- Dashboard sandbox — https://dashboard.sandbox.midtrans.com (https://dashboard.sandbox.midtrans.com)
- Dokumentasi Snap — https://docs.midtrans.com/docs/snap-snap-integration-guide (https://docs.midtrans.com/docs/snap-snap-integration-guide)
- HTTP Notification — https://docs.midtrans.com/docs/https-notification-webhooks (https://docs.midtrans.com/docs/https-notification-webhooks)
- SDK PHP — https://github.com/Midtrans/midtrans-php (https://github.com/Midtrans/midtrans-php)

<!-- Source: configuration/admin-panel.md -->

# Admin Panel Architecture

Dokumen ini menjelaskan bagaimana data mengalir dari **Controller → DataTable → Blade View** dan bagaimana **Livewire** menangani operasi add/edit (termasuk file uploads) di admin panel. Dokumen ini berfungsi sebagai **role model untuk membangun semua 24 modul** yang tercantum di project README.

**Canonical reference implementation:** `resources/views/admin/user-management/`. Folder ini berisi working patterns yang digunakan di seluruh project:

- **Pattern A (Users)**: DataTable + standalone Livewire modal (`AddUserModal`)
- **Pattern B (Roles)**: Pure Livewire list (`RoleList`) + standalone modal (`RoleModal`), no DataTable
- **Pattern C (Permissions)**: DataTable + standalone Livewire modal (`PermissionModal`)
- **Pattern D (Accommodation Wizard)**: DataTable + Multi-Step Livewire Form Wizard modal (`Accommodations\WizardForm`) yang mengelola relasi kompleks 21 tabel ternormalisasi (3NF) dalam `DB::transaction`.

Semua modul harus mengikuti salah satu dari 4 patterns ini.

## Daftar Isi


- Struktur File
- Tiga Pattern Modul
- Controller Pattern
- DataTable Pattern (Yajra DataTables)
- Blade View Pattern (DataTable Page)
- Livewire Component Pattern
- Livewire Blade Pattern (Modal)
- Modal Communication Flow
- Checklist Implementasi Modul Baru


## Struktur File

```
laravel/
├── app/
│   ├── Http/Controllers/Admin/
│   │   ├── RoleManagementController.php
│   │   ├── UserManagementController.php
│   │   ├── PermissionManagementController.php
│   │   ├── PropertyController.php
```

```
│   │   ├── BookingController.php
│   │   ├── AvailabilityController.php
│   │   ├── PropertyPriceController.php
│   │   ├── VoucherController.php
│   │   ├── PropertyRecommendationController.php
│   │   ├── ConversationController.php
│   │   ├── FaqController.php
│   │   ├── NotificationLogController.php
│   │   ├── UserProfileController.php
│   │   ├── OrderController.php
│   │   ├── PaymentController.php
│   │   ├── PaymentMethodController.php
│   │   ├── WithdrawalController.php
│   │   ├── ReviewController.php
│   │   ├── WishlistController.php
│   │   ├── ReferralController.php
│   │   ├── BannerController.php
│   │   ├── CmsPageController.php
│   │   ├── AuditLogController.php
│   │   ├── BuildingListController.php
│   │   ├── PropertySearchLogController.php
│   │   └── ReportController.php
│   ├── DataTables/
│   │   ├── PermissionsDataTable.php
│   │   ├── UsersAssignedRoleDataTable.php
│   │   ├── UsersDataTable.php
│   │   ├── PropertyDataTable.php
│   │   ├── BookingDataTable.php
│   │   ├── AvailabilityDataTable.php
│   │   ├── PropertyPriceDataTable.php
│   │   ├── VoucherDataTable.php
│   │   ├── PropertyRecommendationDataTable.php
│   │   ├── ConversationDataTable.php
│   │   ├── FaqDataTable.php
│   │   ├── NotificationLogDataTable.php
│   │   ├── UserProfileDataTable.php
│   │   ├── OrderDataTable.php
│   │   ├── PaymentDataTable.php
│   │   ├── PaymentMethodDataTable.php
│   │   ├── WithdrawalDataTable.php
│   │   ├── ReviewDataTable.php
│   │   ├── WishlistDataTable.php
│   │   ├── ReferralDataTable.php
│   │   ├── BannerDataTable.php
│   │   ├── CmsPageDataTable.php
│   │   ├── AuditLogDataTable.php
│   │   ├── BuildingListDataTable.php
│   │   └── PropertySearchLogDataTable.php
│   └── Livewire/
│       ├── Permission/
│       │   ├── RoleList.php
│       │   ├── RoleModal.php
│       │   └── PermissionModal.php
│       ├── User/
│       │   └── AddUserModal.php
│       ├── Properties/
│       │   ├── Form.php
│       │   └── PropertyModal.php
│       ├── Bookings/
│       │   ├── Form.php
│       │   └── BookingModal.php
│       ├── Availabilities/
│       │   ├── Form.php
│       │   └── AvailabilityModal.php
│       ├── PropertyPrices/
│       │   ├── Form.php
│       │   └── PropertyPriceModal.php
│       ├── Vouchers/
│       │   ├── Form.php
│       │   └── VoucherModal.php
│       ├── PropertyRecommendations/
│       │   ├── Form.php
│       │   └── PropertyRecommendationModal.php
│       ├── Conversations/
│       │   ├── Form.php
│       │   └── ConversationModal.php
│       ├── Faqs/
│       │   ├── Form.php
│       │   └── FaqModal.php
│       ├── NotificationLogs/
│       │   ├── Form.php
│       │   └── NotificationLogModal.php
│       ├── UserProfiles/
│       │   ├── Form.php
│       │   └── UserProfileModal.php
│       ├── Orders/
```

```
│       │   ├── Form.php
│       │   └── OrderModal.php
│       ├── Payments/
│       │   ├── Form.php
│       │   └── PaymentModal.php
│       ├── PaymentMethods/
│       │   ├── Form.php
│       │   └── PaymentMethodModal.php
│       ├── Withdrawals/
│       │   ├── Form.php
│       │   └── WithdrawalModal.php
│       ├── Reviews/
│       │   ├── Form.php
│       │   └── ReviewModal.php
│       ├── Wishlists/
│       │   ├── Form.php
│       │   └── WishlistModal.php
│       ├── Referrals/
│       │   ├── Form.php
│       │   └── ReferralModal.php
│       ├── Banners/
│       │   ├── Form.php
│       │   └── BannerModal.php
│       ├── CmsPages/
│       │   ├── Form.php
│       │   └── CmsPageModal.php
│       ├── AuditLogs/
│       │   ├── Form.php
│       │   └── AuditLogModal.php
│       ├── BuildingLists/
│       │   ├── Form.php
│       │   └── BuildingListModal.php
│       └── PropertySearchLogs/
│           ├── Form.php
│           └── PropertySearchLogModal.php
└── resources/views/
    ├── admin/
    │   ├── user-management/
    │   │   ├── users/
    │   │   │   ├── list.blade.php
    │   │   │   ├── show.blade.php
    │   │   │   └── columns/
    │   │   │       ├── _user.blade.php
    │   │   │       └── _actions.blade.php
    │   │   ├── roles/
    │   │   │   ├── list.blade.php
    │   │   │   ├── show.blade.php
    │   │   │   └── columns/
    │   │   │       ├── _user.blade.php
    │   │   │       └── _actions.blade.php
    │   │   └── permissions/
    │   │       ├── list.blade.php
    │   │       └── columns/
    │   │           ├── _assign-to.blade.php
    │   │           └── _actions.blade.php
    │   ├── properties/
    │   │   ├── index.blade.php
    │   │   └── columns/
    │   │       └── _actions.blade.php
    │   ├── bookings/
    │   │   ├── index.blade.php
    │   │   └── columns/
    │   │       └── _actions.blade.php
    │   ├── availabilities/
    │   │   ├── index.blade.php
    │   │   └── columns/
    │   │       └── _actions.blade.php
    │   ├── property_prices/
    │   │   ├── index.blade.php
    │   │   └── columns/
    │   │       └── _actions.blade.php
    │   ├── vouchers/
    │   │   ├── index.blade.php
    │   │   └── columns/
    │   │       └── _actions.blade.php
    │   ├── property_recommendations/
    │   │   ├── index.blade.php
    │   │   └── columns/
    │   │       └── _actions.blade.php
    │   ├── conversations/
    │   │   ├── index.blade.php
    │   │   └── columns/
    │   │       └── _actions.blade.php
    │   ├── faqs/
    │   │   ├── index.blade.php
    │   │   └── columns/
```

```
│   │   │       └── _actions.blade.php
│   ├── notification_logs/
│   │   ├── index.blade.php
│   │   └── columns/
│   │       └── _actions.blade.php
│   ├── user_profiles/
│   │   ├── index.blade.php
│   │   └── columns/
│   │       └── _actions.blade.php
│   ├── orders/
│   │   ├── index.blade.php
│   │   └── columns/
│   │       └── _actions.blade.php
│   ├── payments/
│   │   ├── index.blade.php
│   │   └── columns/
│   │       └── _actions.blade.php
│   ├── payment_methods/
│   │   ├── index.blade.php
│   │   └── columns/
│   │       └── _actions.blade.php
│   ├── withdrawals/
│   │   ├── index.blade.php
│   │   └── columns/
│   │       └── _actions.blade.php
│   ├── reviews/
│   │   ├── index.blade.php
│   │   └── columns/
│   │       └── _actions.blade.php
│   ├── wishlists/
│   │   ├── index.blade.php
│   │   └── columns/
│   │       └── _actions.blade.php
│   ├── referrals/
│   │   ├── index.blade.php
│   │   └── columns/
│   │       └── _actions.blade.php
│   ├── banners/
│   │   ├── index.blade.php
│   │   └── columns/
│   │       └── _actions.blade.php
│   ├── cms_pages/
│   │   ├── index.blade.php
│   │   └── columns/
│   │       └── _actions.blade.php
│   ├── audit_logs/
│   │   ├── index.blade.php
│   │   └ columns/
│   │       └── _actions.blade.php
│   ├── building_lists/
│   │   ├── index.blade.php
│   │   ├── create.blade.php
│   │   ├── edit.blade.php
│   │   └── columns/
│   │       └── _actions.blade.php
│   └── property_search_logs/
│       ├── index.blade.php
│       └── columns/
│           └── _actions.blade.php
└── livewire/
    ├── user/
    │   └── add-user-modal.blade.php
    ├── permission/
    │   ├── role-list.blade.php
    │   ├── role-modal.blade.php
    │   └── permission-modal.blade.php
    ├── properties/
    │   └── form.blade.php
    ├── bookings/
    │   └── form.blade.php
    ├── availabilities/
    │   └── form.blade.php
    ├── property_prices/
    │   └── form.blade.php
    ├── vouchers/
    │   └── form.blade.php
    ├── property_recommendations/
    │   └── form.blade.php
    ├── conversations/
    │   └── form.blade.php
    ├── faqs/
    │   └── form.blade.php
    ├── notification_logs/
    │   └── form.blade.php
    ├── user_profiles/
    │   └── form.blade.php
```

```
        ├── orders/
        │   └── form.blade.php
        ├── payments/
        │   └── form.blade.php
        ├── payment_methods/
        │   └── form.blade.php
        ├── withdrawals/
        │   └── form.blade.php
        ├── reviews/
        │   └── form.blade.php
        ├── wishlists/
        │   └── form.blade.php
        ├── referrals/
        │   └── form.blade.php
        ├── banners/
        │   └── form.blade.php
        ├── cms_pages/
        │   └── form.blade.php
        ├── audit_logs/
        │   └── form.blade.php
        ├── building-lists/
        │   └── form.blade.php
        └── property_search_logs/
            └── form.blade.php
```

### Tiga Pattern Modul

Project menggunakan **3 distinct module patterns**. Pilih yang sesuai dengan kebutuhan UI modul:

| Pattern | When to Use | List View | Add/Edit |
|---|---|---|---|
| **A: DataTable + Livewire Modal** | Large datasets needing search/sort/export | Yajra DataTable | Standalone Livewire modal |
| **B: Pure Livewire** | Simple card/grid layouts, small datasets | Livewire list component | Standalone Livewire modal |
| **C: Base Form + Modal Wrapper** | Reusable form logic across create/edit | Same as Pattern A | Base `Form.php` + thin `Modal.php` wrapper |

**Canonical examples:**

- Pattern A: `user-management/users` (UsersDataTable + AddUserModal)
- Pattern B: `user-management/roles` (RoleList + RoleModal)
- Pattern C: Most CRUD modules (e.g., Properties, Bookings)

---

## Controller Pattern

Controllers di admin panel adalah **thin**. Mereka terutama merender views dan delegasi data rendering ke DataTables atau Livewire components.

### Pattern A: DataTable Controller (Users, Permissions, dan kebanyakan modul CRUD)

```php
<?php

namespace App\Http\Controllers\Admin;

use App\DataTables\UsersDataTable;
use App\Http\Controllers\Controller;
use Illuminate\Http\Request;

class UserManagementController extends Controller
{
    public function index(UsersDataTable $dataTable)
    {
        return $dataTable->render('admin.user-management.users.list');
    }

    public function show(User $user)
    {
        return view('admin.user-management.users.show', compact('user'));
    }

    // CRUD stubs — actual logic handled by Livewire modal
    public function create() {}
    public function store(Request $request) {}
    public function edit(User $user) {}
    public function update(Request $request, User $user) {}
    public function destroy(User $user) {}
}
```

### Pattern B: Pure Livewire Controller (Roles)

```php
<?php

namespace App\Http\Controllers\Admin;

use App\Http\Controllers\Controller;
use Illuminate\Http\Request;
use Spatie\Permission\Models\Role;

class RoleManagementController extends Controller
{
    public function index()
    {
        return view('admin.user-management.roles.list');
    }

    public function show(Role $role, UsersAssignedRoleDataTable $dataTable)
    {
        return $dataTable->with('role', $role)
            ->render('admin.user-management.roles.show', compact('role'));
    }

    // CRUD stubs — actual logic handled by Livewire modal
    public function create() {}
    public function store(Request $request) {}
    public function edit(Role $role) {}
    public function update(Request $request, Role $role) {}
    public function destroy(Role $role) {}
}
```

**Pattern C: Permission-Style Controller**

```php
<?php

namespace App\Http\Controllers\Admin;

use App\DataTables\PermissionsDataTable;
use App\Http\Controllers\Controller;
use Illuminate\Http\Request;

class PermissionManagementController extends Controller
{
    public function index(PermissionsDataTable $dataTable)
    {
        return $dataTable->render('admin.user-management.permissions.list');
    }

    public function show(string $id) {}

    // CRUD stubs — actual logic handled by Livewire modal
    public function create() {}
    public function store(Request $request) {}
    public function edit(string $id) {}
    public function update(Request $request, string $id) {}
    public function destroy(string $id) {}
}
```

**Key rules:**

- `index()` returns a Blade view that embeds a DataTable and/or Livewire component
- `show()` returns a detail view, optionally with a sub-DataTable
- Actual add/edit/delete logic is handled by **Livewire**, not the controller
- `create()`, `store()`, `edit()`, `update()`, `destroy()` can remain empty stubs
- Route model binding is used where applicable (e.g., `User $user`, `Role $role`)

---

## DataTable Pattern (Yajra DataTables)

DataTables handle **server-side rendering** of tabular data with search, sort, and pagination.

**Contoh: `PermissionsDataTable.php`**

```php
<?php

namespace App\DataTables;

use Illuminate\Database\Eloquent\Builder as QueryBuilder;
use Spatie\Permission\Models\Permission;
use Yajra\DataTables\EloquentDataTable;
use Yajra\DataTables\Html\Builder as HtmlBuilder;
use Yajra\DataTables\Html\Column;
use Yajra\DataTables\Services\DataTable;

class PermissionsDataTable extends DataTable
{
    public function dataTable(QueryBuilder $query): EloquentDataTable
    {
        return (new EloquentDataTable($query))
            ->editColumn('name', function (Permission $permission) {
                return ucwords($permission->name);
            })
            ->addColumn('assigned_to', function (Permission $permission) {
                $roles = $permission->roles;
                return view('admin.user-management.permissions.columns._assign-to', compact('roles'));
            })
            ->editColumn('created_at', function (Permission $permission) {
                return $permission->created_at->format('d M Y, h:i a');
            })
            ->addColumn('actions', function (Permission $permission) {
                return view('admin.user-management.permissions.columns._actions', compact('permission'));
            })
            ->setRowId('id');
    }

    public function query(Permission $model): QueryBuilder
    {
        return $model->newQuery()->with('roles');
    }

    public function html(): HtmlBuilder
    {
        return $this->builder()
            ->setTableId('permissions-table')
            ->columns($this->getColumns())
            ->minifiedAjax()
            ->dom('rt' . "<'row'<'col-sm-12 col-md-5'l><'col-sm-12 col-md-7'p>>",)
            ->addTableClass('table align-middle table-row-dashed fs-6 gy-5 dataTable no-footer text-gray-600 fw-semibold')
            ->setTableHeadClass('text-start text-muted fw-bold fs-7 text-uppercase gs-0')
            ->orderBy(0)
            ->drawCallback("function() {" . file_get_contents(resource_path('views/admin/user-management/permissions/columns/_draw-
scripts.js')) . "}")
            ->parameters(['destroy' => true]);
    }

    public function getColumns(): array
    {
        return [
            Column::make('name'),
            Column::make('assigned_to'),
            Column::make('created_at')->addClass('text-nowrap'),
            Column::computed('actions')
                ->addClass('text-end text-nowrap')
                ->exportable(false)
                ->printable(false),
        ];
    }

    protected function filename(): string
    {
        return 'Permissions_' . date('YmdHis');
    }
}
```

---

## Blade View Pattern (DataTable Page)

**Contoh:** `admin/user-management/permissions/list.blade.php`

```blade
<x-default-layout>

    @section('title')
        Permissions
    @endsection

    @section('breadcrumbs')
        {{ Breadcrumbs::render('user-management.permissions.index') }}
    @endsection

    <div class="card">
        <div class="card-header border-0 pt-6">
            <div class="card-title">
                <div class="d-flex align-items-center position-relative my-1">
                    {!! getIcon('magnifier','fs-3 position-absolute ms-5') !!}
                    <input type="text" data-kt-user-table-filter="search"
                           class="form-control form-control-solid w-250px ps-13"
                           placeholder="Search permission" id="mySearchInput"/>
                </div>
            </div>
            <div class="card-toolbar">
                <button type="button" class="btn btn-light-primary"
                        data-bs-toggle="modal"
                        data-bs-target="#kt_modal_update_permission">
                    Add Permission
                </button>
                <livewire:permission.permission-modal></livewire:permission.permission-modal>
            </div>
        </div>

        <div class="card-body py-4">
            <div class="table-responsive">
                {!! $dataTable->table() !!}
            </div>
        </div>
    </div>

    @push('scripts')
        {!! $dataTable->scripts() !!}
        <script>
            document.getElementById('mySearchInput').addEventListener('keyup', function () {
                window.LaravelDataTables['permissions-table'].search(this.value).draw();
            });

            document.addEventListener('livewire:load', function () {
                Livewire.on('success', function () {
                    $('#kt_modal_update_permission').modal('hide');
                    window.LaravelDataTables['permissions-table'].ajax.reload();
                });
            });
        </script>
    @endpush

</x-default-layout>
```

**Key rules:**

- `{!! $dataTable->table() !!}` renders the table HTML + ajax source
- `{!! $dataTable->scripts() !!}` loads the DataTables JS initialization
- The Livewire modal is embedded at the bottom: `<livewire:permission.permission-modal></livewire:permission.permission-modal>`
- JavaScript listens for `livewire:load` and `Livewire.on('success', ...)` to hide modal and reload table

---

## Livewire Component Pattern

Livewire handles **add, edit, and delete** logic entirely on the frontend with backend validation.

Project menggunakan **3 Livewire component patterns**. Semua didemonstrasikan di `resources/views/admin/user-management/`.

### Pattern A: DataTable + Standalone Modal (Users)

**List View (Blade)**

```blade
{{-- resources/views/admin/user-management/users/list.blade.php --}}
<div class="card">
    <div class="card-header border-0 pt-6">
        <div class="card-title">
            <input type="text" placeholder="Search user" id="mySearchInput"/>
        </div>
        <div class="card-toolbar">
            <button type="button" class="btn btn-primary"
                    data-bs-toggle="modal" data-bs-target="#kt_modal_add_user">
                Add User
            </button>
            <livewire:user.add-user-modal></livewire:user.add-user-modal>
        </div>
    </div>
    <div class="card-body py-4">
        <div class="table-responsive">
            {!! $dataTable->table() !!}
        </div>
    </div>
</div>

@push('scripts')
    {!! $dataTable->scripts() !!}
    <script>
        document.addEventListener('livewire:load', function () {
            Livewire.on('success', function () {
                $('#kt_modal_add_user').modal('hide');
                window.LaravelDataTables["users-table"].ajax.reload();
            });
        });
    </script>
@endpush
```

**Standalone Modal Component (`AddUserModal.php`)**

```php
<?php

namespace App\Livewire\User;

use App\Models\User;
use App\Models\UserProfile;
use Livewire\Attributes\On;
use Livewire\Component;
use Livewire\WithFileUploads;
use Illuminate\Support\Facades\DB;
use Spatie\Permission\Models\Role;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Hash;

class AddUserModal extends Component
{
    use WithFileUploads;

    public $user_id;
    public $name;
    public $password;
    public $email;
    public $role;
    public $avatar;
    public $saved_avatar;

    public $type = 'consumer';
    public $phone;
    public $company_name;
    public $npwp;
    public $address;

    public $edit_mode = false;

    protected function rules()
    {
        return [
            'name' => 'required|string|max:255',
            'email' => $this->edit_mode
                ? 'required|email|max:255|unique:users,email,'.$this->user_id
                : 'required|email|max:255|unique:users,email',
            'password' => $this->edit_mode ? 'nullable|string|min:8' : 'required|string|min:8',
            'role' => 'nullable|string',
            'type' => 'nullable|string|in:consumer,host,corporate',
            'phone' => 'nullable|string|max:50',
            'company_name' => 'nullable|string|max:255',
            'npwp' => 'nullable|string|max:100',
            'address' => 'nullable|string',
        ];
    }
```

```php
    public function render()
    {
        $roles = Role::all();
        return view('livewire.user.add-user-modal', compact('roles'));
    }

    public function submit()
    {
        $this->validate();

        DB::transaction(function () {
            $userData = [
                'name' => $this->name,
                'email' => $this->email,
                'email_verified_at' => now(),
            ];

            if ($this->password) {
                $userData['password'] = Hash::make($this->password);
            }

            $user = $this->user_id
                ? tap(User::findOrFail($this->user_id))->update($userData)
                : User::create($userData);

            if ($this->role) {
                if ($this->edit_mode) {
                    $user->syncRoles([$this->role]);
                } else {
                    $user->assignRole($this->role);
                }
            }

            UserProfile::updateOrCreate(
                ['user_id' => $user->id],
                [
                    'type' => $this->type ?? 'consumer',
                    'phone' => $this->phone,
                    'company_name' => $this->company_name,
                    'npwp' => $this->npwp,
                    'address' => $this->address,
                ]
            );

            $this->dispatch('success', $this->edit_mode ? __('User updated successfully') : __('User created successfully'));
        });

        $this->resetInputFields();
    }

    #[On('update_user')]
    public function updateUser($id)
    {
        $this->edit_mode = true;

        $user = User::with('userProfile')->findOrFail($id);
        $this->user_id = $user->id;
        $this->name = $user->name;
        $this->email = $user->email;
        $this->role = $user->roles?->first()?->name ?? '';

        if ($user->userProfile) {
            $this->type = $user->userProfile->type ?? 'consumer';
            $this->phone = $user->userProfile->phone;
            $this->company_name = $user->userProfile->company_name;
            $this->npwp = $user->userProfile->npwp;
            $this->address = $user->userProfile->address;
        }
    }

    public function resetInputFields()
    {
        $this->reset([
            'user_id', 'name', 'password', 'email', 'role',
            'avatar', 'saved_avatar', 'type', 'phone',
            'company_name', 'npwp', 'address'
        ]);
        $this->edit_mode = false;
    }

    public function hydrate()
    {
        $this->resetErrorBag();
        $this->resetValidation();
    }
}
```

**Key rules for Pattern A:**

- Single standalone modal component (no base Form + Modal wrapper)
- Uses `edit_mode` boolean to switch between create/edit
- Edit event: `#[On('update_user')]` loads data into component properties
- Save method: `submit()`
- Success event: `$this->dispatch('success', message)` — Blade JS listens and reloads DataTable
- Modal embedded inside `card-toolbar` div
- `wire:ignore.self` on modal div

### Pattern B: Pure Livewire List + Modal (Roles)

**Blade View**

```
{{-- resources/views/admin/user-management/roles/list.blade.php --}}
<x-default-layout>

@section('title') Roles @endsection

@section('breadcrumbs')
    {{ Breadcrumbs::render('user-management.roles.index') }}
@endsection

<!--begin::Content container-->
<div id="kt_app_content_container" class="app-container container-xxl">
    <livewire:permission.role-list></livewire:permission.role-list>
</div>
<!--end::Content container-->

<!--begin::Modal-->
<livewire:permission.role-modal></livewire:permission.role-modal>
<!--end::Modal-->

</x-default-layout>
```

**List Component (`RoleList.php`)**

```
<?php

namespace App\Livewire\Permission;

use Illuminate\Database\Eloquent\Collection;
use Livewire\Attributes\On;
use Livewire\Component;
use Spatie\Permission\Models\Role;

class RoleList extends Component
{
    public array|Collection $roles;

    public function render()
    {
        $this->roles = Role::with('permissions')->get();
        return view('livewire.permission.role-list');
    }

    #[On('success')]
    public function updateRoleList()
    {
        $this->roles = Role::with('permissions')->get();
    }

    public function hydrate()
    {
        $this->resetErrorBag();
        $this->resetValidation();
    }
}
```

**Modal Component (`RoleModal.php`)**

```
<?php

namespace App\Livewire\Permission;

use Illuminate\Database\Eloquent\Collection;
use Illuminate\Support\Str;
use Livewire\Attributes\On;
use Livewire\Component;
use Spatie\Permission\Models\Permission;
use Spatie\Permission\Models\Role;

class RoleModal extends Component
{
    public $name;
    public $checked_permissions;
```

```
    public $check_all;

    public Role $role;
    public Collection $permissions;

    protected $rules = [
        'name' => 'required|string',
    ];

    #[On('modal.show.role_name')]
    public function mountRole($role_name = '')
    {
        if (empty($role_name)) {
            $this->role = new Role;
            $this->name = '';
            return;
        }

        $role = Role::where('name', $role_name)->first();
        if (is_null($role)) {
            $this->dispatch('error', 'The selected role [' . $role_name . '] is not found');
            return;
        }

        $this->role = $role;
        $this->name = $this->role->name;
        $this->checked_permissions = $this->role->permissions->pluck('name');
    }

    public function mount()
    {
        $this->permissions = Permission::all();
        $this->checked_permissions = [];
    }

    public function render()
    {
        $permissions_by_group = [];
        foreach ($this->permissions ?? [] as $permission) {
            $ability = Str::after($permission->name, ' ');
            $permissions_by_group[$ability][] = $permission;
        }

        return view('livewire.permission.role-modal', compact('permissions_by_group'));
    }

    public function submit()
    {
        $this->validate();

        $this->role->name = $this->name;
        if ($this->role->isDirty()) {
            $this->role->save();
        }

        $this->role->syncPermissions($this->checked_permissions);

        $this->dispatch('success', 'Permissions for ' . ucwords($this->role->name) . ' role updated');
    }

    public function checkAll()
    {
        if ($this->check_all) {
            $this->checked_permissions = $this->permissions->pluck('name');
        } else {
            $this->checked_permissions = [];
        }
    }

    public function hydrate()
    {
        $this->resetErrorBag();
        $this->resetValidation();
    }
}
```

**Key rules for Pattern B:**

- No DataTable; list is rendered by Livewire (`role-list.blade.php`)
- List component (`RoleList`) listens for `#[On('success')]` to refresh data
- Modal component (`RoleModal`) uses `mountRole()` instead of `edit_mode`
- Save method is `submit()` (not `save()`)
- Event: `#[On('modal.show.role_name')]`
- Modal embedded at bottom of Blade view

### Pattern C: DataTable + Standalone Modal with mountPermission() (Permissions)

**Blade View**

```
{{-- resources/views/admin/user-management/permissions/list.blade.php --}}
<div class="card">
    <div class="card-header border-0 pt-6">
        <div class="card-title">
            <input type="text" placeholder="Search permission" id="mySearchInput"/>
        </div>
        <div class="card-toolbar">
            <button type="button" class="btn btn-light-primary"
                    data-bs-toggle="modal" data-bs-target="#kt_modal_update_permission">
                Add Permission
            </button>
        </div>
    </div>
    <div class="card-body py-4">
        <div class="table-responsive">
            {!! $dataTable->table() !!}
        </div>
    </div>
</div>

<livewire:permission.permission-modal></livewire:permission.permission-modal>

@push('scripts')
    {!! $dataTable->scripts() !!}
    <script>
        document.addEventListener('livewire:load', function () {
            Livewire.on('success', function () {
                $('#kt_modal_update_permission').modal('hide');
                window.LaravelDataTables['permissions-table'].ajax.reload();
            });
        });
    </script>
@endpush
```

**Modal Component (`PermissionModal.php`)**

```php
<?php

namespace App\Livewire\Permission;

use Livewire\Attributes\On;
use Livewire\Component;
use Spatie\Permission\Models\Permission;

class PermissionModal extends Component
{
    public $name;
    public Permission $permission;

    protected $rules = [
        'name' => 'required|string',
    ];

    public function render()
    {
        return view('livewire.permission.permission-modal');
    }

    #[On('modal.show.permission_name')]
    public function mountPermission($permission_name = null)
    {
        if (empty($permission_name)) {
            $this->permission = new Permission;
            $this->name = '';
            return;
        }

        $permission = Permission::find($permission_name);
        if (is_null($permission)) {
            $this->dispatch('error', 'The selected permission is not found');
            return;
        }

        $this->permission = $permission;
        $this->name = $this->permission->name;
    }

    public function submit()
    {
        $this->validate();

        $this->permission->name = strtolower($this->name);
        if ($this->permission->isDirty()) {
            $this->permission->save();
        }

        $this->dispatch('success', 'Permission updated');
    }

    #[On('delete_permission')]
    public function delete($id)
    {
        $permission = Permission::find($id);
        if (!is_null($permission)) {
            $permission->delete();
        }
        $this->dispatch('success', 'Permission deleted');
    }

    public function hydrate()
    {
        $this->resetErrorBag();
        $this->resetValidation();
    }
}
```

**Key rules for Pattern C:**

- DataTable for list, standalone modal for add/edit
- Modal uses `mountPermission()` instead of `edit()` or `edit_mode`
- Save method is `submit()`
- Event: `#[On('modal.show.permission_name')]`
- Delete event: `#[On('delete_permission')]`
- Success dispatches message string: `$this->dispatch('success', 'Permission updated')`

### Pattern D: Base Form + Modal Wrapper (Most CRUD Modules)

Most modules (Properties, Bookings, etc.) use a **base `Form.php`** class plus a thin **`Modal.php`** wrapper.

**Example: `PropertyPrices/Form.php` (base form)**

```php
<?php

namespace App\Livewire\PropertyPrices;

use App\Models\PropertyPrice;
use App\Models\Property;
use Livewire\Attributes\On;
use Livewire\Component;

class Form extends Component
{
    public ?int $propertypriceId = null;

    public $property_id = null;
    public $price_type = null;
    public $amount = '';
    public $cleaning_fee = '';
    public $deposit = '';
    public $service_fee = '';
    public $extra_guest_fee = '';

    protected function rules(): array
    {
        return [
            'property_id' => 'required|exists:properties,id',
            'price_type' => 'required|in:daily,weekly,monthly,yearly',
            'amount' => 'required|numeric|min:0',
            'cleaning_fee' => 'nullable|numeric|min:0',
            'deposit' => 'nullable|numeric|min:0',
            'service_fee' => 'nullable|numeric|min:0',
            'extra_guest_fee' => 'nullable|numeric|min:0',
        ];
    }

    #[On('modal.show.property_price')]
    public function edit($id)
    {
        if ($id) {
            $propertyprice = PropertyPrice::findOrFail($id);
            $this->propertypriceId = $propertyprice->id;
            $this->property_id = $propertyprice->property_id;
            $this->price_type = $propertyprice->price_type;
            $this->amount = $propertyprice->amount;
            $this->cleaning_fee = $propertyprice->cleaning_fee;
            $this->deposit = $propertyprice->deposit;
            $this->service_fee = $propertyprice->service_fee;
            $this->extra_guest_fee = $propertyprice->extra_guest_fee;
        } else {
            $this->reset(['property_id', 'price_type', 'amount', 'cleaning_fee', 'deposit', 'service_fee', 'extra_guest_fee']);
            $this->propertypriceId = null;
        }
    }

    public function save()
    {
        $validated = $this->validate();

        foreach (['cleaning_fee', 'deposit', 'service_fee', 'extra_guest_fee'] as $key) {
            if ($validated[$key] === '') {
                $validated[$key] = null;
            }
        }

        if ($this->propertypriceId) {
            PropertyPrice::findOrFail($this->propertypriceId)->update($validated);
            session()->flash('success', 'Manajemen Harga updated successfully.');
        } else {
            PropertyPrice::create($validated);
            session()->flash('success', 'Manajemen Harga created successfully.');
        }

        $this->dispatch('success');
    }

    public function render()
    {
        $property_idOptions = Property::pluck('name', 'id');
        return view('livewire.property_prices.form', compact('property_idOptions'));
    }

    public function hydrate()
    {
        $this->resetErrorBag();
        $this->resetValidation();
    }
}
```

**Example: `PropertyPrices/PropertyModal.php` (thin wrapper)**

```php
<?php

namespace App\Livewire\PropertyPrices;

class PropertyModal extends Form
{
    public function render()
    {
        return parent::render();
    }
}
```

**Key rules for Pattern D:**

- The **base `Form.php`** contains all properties, rules, `edit()` handler, `save()` method, and `render()`
- The **`Modal.php`** wrapper only overrides `render()` to call `parent::render()`
- Event handler: `edit($id)`
- Save method: `save()`
- Event: `#[On('modal.show.module_name')]`
- `hydrate()` resets validation errors

---

## Livewire Blade Pattern (Modal)

### Struktur Modal Standar

```blade
<div class="modal fade" id="kt_modal_add_resource" tabindex="-1" aria-hidden="true" wire:ignore.self>
    <div class="modal-dialog modal-dialog-centered mw-650px">
        <div class="modal-content">
            <div class="modal-header" id="kt_modal_add_resource_header">
                <h2 class="fw-bold">Add Resource</h2>
                <div class="btn btn-icon btn-sm btn-active-icon-primary" data-kt-dismiss-modal="true">
                    {!! getIcon('cross','fs-1') !!}
                </div>
            </div>

            <div class="modal-body px-5 my-7">
                <form id="kt_modal_add_resource_form" class="form" wire:submit.prevent="submit">
                    <div class="d-flex flex-column scroll-y me-n7 pe-7" id="kt_modal_add_resource_scroll"
                         data-kt-scroll="true" data-kt-scroll-activate="{default: false, lg: true}"
                         data-kt-scroll-max-height="auto"
                         data-kt-scroll-dependencies="#kt_modal_add_resource_header"
                         data-kt-scroll-wrappers="#kt_modal_add_resource_scroll"
                         data-kt-scroll-offset="300px">

                        <!-- Form fields here -->

                    </div>
                </form>
            </div>

            <div class="modal-footer flex-center">
                <button type="reset" class="btn btn-light me-3" data-kt-dismiss-modal="true"
wire:loading.attr="disabled">Discard</button>
                <button type="submit" class="btn btn-primary" form="kt_modal_add_resource_form">
                    <span class="indicator-label" wire:loading.remove>Submit</span>
                    <span class="indicator-progress" wire:loading wire:target="submit">
                        Please wait...
                        <span class="spinner-border spinner-border-sm align-middle ms-2"></span>
                    </span>
                </button>
            </div>
        </div>
    </div>
</div>

@push('scripts')
<script>
    const modal = document.querySelector('#kt_modal_add_resource');

    modal.addEventListener('show.bs.modal', (e) => {
        const {module}Id = e.relatedTarget ? e.relatedTarget.getAttribute('data-{module}-id') : null;
        Livewire.emit('modal.show.{module_name', {module}Id);
    });
</script>
@endpush
```

**Key rules:**

- `wire:ignore.self` on the modal div prevents Livewire from re-rendering the modal
- `wire:model.defer` for two-way binding (deferred until submit)
- `wire:submit.prevent="submit"` intercepts form submit
- `wire:loading` / `wire:loading.remove` for loading states
- `@error('field')` shows validation errors
- JavaScript `show.bs.modal` event dispatches a Livewire event with the selected record ID

### Modal Communication Flow

The modal communication follows this exact sequence:

```
1. User clicks button in Blade (e.g., "Edit Role")
   → data-bs-toggle="modal" data-bs-target="#kt_modal_update_role"
   → data-role-id="{{ $role->name }}" is passed via HTML attribute

2. Bootstrap shows the modal
   → 'show.bs.modal' event fires

3. JavaScript listener catches the event
   → Livewire.dispatch('modal.show.role_name', { role_name: roleId })

4. Livewire component receives the event via #[On('modal.show.role_name')]
   → mountPermission() / mountRole() is called
   → Model is loaded or new instance created
   → Form fields are populated

5. User fills form and clicks Submit
   → wire:submit.prevent="submit"
   → Validation runs
   → Model is saved

6. Livewire dispatches 'success' event
   → $this->dispatch('success', 'Permission updated')

7. Blade JavaScript listens for 'success'
   → $('#modal').modal('hide')
   → window.LaravelDataTables['table-id'].ajax.reload()
```

---

### Three Module Patterns Summary

| Pattern | When to Use | List View | Add/Edit |
|---|---|---|---|
| **A: DataTable + Livewire Modal** | Large datasets needing search/sort/export | Yajra DataTable | Standalone Livewire modal |
| **B: Pure Livewire** | Simple card/grid layouts, small datasets | Livewire list component | Standalone Livewire modal |
| **C: Base Form + Modal Wrapper** | Reusable form logic across create/edit | Same as Pattern A | Base `Form.php` + thin `Modal.php` wrapper |

---

### Checklist Implementasi Modul Baru

- [ ] Buat folder `resources/views/admin/{module}/`
- [ ] Buat `index.blade.php` dengan card + DataTable + Livewire modal
- [ ] Buat `columns/_actions.blade.php` dengan variabel yang sesuai DataTable
- [ ] Buat DataTable class di `app/DataTables/{Module}DataTable.php`
- [ ] Buat Controller di `app/Http/Controllers/Admin/{Module}Controller.php`
- [ ] Daftarkan route di `routes/admin_generated.php`
- [ ] Daftarkan breadcrumb di `routes/breadcrumbs.php`
- [ ] Gunakan `<x-default-layout>` sebagai wrapper
- [ ] Include `@push('scripts')` untuk DataTable scripts + Livewire events
- [ ] Gunakan kelas CSS standar: `card`, `card-header`, `card-body`, `btn btn-primary`, `form-control form-control-solid`

<!-- Source: configuration/ui-patterns.md -->

# Standard UI Patterns

Dokumen ini mendefinisikan pola UI standar untuk blade admin Laravel. Semua blade baru harus mengikuti pola ini agar konsisten.

### Daftar Isi


- Layout Wrapper
- Pola Index / DataTable (List)
- Pola Action Column (DataTable)
- Pola Form (Livewire Modal)
- Pola Show / Detail Page
- Pola Multi-Step Form Wizard (Property & Hotel)
- Checklist Blade Baru


### 1. Layout Wrapper

Semua halaman admin dibungkus dengan layout default:

```
<x-default-layout>

    @section('title', 'Page Title')

    <!-- Konten di sini -->

</x-default-layout>
```

- `title`: String judul halaman.
- `breadcrumbs`: Render breadcrumb dari package `diegoegidio/laravel-breadcrumbs`. Pastikan route breadcrumb terdaftar di `routes/breadcrumbs.php`.

---

## 2. Pola Index / DataTable (List)

### 2.1 Struktur Card DataTable

Pola standar untuk halaman daftar dengan DataTables:

```
<x-default-layout>

    @section('title', 'Resource Name')

    <div class="card">
        <div class="card-header border-0 pt-6">
            <div class="card-title">
                <div class="d-flex align-items-center position-relative my-1">
                    {!! getIcon('magnifier', 'fs-3 position-absolute ms-5') !!}
                    <input type="text" data-kt-user-table-filter="search" class="form-control form-control-solid w-250px ps-13"
placeholder="Search..." id="mySearchInput"/>
                </div>
            </div>
            <div class="card-toolbar">
                <button type="button" class="btn btn-primary" data-bs-toggle="modal" data-bs-target="#kt_modal_add_resource">
                    {!! getIcon('plus', 'fs-2', '', 'i') !!}
                    Add Resource
                </button>
            </div>
        </div>
        <div class="card-body py-4">
            <div class="table-responsive">
                {!! $dataTable->table() !!}
            </div>
        </div>
    </div>

    @push('scripts')
        {!! $dataTable->scripts() !!}
        <script>
            document.getElementById('mySearchInput').addEventListener('keyup', function () {
                window.LaravelDataTables['resource-table'].search(this.value).draw();
            });
        </script>
    @endpush

</x-default-layout>
```

### 2.2 Alternatif Index Sederhana

Jika tidak butuh search + modal, gunakan struktur minimal:

```
<x-default-layout>

    @section('title', 'Resource Name')

    <div class="card">
        <div class="card-header d-flex justify-content-between align-items-center">
            <h5 class="mb-0">Resource Name</h5>
            <a href="{{ route('admin.resource.create') }}" class="btn btn-primary btn-sm">+ Tambah</a>
        </div>
        <div class="card-body">
            {!! $dataTable->table() !!}
        </div>
    </div>

    @push('scripts')
        {!! $dataTable->scripts() !!}
    @endpush

</x-default-layout>
```

---

## 3. Pola Action Column (DataTable)

Setiap DataTable memiliki kolom aksi. File blade action disimpan di:

```
resources/views/admin/{module}/columns/_actions.blade.php
```

### 3.1 Standar Action Column

```
<a href="{{ route('admin.resource.edit', $resource) }}" class="btn btn-sm btn-warning">Edit</a>
<form action="{{ route('admin.resource.destroy', $resource) }}" method="POST" class="d-inline" onsubmit="return confirm('Hapus data ini?')">
    @csrf
    @method('DELETE')
    <button type="submit" class="btn btn-sm btn-danger">Hapus</button>
</form>
```

- Variabel `$resource` harus sesuai dengan nama variabel yang di-pass di DataTable.
- Route `edit` dan `destroy` harus terdaftar di `routes/admin_generated.php`.

### 3.2 Contoh Variabel per Modul

| Modul | Variabel | Route Prefix |
|---|---|---|
| users | `$user` | `admin.users` |
| properties | `$property` | `admin.properties` |
| bookings | `$booking` | `admin.bookings` |
| payments | `$payment` | `admin.payments` |
| orders | `$order` | `admin.orders` |
| banners | `$banner` | `admin.banners` |
| referrals | `$referral` | `admin.referrals` |

---

## 4. Pola Modal (Create / Edit)

### 4.1 Struktur Modal Standar

```
<div class="modal fade" id="kt_modal_add_resource" tabindex="-1" aria-hidden="true">
    <div class="modal-dialog modal-dialog-centered mw-650px">
        <div class="modal-content">
            <div class="modal-header" id="kt_modal_add_resource_header">
                <h2 class="fw-bold">Add Resource</h2>
                <div class="btn btn-icon btn-sm btn-active-icon-primary" data-bs-dismiss="modal" aria-label="Close">
                    {!! getIcon('cross','fs-1') !!}
                </div>
            </div>
            <div class="modal-body px-5 my-7">
                <form id="kt_modal_add_resource_form" class="form" wire:submit.prevent="submit">
                    <!-- Form fields here -->
                </form>
            </div>
            <div class="modal-footer flex-center">
                <button type="reset" class="btn btn-light me-3" data-bs-dismiss="modal" aria-label="Close" wire:loading.attr="disabled">Discard</button>
                <button type="submit" class="btn btn-primary" form="kt_modal_add_resource_form">
                    <span class="indicator-label" wire:loading.remove>Submit</span>
                    <span class="indicator-progress" wire:loading wire:target="submit">
                        Please wait...
                        <span class="spinner-border spinner-border-sm align-middle ms-2"></span>
                    </span>
                </button>
            </div>
        </div>
    </div>
</div>
```

---

## 5. Pola Form (Create / Edit Page)

### 5.1 Form via Livewire Component

```
<x-default-layout>

    @section('title', 'Tambah Resource')

    <div class="card">
        <div class="card-header"><h5 class="mb-0">Tambah Resource</h5></div>
        <div class="card-body">
            @livewire('resource.form')
        </div>
    </div>

</x-default-layout>
```

### 5.2 Kelas Form yang Digunakan

| Kelas | Keterangan |
|---|---|
| `form-control form-control-solid` | Input text/email/number standar |
| `form-select form-select-solid` | Select dropdown |
| `form-check-input` | Checkbox/radio |
| `fv-row mb-7` | Wrapper input group |
| `fs-6 fw-semibold mb-2` | Label standar |

---

## 6. Pola Show / Detail Page

```
<x-default-layout>

    @section('title', 'Resource Detail')

    <div class="d-flex flex-column flex-lg-row">
        <div class="flex-column flex-lg-row-auto w-lg-250px w-xl-350px mb-10">
            <div class="card mb-5 mb-xl-8">
                <div class="card-body">
                    <!-- Sidebar content here -->
                </div>
            </div>
        </div>
        <div class="flex-lg-row-fluid ms-lg-15">
            <div class="card card-flush mb-6 mb-xl-9">
                <div class="card-header pt-5">
                    <div class="card-title">
                        <h2 class="d-flex align-items-center">Title</h2>
                    </div>
                </div>
                <div class="card-body pt-0">
                    <!-- Content here -->
                </div>
            </div>
        </div>
    </div>

</x-default-layout>
```

---

## 7. Pola Kolom DataTable Kustom

### 7.1 Kolom dengan View Kustom

```
// Di DataTable class
->editColumn('user', function (User $user) {
    return view('admin.users.columns._user', compact('user'));
})
```

```
<!-- resources/views/admin/users/columns/_user.blade.php -->
<div class="symbol symbol-circle symbol-50px overflow-hidden me-3">
    <!-- Avatar -->
</div>
<div class="d-flex flex-column">
    <a href="{{ route('admin.users.show', $user) }}" class="text-gray-800 text-hover-primary mb-1">
        {{ $user->name }}
    </a>
    <span>{{ $user->email }}</span>
</div>
```

### 7.2 Kolom Action dengan Raw HTML

```
->addColumn('action', function (User $user) {
    return view('admin.users.columns._actions', compact('user'))->render();
})
->rawColumns(['action']);
```

---

## 8. Pola Script & Push

Setiap halaman yang menggunakan DataTable harus menyertakan script di `@push('scripts')`:

```
@push('scripts')
    {!! $dataTable->scripts() !!}
    <script>
        // Custom JS untuk DataTable search, modal reload, dll
    </script>
@endpush
```

---

## 9. Pola Ikon

Gunakan helper `getIcon()` untuk konsistensi ikon:

```
{!! getIcon('magnifier', 'fs-3 position-absolute ms-5') !!}
{!! getIcon('plus', 'fs-2', '', 'i') !!}
```

Format: `getIcon('icon-name', 'size-class', 'extra-class', 'tag')`

---

## 10. Pola Tabel Responsif

Semua tabel DataTable dibungkus dengan wrapper responsif:

```
<div class="table-responsive">
    {!! $dataTable->table() !!}
</div>
```

---

## 11. Pola Struktur Folder

```
resources/views/admin/
├── {module}/
│   ├── columns/
│   │   ├── _actions.blade.php
│   │   ├── _user.blade.php
│   │   └── _draw-scripts.js
│   ├── modals/
│   │   ├── _form.blade.php
│   │   └── _detail.blade.php
│   ├── index.blade.php
│   ├── create.blade.php
│   ├── edit.blade.php
│   └── show.blade.php
```

---

## 12. Checklist Implementasi Modul Baru

- [ ] Buat folder `resources/views/admin/{module}/`
- [ ] Buat `index.blade.php` dengan card + DataTable
- [ ] Buat `columns/_actions.blade.php` dengan variabel yang sesuai DataTable
- [ ] Buat DataTable class di `app/DataTables/{Module}DataTable.php`
- [ ] Buat Controller di `app/Http/Controllers/Admin/{Module}Controller.php`
- [ ] Daftarkan route di `routes/admin_generated.php`
- [ ] Daftarkan breadcrumb di `routes/breadcrumbs.php`
- [ ] Gunakan `<x-default-layout>` sebagai wrapper
- [ ] Include `@push('scripts')` untuk DataTable scripts
- [ ] Gunakan kelas CSS standar: `card`, `card-header`, `card-body`, `btn btn-primary`, `form-control form-control-solid`

---

## 13. Pola Livewire Modal

Semua form Create/Edit harus menggunakan Livewire modal. Livewire form component harus dibungkus dalam modal Bootstrap agar konsisten dengan UI standar.

### 13.1 Struktur Livewire Modal Component

```
<div class="modal fade" id="kt_modal_add_{module}" tabindex="-1" aria-hidden="true" wire:ignore.self>
    <div class="modal-dialog modal-dialog-centered mw-650px">
        <div class="modal-content">
            <div class="modal-header" id="kt_modal_add_{module}_header">
                <h2 class="fw-bold">@if({module}Id) Edit {Module Title} @else Add {Module Title} @endif</h2>
                <div class="btn btn-icon btn-sm btn-active-icon-primary" data-bs-dismiss="modal" aria-label="Close">
                    {!! getIcon('cross','fs-1') !!}
                </div>
            </div>
            <div class="modal-body px-5 my-7">
                <form id="kt_modal_add_{module}_form" class="form" wire:submit.prevent="submit">
                    <!-- Form fields here -->
                </form>
            </div>
            <div class="modal-footer flex-center">
                <button type="reset" class="btn btn-light me-3" data-bs-dismiss="modal" aria-label="Close"
wire:loading.attr="disabled">Discard</button>
                <button type="submit" class="btn btn-primary" form="kt_modal_add_{module}_form">
                    <span class="indicator-label" wire:loading.remove>Submit</span>
                    <span class="indicator-progress" wire:loading wire:target="submit">
                        Please wait...
                        <span class="spinner-border spinner-border-sm align-middle ms-2"></span>
                    </span>
                </button>
            </div>
        </div>
    </div>
</div>
```

### 13.2 Include Modal di Index Page

```
<livewire:{module}.{module}-modal></livewire:{module}.{module}-modal>
```

### 13.3 Action Column dengan Modal Trigger

```
<button type="button" class="btn btn-sm btn-warning" data-{module}-id="{{ $resource->id }}" data-bs-toggle="modal" data-bs-
target="#kt_modal_add_{module}">
    Edit
</button>
```

### 13.4 Livewire Component Class

```
protected $listeners = ['modal.show.{module_name}' => 'edit'];

public function edit({module}Id)
{
    {module}Id = {module}Id ?: null;
    // Load data...
}
```

---

## 14. Pola JavaScript (Custom App JS)

Setiap modul admin yang memiliki halaman index dengan DataTable dan/atau modal Livewire harus memiliki file JavaScript kustom di `resources/_themes/src/js/custom/apps/{module}/`.

### 14.1 Struktur Folder JS

```
resources/_themes/src/js/custom/apps/
├── {module}/
│   ├── list.js
│   ├── add.js
│   └── edit.js
```

### 14.2 Pola `list.js` (DataTable + Search)

```
"use strict";

var KT{Module}List = function () {
    var table = document.getElementById('kt_table_{module}');
    var datatable;

    var initTable = function () {
        if (!table) return;
        datatable = $(table).DataTable({
            "info": false,
            'order': [],
            'columnDefs': [
                { orderable: false, targets: 0 },
            ]
        });
    }

    var handleSearchDatatable = () => {
        const filterSearch = document.querySelector('[data-kt-{module}-table-filter="search"]');
        if (filterSearch) {
            filterSearch.addEventListener('keyup', function (e) {
                datatable.search(e.target.value).draw();
            });
        }
    }

    return {
        init: function () {
            initTable();
            handleSearchDatatable();
        }
    };
}();

KTUtil.onDOMContentLoaded(function () {
    KT{Module}List.init();
});
```

### 14.3 Menyertakan JS di Controller

```
public function index()
{
    addJavascriptFile('assets/js/custom/apps/{module}/list.js');
    return view('admin.{module}.index', compact('dataTable'));
}
```

### 14.4 Compile JS

Setiap kali menambah atau mengubah JS, jalankan:

```
npm run prod
```

---

## 15. Konvensi Nama Modal per Modul

| Modul | Modal ID | Data Attribute | Livewire Component |
|---|---|---|---|
| audit_logs | `kt_modal_add_audit_log` | `data-auditlog-id` | `audit_logs.audit_logs-modal` |
| availabilities | `kt_modal_add_availability` | `data-availability-id` | `availabilities.availabilities-modal` |
| bookings | `kt_modal_add_booking` | `data-booking-id` | `bookings.bookings-modal` |
| payments | `kt_modal_add_payment` | `data-payment-id` | `payments.payments-modal` |
| properties | `kt_modal_add_property` | `data-property-id` | `properties.properties-modal` |

| | | | |
|---|---|---|---|
| referrals | `kt_modal_add_referral` | `data-referral-id` | `referrals.referrals-modal` |
| vouchers | `kt_modal_add_voucher` | `data-voucher-id` | `vouchers.vouchers-modal` |
| withdrawals | `kt_modal_add_withdrawal` | `data-withdrawal-id` | `withdrawals.withdrawals-modal` |

---

*Dokumen ini dibuat berdasarkan pola yang ada di `resources/views/admin/user-management/` dan modul admin lainnya.*

<!-- Source: configuration/missing-files.md -->

# Missing Files Analysis

Dokumen ini merangkum hasil audit file dan relasi yang belum terdefinisi di proyek Laravel, termasuk model, relationship, DataTables path mismatch, dan namespace issues.

## Daftar Isi


- Missing / Undefined Model Classes
- Missing Eloquent Model Relationships
- DataTables Action Column Path & Variable Mismatches
- Outdated `pages/apps...` Paths in User Management DataTables
- Livewire Namespace & Template Mismatches
- DataTable Import & Typo Issues


## 1. Missing / Undefined Model Classes

| Model | Status |
|---|---|
| `UserValidation` | ☑ Created — diimport di `UserValidationDataTable`, migration disalin dari project viona |
| `UsersAssignedRole` | ☑ Created — diimport di `UsersAssignedRoleDataTable`, migration dibuat |
| **21 Model Ternormalisasi Produk** | ☑ **All Created** — `Accommodation`, `BuildingList`, `UnitType`, `UnitTypePrice`, `UnitTypeImage`, `FacilityGroup`, `FacilityItem`, `FacilityTag`, `Gallery`, `Review`, `ReviewPhoto`, `AccommodationPolicy`, `AccommodationQuestion`, `NearbyPoint`, `RoomFurnitureGroup`, `RoomFurnitureItem`, `BedFacility`, `AccommodationDescriptionParagraph`, `AccommodationDetail`, `AccommodationDetailGroup`, `AccommodationDetailLine` |

---

## 2. Missing Eloquent Model Relationships

### A. `User` Model (`app/Models/User.php`)

Model ini belum memiliki relasi `hasMany` ke model-model domain. Ini **tidak dikonfigurasi** di setup project saat ini, jadi tidak ditambahkan.

### B. `Accommodation` Model (`app/Models/Accommodation.php`)

Model ini sekarang menjadi single source of truth untuk Hotel dan Property. `Property.php` dan `Hotel.php` telah dihapus. Semua relasi terkonsolidasi di `Accommodation.php` dengan diskriminator `booking_mode` (`room` = Hotel, `unit` = Property).

### C. `RemoteConfigParameter` Model

☑ Fixed — Ditambahkan relasi `createdBy()` dan `updatedBy()` (fields: `created_by`, `updated_by`)

### D. `RemoteConfigPublishLog` Model

☑ Fixed — Ditambahkan relasi `publishedBy()` (field: `published_by`)

---

## 3. DataTables Action Column Path & Variable Mismatches

Semua DataTables di `app/DataTables/` telah diperbaiki untuk menggunakan path `_actions.blade.php` yang benar:

| DataTable Class | View Path | Status |
|---|---|---|
| `AuditLogDataTable` | `admin.audit_logs.columns._actions` | ☑ Fixed |
| `AvailabilityDataTable` | `admin.availabilities.columns._actions` | ☑ Fixed |
| `BannerDataTable` | `admin.banners.columns._actions` | ☑ Fixed |
| `BookingDataTable` | `admin.bookings.columns._actions` | ☑ Fixed |
| `CmsPageDataTable` | `admin.cms_pages.columns._actions` | ☑ Fixed |
| `ConversationDataTable` | `admin.conversations.columns._actions` | ☑ Fixed |
| `FaqDataTable` | `admin.faqs.columns._actions` | ☑ Fixed |
| `NotificationLogDataTable` | `admin.notification_logs.columns._actions` | ☑ Fixed |
| `OrderDataTable` | `admin.orders.columns._actions` | ☑ Fixed |
| `PaymentDataTable` | `admin.payments.columns._actions` | ☑ Fixed |
| `PaymentMethodDataTable` | `admin.payment_methods.columns._actions` | ☑ Fixed |
| `PropertyDataTable` | `admin.properties.columns._actions` | ☑ Fixed |
| `PropertyPriceDataTable` | `admin.property_prices.columns._actions` | ☑ Fixed |
| `PropertyRecommendationDataTable` | `admin.property_recommendations.columns._actions` | ☑ Fixed |
| `PropertySearchLogDataTable` | `admin.property_search_logs.columns._actions` | ☑ Fixed |
| `ReferralDataTable` | `admin.referrals.columns._actions` | ☑ Fixed |
| `ReviewDataTable` | `admin.reviews.columns._actions` | ☑ Fixed |
| `UserProfileDataTable` | `admin.user_profiles.columns._actions` | ☑ Fixed |
| `VoucherDataTable` | `admin.vouchers.columns._actions` | ☑ Fixed |
| `WishlistDataTable` | `admin.wishlists.columns._actions` | ☑ Fixed |

| | | |
|---|---|---|
| `WithdrawalDataTable` | `admin.withdrawals.columns._actions` | ☑ Fixed |

Nama variabel di `_actions.blade.php` sekarang cocok dengan variabel yang di-pass oleh DataTable closures masing-masing.

---

## 4. Outdated `pages/apps...` Paths in User Management DataTables

☑ Fixed — Semua 4 User Management DataTables diperbarui untuk menggunakan path `admin/user-management...`:

1. `PermissionsDataTable.php` — path `_assign-to` dan `_actions`
2. `UsersDataTable.php` — path `_user` dan `_actions`
3. `UserValidationDataTable.php` — path `_user` dan `_actions`
4. `UsersAssignedRoleDataTable.php` — path `_user` dan `_actions`

---

## 5. Livewire Namespace & Template Mismatches

### A. Namespace Mismatch

☑ Fixed — Semua Livewire Permission components diperbarui dari `App\Http\Livewire` ke `App\Livewire`:

- `app/Livewire/Permission/RoleModal.php`
- `app/Livewire/Permission/RoleList.php`
- `app/Livewire/Permission/PermissionModal.php`

### B. View Template Paths

☑ Verified — Semua Livewire form render methods cocok dengan `resources/views/livewire/...`.

---

## 6. DataTable Import & Typo Issues

### A. `UsersAssignedRoleDataTable.php`

☑ Fixed — Dikoreksi import dari `App\Models\UsersAssingedRole` ke `App\Models\UsersAssignedRole`

### B. Missing Model

☑ Created — `app/Models/UsersAssignedRole.php` dengan relasi `user()` dan `role()`

### C. Missing Migration

☑ Created — `database/migrations/2026_06_16_000003_create_users_assigned_roles_table.php`

---

## 7. RolePermissionSeeder Status

`RolePermissionSeeder.php` bekerja dengan benar:

- Membuat roles: `admin`, `host`, `consumer`
- Assign `admin` role ke `admin@example.com`
- Assign `host` role ke `budi.host@example.com` dan `siti.host@example.com`

---

## 8. Admin Blade Layout Conversion

☑ Completed — Semua 65 admin blade files dikonversi dari `@extends('layouts.admin')` ke `<x-default-layout>` component-based approach.

☑ Completed — Dibuat 21 direktori `columns/` dengan file `_actions.blade.php` untuk semua section admin dengan DataTables.

---

## Summary of Completed Fixes

| Issue | Status |
|---|---|
| DataTables action column paths | ☑ Fixed |
| DataTables variable name mismatches | ☑ Fixed |
| `pages/apps...` paths in User Management DataTables | ☑ Fixed |
| Livewire namespace (`Http\Livewire` → `Livewire`) | ☑ Fixed |
| `UsersAssignedRole` typo in import | ☑ Fixed |
| Missing `UsersAssignedRole` model | ☑ Created |
| Missing `user_validations` migration | ☑ Copied from viona |
| Missing `users_assigned_roles` migration | ☑ Created |
| Missing `createdBy`/`updatedBy` in `RemoteConfigParameter` | ☑ Fixed |
| Missing `publishedBy` in `RemoteConfigPublishLog` | ☑ Fixed |

---

## Remaining Items (Not Configured)

Item berikut **tidak dikonfigurasi** di project saat ini dan **tidak boleh** ditambahkan:

1. **`User` model `hasMany` relationships** — Tidak dikonfigurasi di project saat ini
2. **`Property` model additional relationships** — Tidak dikonfigurasi di project saat ini
3. **Detailed permission creation in seeder** — Tidak dikonfigurasi di project saat ini
4. **`pages/` directory views** — Tidak digunakan di rental project (menggunakan struktur `admin/`)

<!-- Source: sfds.md -->

# Arsitektur Standar Dokumentasi — Livirooms Multi-Accommodation

Dokumentasi ini memetakan struktur file di dalam folder `docs/` agar selaras dengan implementasi nyata pada Laravel 12 backend dan Flutter mobile.

### Struktur Direktori `docs/`

```
docs/
├── INDEX.md                          # Indeks navigasi global
├── README.md                         # Gambaran umum sistem & stack
├── ai/                               # Context pack untuk AI
│   ├── 00-orientation.md             # Titik masuk & aturan navigasi
│   ├── 01-backend-map.md             # Arsitektur backend, 21 model, payment, Livewire
│   ├── 02-frontend-map.md            # Arsitektur mobile, 45 layar, router, model
│   ├── 03-api-contract.md            # Kontrak endpoint API & Dart service mapping
│   └── 04-drift-register.md          # Register perbaikan drift & audit data model
├── configuration/                    # Konfigurasi admin & UI patterns
│   ├── admin-panel.md                # Panduan Livewire wizard & Yajra DataTables
│   ├── missing-files.md              # Log audit inventaris file
│   └── ui-patterns.md                # Template Blade & Livewire UI
├── domain/                           # Master domain contracts
│   └── hotel-vs-property.md          # Kontrak arsitektur 21 tabel (3NF), booking_mode
├── features/                         # Dokumentasi spesifik fitur
│   ├── payments.md                   # Integrasi payment gateway & Midtrans
│   └── referral.md                   # Sistem kode referral & reward
├── payments/                         # Detail transaksi pembayaran
│   └── system.md                     # Spesifikasi 6 endpoint payment & webhook
├── response/                         # Master fixture JSON API
│   ├── detail-property.json          # Sample output PropertyShape
│   ├── home.json                     # Sample output Home Feed
│   └── hotels.json                   # Sample output HotelShape
└── workflows/                        # Alur pengujian & eksekusi API
    └── run_api_sequence.xml          # Urutan request API end-to-end
```

<!-- Source: mitra-accommodation-api.md -->

# Mitra Accommodation — API Contract & Migration Guide

> Dokumentasi teknis modul Mitra Accommodation: respons API `room` vs `unit`, skema payload request, dan panduan migrasi.

### 1. Ikhtisar

| `booking_mode` | Tipe | Contoh | Booking Jalur |
|---|---|---|---|
| `room` | Hotel / Guest House / Inventory-based | Hotel, Motel, Penginapan | Per tipe kamar (`unit_types`) |
| `unit` | Apartemen / Vila / Kost / Property-based | Apartment, Villa, Rumah | Per unit utuh |

Flutter: `accommodation_form_screen.dart`

- `bookingMode` null → mode selector.
- `room` → 5 tab.
- `unit` → 4 tab.

---

### 2. API Response — Room Mode (`room`)

Endpoint: `GET host/hotels/{id}` atau `GET accommodations/{id}`

Resource: `HostAccommodationResource::roomShape()` atau `AccommodationResource::hotelShape()`


```
{
  "status": true,
  "code": 200,
  "message": "Detail hotel berhasil dimuat.",
  "data": {
    "id": 1,
    "booking_mode": "room",
    "star_rating": 5,
    "open_year": 2010,
    "check_in_time": "14:00:00",
    "check_out_time": "12:00:00",
    "deposit_amount": 500000.00,
    "name": "Merlynn Park Hotel",
    "description": "...",
    "address": "Jl. K.H. Hasyim Ashari No. 29-31, Jakarta Pusat",
    "city_name": "Jakarta Pusat",
    "latitude": -6.166667,
    "longitude": 106.816667,
    "cancellation_policy": "Pembatalan gratis hingga 24 jam sebelum check-in.",
    "price_from": 685000,
    "price_original": 950000,
```

```
    "gallery": ["https://images.unsplash.com/photo-..."],
    "facilities": [
      {
        "category": "Fasilitas Utama",
        "icon": "star",
        "items": [
          {"name": "Resepsionis & Keamanan 24 Jam", "is_available": true, "badge": null}
        ]
      }
    ],
    "facility_items": [
      {"id": 1, "name": "AC", "icon": "ac", "context": "facility", "is_available": true, "badge": null}
    ],
    "nearby_points": [
      {"name": "Stasiun Gambir", "type": "train", "distance": "1,2 km", "duration": "15 menit", "icon": "train"}
    ],
    "questions": [
      {"id": 1, "question": "Apakah tersedia laundry?", "answer": "Ya, 24 jam.", "author_name": "Manager", "author_role": "Staff"}
    ],
    "policies": {
      "general": [{"section": "general", "icon": "info", "tone": "neutral", "title": "Informasi Umum", "paragraphs": ["Check-in mulai
pukul 14:00."], "sort_order": 1}],
      "cancellation": [{"section": "cancellation", "icon": "cancel", "tone": "warning", "title": "Pembatalan", "paragraphs":
["Pembatalan gratis hingga 24 jam."], "sort_order": 1}],
      "smoking_policy": [{"section": "smoking_policy", "icon": "smoke_free", "tone": "info", "title": "Merokok", "paragraphs":
["Semua kamar bebas rokok."], "sort_order": 1}],
      "extra_bed_policy": [{"section": "extra_bed_policy", "icon": "bed", "tone": "info", "title": "Ranjang Tambahan", "paragraphs":
["Tersedia atas permintaan."], "sort_order": 1}],
      "deposit": [{"section": "deposit", "icon": "account_balance_wallet", "tone": "info", "title": "Deposit", "paragraphs":
["Deposit wajib Rp 500.000."], "sort_order": 1}],
      "check_in": [{"section": "check_in", "icon": "login", "tone": "info", "title": "Check-in", "paragraphs": ["Tunjukkan
KTP/Passport."], "sort_order": 1}],
      "check_out": [{"section": "check_out", "icon": "logout", "tone": "info", "title": "Check-out", "paragraphs": ["Kunci kamar
serahkan ke resepsionis."], "sort_order": 1}],
      "room_policy": [{"section": "room_policy", "icon": "hotel", "tone": "neutral", "title": "Kamar", "paragraphs": ["Dilarang
membawa tamu tidak tercatat."], "sort_order": 1}]
    },
    "check_in_info": "Check-in mulai pukul 14:00.",
    "check_out_info": "Check-out sebelum pukul 12:00.",
    "reception_hours": "Resepsionis beroperasi 24 jam setiap hari.",
    "min_age": 18,
    "child_policy": "Anak usia 0-5 tahun menginap gratis.",
    "extra_bed_policy": "Ranjang tambahan tersedia atas permintaan.",
    "smoking_policy": "Semua kamar adalah bebas rokok (Non-Smoking).",
    "deposit_amount": 500000,
    "deposit_currency": "IDR",
    "receptionist_24h": true,
    "allow_children": true,
    "extra_bed_available": true,
    "unit_types": [
      {
        "id": 1,
        "name": "Executive Room",
        "description": "Kamar Executive dengan fasilitas lengkap.",
        "bed_configuration": "1 King Bed atau 2 Single Bed",
        "room_size_sqm": 35.0,
        "base_price": 685000.0,
        "price_original": 950000.0,
        "capacity_adults": 2,
        "capacity_children": 1,
        "max_guests": 3,
        "breakfast_included": true,
        "has_window": true,
        "no_smoking": true,
        "free_wifi": true,
        "total_inventory": 20,
        "is_active": true,
        "facilities": ["Bathtub Marmer", "AC", "TV Kabel Layar Datar"],
        "images": [{"url": "https://images.unsplash.com/photo-...", "caption": "Executive Room - Main View"}],
        "prices": [{"price_type": "daily", "amount": 685000, "cleaning_fee": 0, "deposit": 500000, "service_fee": 68500}]
      }
    ],
    "bed_facilities": [{"icon": "king_bed", "label": "King Bed", "note": "(180x200)", "count": 1}],
    "room_furniture_groups": [
      {
        "id": 1,
        "room": "Living Room",
        "items": [{"id": 1, "name": "Sofa L Shape", "is_bed_config": false}],
        "bed_config": {"bed_type": "King Bed", "count": 1},
        "sort_order": 1
      }
    ],
    "status": "active",
    "created_at": "2026-08-13T04:12:33.000000Z",
    "updated_at": "2026-08-15T09:41:07.000000Z"
  }
```

```
}
```

### Field Reference — Room Mode

| Field | Tipe | Keterangan |
|---|---|---|
| `id` | int | ID akomodasi |
| `booking_mode` | string | Selalu `"room"` |
| `star_rating` | int | Bintang hotel (1-5) |
| `open_year` | int | Tahun operasional |
| `check_in_time` | string | Format `HH:MM:SS` |
| `check_out_time` | string | Format `HH:MM:SS` |
| `deposit_amount` | float/null | Deposit tunai wajib |
| `facilities` | array | Grup fasilitas ber-kategori |
| `facility_items` | array | Item fasilitas multi-konteks |
| `policies` | object | Kebijakan terstruktur per section |
| `unit_types` | array | Tipe kamar (harga + inventaris) |
| `nearby_points` | array | Titik transit/lokasi sekitar |
| `questions` | array | FAQ hotel |
| `bed_facilities` | array | Kartu ranjang |
| `room_furniture_groups` | array | Grup perabot ruangan |

---

## 3. API Response — Unit Mode (`unit`)

Endpoint: `GET host/properties/{id}` atau `GET accommodations/{id}`

Resource: `HostAccommodationResource::unitShape()` atau `AccommodationResource::propertyShape()`


```
{
  "status": true,
  "code": 200,
  "message": "Detail properti berhasil dimuat.",
  "data": {
    "id": 1,
    "host_id": 10,
    "city_id": 1,
    "property_type": "apartment",
    "booking_mode": "unit",
    "name": "Sunset Beach Apartment",
    "building_name": "Sunset Towers",
    "description": "A beautiful apartment with ocean view",
    "address": "Jl. Pantai Indah No. 10, Bali",
    "latitude": -8.65,
    "longitude": 115.2167,
    "guest_capacity": 4,
    "bedrooms": 2,
    "bathrooms": 1,
    "star_rating": null,
    "check_in_time": "14:00:00",
    "check_out_time": "12:00:00",
    "has_breakfast": false,
    "facility_tags": ["WiFi", "Pool", "Gym", "Parking"],
    "gallery": ["https://images.unsplash.com/photo-..."],
    "cover_image": "https://images.unsplash.com/photo-...",
    "status": "active",
    "cancellation_policy": "Pembatalan gratis hingga 24 jam sebelum check-in.",
    "city_name": "Bali",
    "province": "Bali",
    "price_from": 500000,
    "price_original": 950000,
    "rating": 9.6,
    "review_count": 12,
    "prices": [
      {"price_type": "daily", "amount": 500000, "cleaning_fee": 50000, "deposit": 500000, "service_fee": 50000},
      {"price_type": "weekly", "amount": 3000000, "cleaning_fee": 50000, "deposit": 500000, "service_fee": 300000}
    ],
    "host": {"id": 10, "name": "Budi Santoso"},
    "type_label": "Apartment",
    "area_label": "Bali",
    "specs": [{"icon": "furnished", "label": "Full Furnished"}],
    "is_favorite": false,
    "intro": "Cari hunian nyaman? Sewa unit ini...",
    "description_extra": ["Apartemen ini berada di kawasan pantai indah."],
    "description_extra_heading": "Rekomendasi Tempat Rekreasi Terdekat:",
    "map_preview": "https://maps.google.com/?q=-8.65,115.2167",
    "price_caption": "per bulan mulai dari",
    "extra_media_count": 0,
    "is_flash_sale": true,
    "bed_facilities": [{"icon": "king_bed", "label": "King Bed", "note": "(180x200)", "count": 1}],
    "building_facilities": [{"icon": "pool", "label": "Kolam Renang", "note": "24 Jam", "count": null}],
    "building_facilities_all": ["Kolam Renang", "Gym", "Keamanan 24 Jam"],
    "room_furniture": [{"room": "Living Room", "items": [{"name": "Sofa L Shape", "is_bed_config": false}]}],
    "room_furniture_groups": [
      {
```

```
        "id": 1,
        "room": "Living Room",
        "items": [{"id": 1, "name": "Sofa L Shape", "is_bed_config": false}],
        "bed_config": {"bed_type": "King Bed", "count": 1},
        "sort_order": 1
      }
    ],
    "check_in_notes": [{"icon": "key", "tone": "info", "title": "Check-in Mandiri", "text": "Check-in dilakukan secara mandiri
melalui locker pintar."}],
    "deposit_amount": 11175000,
    "deposit_notes": [{"currency": "IDR", "text": "Deposit dikembalikan dalam 3 hari kerja."}],
    "unit_detail": [{"icon": "floor", "label": "Lantai", "value": "12"}],
    "nearby_points": [{"name": "Pantai Kuta", "type": "beach", "distance": "500 m", "duration": "6 menit", "icon": "beach_access"}],
    "guest_media": [],
    "map_preview": "https://maps.google.com/?q=-8.65,115.2167",
    "building_units": {"id": 1, "name": "Sunset Towers", "image": null, "available_units": null},
    "reviews": [{"id": 1, "booking_id": 1, "property_id": 1, "user_id": 2, "user_name": "Siti Nurhaliza", "rating": 9.5, "comment":
"Unit sangat nyaman!", "host_reply": "Terima kasih!", "photos": [], "created_at": "2026-07-20T14:30:00+07:00"}],
    "question_count": 2,
    "questions": [
      {"id": 1, "question": "Apakah tersedia parkir?", "answer": "Ya, 1 slot per unit.", "author_name": "Budi Santoso",
"author_role": "Host", "asked_by": "host"}
    ],
    "facility_items": [
      {"id": 1, "name": "WiFi", "icon": "wifi", "context": "facility", "is_available": true, "badge": "Gratis"},
      {"id": 5, "name": "Sofa", "icon": "weekend", "context": "furniture", "is_available": true, "badge": null},
      {"id": 7, "name": "Full Furnished", "icon": "weekend", "context": "spec", "is_available": true, "badge": null}
    ],
    "bed_facilities": [{"icon": "king_bed", "label": "King Bed", "note": "(180x200)", "count": 1}],
    "unit_types": [
      {
        "id": 1,
        "name": "Default Unit",
        "slug": "sunset-beach-apartment-default-unit",
        "bed_configuration": "1 King Bed + 2 Single Bed",
        "base_price": 500000,
        "price_original": 950000,
        "total_inventory": 5,
        "remaining": 3,
        "capacity_adults": 4,
        "capacity_children": 2,
        "max_guests": 6,
        "bedrooms": 2,
        "bathrooms": 1,
        "room_size_sqm": 85.0,
        "has_breakfast": false,
        "has_window": true,
        "no_smoking": true,
        "free_wifi": true,
        "is_active": true,
        "prices": [{"price_type": "daily", "amount": 500000, "cleaning_fee": 50000, "deposit": 500000, "service_fee": 50000}],
        "images": [{"url": "https://images.unsplash.com/photo-...", "caption": "Main View"}],
        "facility_items": [{"id": 1, "name": "WiFi", "icon": "wifi", "context": "facility", "is_available": true}]
      }
    ],
    "policies": {
      "general": [{"section": "general", "icon": "info", "tone": "neutral", "title": "Informasi Umum", "paragraphs": ["Check-in mulai
pukul 14:00."], "sort_order": 1}],
      "check_in": [{"section": "check_in", "icon": "login", "tone": "info", "title": "Prosedur Check-in", "paragraphs": ["Tunjukkan
KTP/Passport asli."], "sort_order": 1}],
      "check_out": [{"section": "check_out", "icon": "logout", "tone": "info", "title": "Prosedur Check-out", "paragraphs":
["Serahkan kunci akses kartu di resepsionis."], "sort_order": 1}],
      "cancellation": [{"section": "cancellation", "icon": "cancel", "tone": "warning", "title": "Pembatalan", "paragraphs":
["Pembatalan gratis hingga 24 jam."], "sort_order": 1}],
      "smoking_policy": [{"section": "smoking_policy", "icon": "smoke_free", "tone": "info", "title": "Merokok", "paragraphs": ["Area
dalam unit bebas rokok."], "sort_order": 1}],
      "extra_bed_policy": [{"section": "extra_bed_policy", "icon": "bed", "tone": "info", "title": "Ranjang Tambahan", "paragraphs":
["Tidak tersedia."], "sort_order": 1}],
      "deposit": [{"section": "deposit", "icon": "account_balance_wallet", "tone": "info", "title": "Deposit", "paragraphs":
["Deposit sesuai durasi sewa."], "sort_order": 1}],
      "room_policy": [{"section": "room_policy", "icon": "hotel", "tone": "neutral", "title": "Hunian", "paragraphs": ["Dilarang
mengadakan acara tanpa seizin host."], "sort_order": 1}]
    },
    "check_in_info": "Check-in mulai pukul 14:00.",
    "check_out_info": "Check-out sebelum pukul 12:00.",
    "reception_hours": "Resepsionis beroperasi 24 jam.",
    "min_age": 18,
    "child_policy": "Anak usia 0-5 tahun menginap gratis.",
    "extra_bed_policy": "Ranjang tambahan tidak tersedia.",
    "smoking_policy": "Area dalam unit adalah area bebas rokok.",
    "receptionist_24h": true,
    "allow_children": true,
    "extra_bed_available": true,
    "unit_detail": [{"icon": "floor", "label": "Lantai", "value": "12"}],
    "description_paragraphs": ["Apartemen ini berada di kawasan pantai indah.", "Gedung ini dilengkapi keamanan 24 jam."],
    "details": [{"icon": "floor", "label": "Lantai", "value": "12"}],
```

```
    "created_at": "2026-08-13T04:12:33.000000Z",
    "updated_at": "2026-08-15T09:41:07.000000Z"
  }
}
```

### Field Reference — Unit Mode

| Field | Tipe | Keterangan |
|---|---|---|
| `id` | int | ID akomodasi |
| `booking_mode` | string | Selalu `"unit"` |
| `property_type` | string | `apartment`, `house`, `villa`, dll |
| `building_name` | string | Nama gedung/tower |
| `guest_capacity` | int | Kapasitas tamu maksimal |
| `bedrooms` | int | Jumlah kamar tidur |
| `bathrooms` | int | Jumlah kamar mandi |
| `has_breakfast` | bool | Tersedia sarapan |
| `facility_tags` | array | Tag fasilitas ringkas |
| `prices` | array | Harga multi-durasi |
| `host` | object | Ringkasan host |
| `type_label` | string | Label tipe properti |
| `area_label` | string | Label kota/area |
| `specs` | array | Spesifikasi singkat |
| `intro` | string | Headline marketing |
| `description_extra` | array | Paragraf deskripsi tambahan |
| `bed_facilities` | array | Kartu ranjang |
| `building_facilities` | array | Fasilitas gedung |
| `room_furniture` | array | Perabot ruangan |
| `room_furniture_groups` | array | Grup perabot terstruktur |
| `check_in_notes` | array | Catatan check-in |
| `deposit_notes` | array | Catatan deposit |
| `unit_detail` | array | Detail unit (lantai, luas, view) |
| `nearby_points` | array | Lokasi sekitar |
| `guest_media` | array | Media tambahan dari tamu |
| `building_units` | object/null | Mirror gedung/tower |
| `questions` | array | FAQ properti |
| `facility_items` | array | Item fasilitas multi-konteks (`facility`, `furniture`, `spec`) |

---

## 4. Request Payload — Submit Form

### 4.1 Room Mode — `POST host/hotels`

```
{
  "city_id": 1,
  "name": "Merlynn Park Hotel",
  "address": "Jl. K.H. Hasyim Ashari No. 29-31, Jakarta Pusat",
  "description": "Hotel bintang 5 bergaya megah...",
  "latitude": -6.166667,
  "longitude": 106.816667,
  "open_year": 2010,
  "check_in_time": "14:00:00",
  "check_out_time": "12:00:00",
  "deposit_amount": 500000,
  "check_in_info": "Check-in mulai pukul 14:00...",
  "check_out_info": "Check-out sebelum pukul 12:00.",
  "reception_hours": "Resepsionis beroperasi 24 jam setiap hari.",
  "min_age": 18,
  "child_policy": "Anak usia 0-5 tahun menginap gratis...",
  "extra_bed_policy": "Ranjang tambahan tersedia...",
  "smoking_policy": "Semua kamar adalah bebas rokok...",
  "receptionist_24h": true,
  "allow_children": true,
  "extra_bed_available": true,
  "cancellation_policy": "Pembatalan gratis hingga 24 jam...",
  "facility_groups": [
    {
      "category": "Fasilitas Utama",
      "icon": "star",
      "items": [
        {"name": "Resepsionis & Keamanan 24 Jam", "is_available": true, "badge": null}
      ]
    }
  ],
  "policies": [
    {"section": "general", "icon": "info", "tone": "neutral", "title": "Informasi Umum", "paragraphs": ["Check-in mulai pukul
14:00."]},
    {"section": "cancellation", "icon": "cancel", "tone": "warning", "title": "Pembatalan", "paragraphs": ["Pembatalan gratis hingga
24 jam."]}
  ],
  "faqs": [
    {"question": "Apakah tersedia laundry?", "answer": "Ya, 24 jam.", "author_name": "Manager", "author_role": "Staff"}
  ],
  "transport_points": [
    {"name": "Stasiun Gambir", "type": "train", "distance": "1,2 km", "duration": "15 menit", "icon": "train"}
  ],
  "gallery": ["https://images.unsplash.com/photo-..."],
  "status": "active"
}
```


**Room Types — `POST host/hotels/{hotelId}/room-types`**


```
{
  "name": "Executive Room",
  "description": "Kamar Executive dengan fasilitas lengkap.",
  "capacity_adults": 2,
  "capacity_children": 1,
  "max_guests": 3,
  "room_size_sqm": 35.0,
  "bed_configuration": "1 King Bed atau 2 Single Bed",
  "base_price": 685000,
  "price_original": 950000,
  "breakfast_included": true,
  "has_window": true,
  "no_smoking": true,
  "free_wifi": true,
  "total_inventory": 20,
  "is_active": true,
  "facilities": ["Bathtub Marmer", "AC", "TV Kabel Layar Datar"],
  "images": [{"url": "https://images.unsplash.com/photo-...", "caption": "Executive Room - Main View"}]
}
```


### 4.2 Unit Mode — `POST host/properties`

```
{
  "property_type": "apartment",
  "booking_mode": "unit",
  "name": "Sunset Beach Apartment",
  "building_name": "Sunset Towers",
  "description": "A beautiful apartment with ocean view",
  "address": "Jl. Pantai Indah No. 10, Bali",
  "latitude": -8.65,
  "longitude": 115.2167,
  "open_year": 2020,
  "check_in_time": "14:00:00",
  "check_out_time": "12:00:00",
  "has_breakfast": false,
  "guest_capacity": 4,
  "bedrooms": 2,
  "bathrooms": 1,
  "facilities": ["wifi", "ac", "kitchen", "parking", "swimming_pool"],
  "facility_tags": ["WiFi", "Pool", "Gym", "Parking"],
  "gallery": ["https://images.unsplash.com/photo-..."],
  "status": "active",
  "cancellation_policy": "Pembatalan gratis hingga 24 jam sebelum check-in.",
  "map_preview": "https://maps.google.com/?q=-8.65,115.2167",
  "is_recommended": true,
  "recommendation_score": 9.5,
  "recommendation_reason": "Pilihan terbaik di Bali",
  "prices": [
    {"price_type": "daily", "amount": 500000, "cleaning_fee": 50000, "deposit": 500000, "service_fee": 50000},
    {"price_type": "weekly", "amount": 3000000, "cleaning_fee": 50000, "deposit": 500000, "service_fee": 300000}
  ],
  "other": {
    "province": "Bali",
    "cover": "https://images.unsplash.com/photo-...",
    "cover_image": "https://images.unsplash.com/photo-...",
    "type_label": "Apartment",
    "area_label": "Bali",
    "extra_media_count": 0,
    "specs": [{"icon": "furnished", "label": "Full Furnished"}],
    "is_favorite": false,
    "intro": "Cari hunian nyaman? Sewa unit ini...",
    "description_extra": ["Apartemen ini berada di kawasan pantai indah."],
    "description_extra_heading": "Rekomendasi Tempat Rekreasi Terdekat:",
    "price_caption": "per bulan mulai dari",
    "is_flash_sale": true,
    "bed_facilities": [{"icon": "king_bed", "label": "King Bed", "note": "(180x200)", "count": 1}],
    "building_facilities": [{"icon": "pool", "label": "Kolam Renang", "note": "24 Jam", "count": null}],
    "building_facilities_all": ["Kolam Renang", "Gym", "Keamanan 24 Jam"],
    "room_furniture": [{"room": "Living Room", "items": [{"name": "Sofa L Shape", "is_bed_config": false}]}],
    "room_furniture_groups": [{"id": 1, "room": "Living Room", "items": [{"id": 1, "name": "Sofa L Shape", "is_bed_config": false}],
"bed_config": {"bed_type": "King Bed", "count": 1}, "sort_order": 1}],
    "check_in_notes": [{"icon": "key", "tone": "info", "title": "Check-in Mandiri", "text": "Check-in dilakukan secara mandiri
melalui locker pintar."}],
    "deposit_amount": 11175000,
    "deposit_notes": [{"currency": "IDR", "text": "Deposit dikembalikan dalam 3 hari kerja."}],
    "unit_detail": [{"icon": "floor", "label": "Lantai", "value": "12"}],
    "nearby_points": [{"name": "Pantai Kuta", "type": "beach", "distance": "500 m", "duration": "6 menit", "icon": "beach_access"}],
    "guest_media": [],
    "building_units": {"id": 1, "name": "Sunset Towers", "image": null, "available_units": null},
    "map_preview": "https://maps.google.com/?q=-8.65,115.2167",
    "facility_tags": ["WiFi", "Pool", "Gym", "Parking"],
    "has_breakfast": false
  }
}
```


### 4.3 Endpoint Summary

| Mode | Method | Endpoint | Keterangan |
|---|---|---|---|
| room | POST | /api/v1/host/hotels | Buat hotel baru |
| room | PUT | /api/v1/host/hotels/{id} | Update metadata hotel |
| room | POST | /api/v1/host/hotels/{id}/room-types | Tambah tipe kamar |
| room | PUT | /api/v1/host/room-types/{id} | Edit tipe kamar |
| unit | POST | /api/v1/host/properties | Buat properti baru |
| unit | PUT | /api/v1/host/properties/{id} | Update properti |

---

## 5. JSON Dummy / Fixture Files

| File | Mode | Kegunaan |
|---|---|---|
| flutter/assets/json/accommodation_room_dummy.json | room | Load data hotel ke form |
| flutter/assets/json/accommodation_unit_dummy.json | unit | Load data properti ke form |

### 5.1 Room Dummy Structure

```
{
  "id": 1,
  "booking_mode": "room",
  "name": "Merlynn Park Hotel",
  "star_rating": 5,
  "open_year": 2010,
  "check_in_time": "14:00:00",
  "check_out_time": "12:00:00",
  "deposit_amount": "500000.00",
  "description": "...",
  "address": "...",
  "city_name": "Jakarta Pusat",
  "latitude": -6.166667,
  "longitude": 106.816667,
  "cancellation_policy": "Pembatalan gratis hingga 24 jam sebelum check-in.",
  "facilities": [...],
  "facility_items": [...],
  "policies": {"general": [...], "cancellation": [...], "smoking_policy": [...], "extra_bed_policy": [...], "deposit": [...],
"check_in": [...], "check_out": [...], "room_policy": [...]},
  "unit_types": [...],
  "gallery": [...],
  "questions": [...],
  "nearby_points": [...],
  "check_in_info": "...",
  "check_out_info": "...",
  "reception_hours": "...",
  "min_age": 18,
  "child_policy": "...",
  "extra_bed_policy": "...",
  "smoking_policy": "...",
  "receptionist_24h": true,
  "allow_children": true,
  "extra_bed_available": true,
  "status": "active"
}
```


### 5.2 Unit Dummy Structure

```
{
  "id": 1,
  "booking_mode": "unit",
  "property_type": "apartment",
  "name": "Sunset Beach Apartment",
  "building_name": "Sunset Towers",
  "description": "...",
  "address": "...",
  "city_id": 1,
  "city_name": "Bali",
  "province": "Bali",
  "latitude": -8.65,
  "longitude": 115.2167,
  "guest_capacity": 4,
  "bedrooms": 2,
  "bathrooms": 1,
  "facility_tags": ["WiFi", "Pool", "Gym", "Parking"],
  "facility_items": [...],
  "gallery": [...],
  "cover_image": "...",
  "cancellation_policy": "...",
  "prices": [...],
  "host": {"id": 10, "name": "Budi Santoso"},
  "type_label": "Apartment",
  "area_label": "Bali",
  "specs": [...],
  "intro": "...",
  "description_extra": [...],
  "description_extra_heading": "...",
  "map_preview": "...",
  "price_caption": "per bulan mulai dari",
  "is_flash_sale": true,
  "bed_facilities": [...],
  "building_facilities": [...],
  "building_facilities_all": [...],
  "room_furniture": [...],
  "room_furniture_groups": [...],
  "check_in_notes": [...],
  "deposit_amount": 11175000,
  "deposit_notes": [...],
  "unit_detail": [...],
  "nearby_points": [...],
  "guest_media": [],
  "building_units": {...},
  "questions": [...],
  "policies": {"general": [...], "cancellation": [...], "check_in": [...], "check_out": [...], "smoking_policy": [...],
"extra_bed_policy": [...], "deposit": [...], "room_policy": [...]},
  "unit_types": [...],
  "status": "active"
}
```


---

## 6. Laravel Resource Mapping

### 6.1 Resource Baru

File: `laravel/app/Http/Resources/HostAccommodationResource.php`

Resource ini mengekspos:

- **Room mode (`roomShape`)**: `facilities`, `facility_items`, `policies`, `unit_types`, `nearby_points`, `questions`, `galleries`, `bed_facilities`, `room_furniture_groups`, `intro`, `description_extra`, `description_extra_heading`.
- **Unit mode (`unitShape`)**: `facility_tags`, `gallery`, `prices`, `host`, `specs`, `intro`, `description_extra`, `description_extra_heading`, `bed_facilities`, `building_facilities`, `room_furniture`, `room_furniture_groups`, `check_in_notes`, `deposit_notes`, `unit_detail`, `nearby_points`, `guest_media`, `building_units`, `reviews`, `questions`, `unit_types`.

### 6.2 Eager Loading Contract

**Room mode:**

```
Accommodation::hotels()->with([
    'city', 'buildingList', 'galleries',
    'policies', 'questions', 'nearbyPoints',
    'facilityGroups.items', 'facilityItems', 'facilityTags',
    'unitTypes.images', 'unitTypes.facilityItems',
    'bedFacilities', 'roomFurnitureGroups', 'details',
    'descriptionParagraphs',
])->findOrFail($id);
```

**Unit mode:**

```
Accommodation::properties()->with([
    'city', 'buildingList', 'galleries',
    'policies', 'questions', 'nearbyPoints',
    'facilityItems', 'facilityTags', 'bedFacilities',
    'roomFurnitureGroups', 'details', 'cheapestUnitType'
])->findOrFail($id);
```

### 6.3 Resource yang Digunakan

| Resource | Kegunaan |
|---|---|
| `AccommodationResource` | Public detail (auto-branching `hotelShape`/`propertyShape`) |
| `HostAccommodationResource` | Host-side form detail (menambahkan `policies` sebagai object, `facility_items`, dsb) |
| `AccommodationPolicyResource` | Single policy block + `toNoteArray()` untuk check-in/deposit notes |
| `AccommodationQuestionResource` | FAQ hotel (`toArray`) dan properti (`toPropertyArray`) |
| `NearbyPointResource` | Transport/nearby point (`toArray` untuk hotel, `toPropertyArray` untuk properti) |
| `FacilityGroupResource` | Grup fasilitas hotel |
| `FacilityItemResource` | Item fasilitas multi-konteks (`toSpecArray`, `toBuildingFacilityArray`, `toRoomFurnitureArray`) |
| `UnitTypeResource` | Tipe kamar/unit |
| `UnitTypePriceResource` | Harga periodik properti |
| `BedFacilityResource` | Kartu ranjang |
| `RoomFurnitureGroupResource` | Grup perabot ruangan |
| `GalleryResource` | URL galeri |
| `BuildingListResource` | Gedung/tower (`toArray` untuk hotel, `toPropertyArray` untuk properti) |

---

## 7. Panduan Migrasi — Developer Mobile

### 7.1 File Legacy yang Dihapus

| File Legacy | Pengganti | Catatan |
|---|---|---|
| `flutter/lib/screens/mitra/hotel_form_screen.dart` | `accommodation_form_screen.dart` | Semua logic hotel kini di dalam form terpadu |
| `flutter/lib/screens/mitra/property_form_payload.dart` | Inline `buildPropertyPayload()` di `accommodation_form_screen.dart` | Payload builder tidak lagi sebagai file terpisah |

### 7.2 Perubahan Routing

Route lama:

- `/mitra/properties/form` → `PropertyFormScreen`
- `/mitra/hotels/form` → `HotelFormScreen`

Route baru:

- `/mitra/accommodations/form` → `AccommodationFormScreen`

Mode ditentukan via argumen `bookingMode`:

```
// Room mode (Hotel)
Navigator.pushNamed(
  context,
  Routes.mitraAccommodationForm,
  arguments: {'booking_mode': 'room', 'hotel_id': 1, 'property_id': 1},
);

// Unit mode (Property)
Navigator.pushNamed(
  context,
  Routes.mitraAccommodationForm,
  arguments: {'booking_mode': 'unit', 'property_id': 1},
);

// Mode selector (tampilkan pilihan room/unit terlebih dahulu)
Navigator.pushNamed(
  context,
  Routes.mitraAccommodationForm,
);
```

### 7.3 Perubahan Model & Parsing

**Lama (Hotel):**

```
// hotel_form_screen.dart
final hotel = await _service.getHostHotelDetail(hotelId);
// parsing manual ke _roomTypes, _facilityGroups, _faqs, dsb
```

**Baru (AccommodationFormScreen):**

```
// accommodation_form_screen.dart
if (_isHotel) {
  await _loadHotelData(); // memanggil hotelService.getHostHotelDetail
} else {
  await _loadPropertyData(id); // memanggil hostService.propertyDetail
}
```

### 7.4 Perubahan Submit

**Lama:**

- Hotel: `HotelFormScreen._submit()` → `hotelService.createHotelForProperty()` atau `updateHotelForProperty()`
- Property: `PropertyFormScreen._submit()` → `hostService.createProperty()` atau `updateProperty()`

**Baru:**

- Hotel: `AccommodationFormScreen._submitHotel()` → `hotelService.createHotelForProperty()` atau `updateHotelForProperty()`
- Property: `AccommodationFormScreen._submitUnit()` → `hostService.createProperty()` atau `updateProperty()`

Payload builder untuk unit kini inline di file yang sama:

```
final body = buildPropertyPayload(form: {...}, status: status);
```

### 7.5 JSON Dummy Baru

Gunakan file dummy baru yang terpisah:

| Mode | File Dummy | Catatan |
|---|---|---|
| `room` | `flutter/assets/json/accommodation_room_dummy.json` | Struktur lengkap hotel dengan `policies` sebagai object multi-section |
| `unit` | `flutter/assets/json/accommodation_unit_dummy.json` | Struktur lengkap properti dengan `facility_items` multi-konteks |

### 7.6 Draft Autosave

Draft kini menggunakan key yang sama untuk kedua mode:

```
const _kDraftKey = 'accommodation_form_draft';
```

Field yang disimpan: `name`, `address`, `building_name`, `description`, `property_type`, `city_id`, `guest_capacity`, `bedrooms`, `facilities`, `gallery`.

---

## 8. Panduan Migrasi — Developer Backend

### 8.1 Resource Baru

Buat file `laravel/app/Http/Resources/HostAccommodationResource.php` jika belum ada. Resource ini:

- Meng-extend `JsonResource`
- Memiliki dua method: `roomShape()` dan `unitShape()`
- Menggunakan `whenLoaded()` untuk semua relasi opsional
- Menghasilkan JSON yang konsisten dengan dummy Flutter

### 8.2 Controller Update

Di `HostHotelController`, gunakan `HostAccommodationResource` untuk response host-side:

```
use App\Http\Resources\HostAccommodationResource;

public function show(Request $request, $id)
{
    $hotel = Accommodation::hotels()
        ->where('host_id', $request->user()->id)
        ->with([
            'city', 'buildingList', 'galleries',
            'policies', 'questions', 'nearbyPoints',
            'facilityGroups.items', 'facilityItems', 'facilityTags',
            'unitTypes.images', 'unitTypes.facilityItems',
            'bedFacilities', 'roomFurnitureGroups', 'details'
        ])->findOrFail($id);

    return ApiResponse::success(
        (new HostAccommodationResource($hotel))->resolve(),
        'Detail hotel berhasil dimuat.'
    );
}
```

Di `PropertyController`, gunakan `HostAccommodationResource` untuk response host-side:

```
public function hostShow(Request $request, $id)
{
    $property = Accommodation::properties()
        ->where('host_id', $request->user()->id)
        ->with([
            'city', 'buildingList', 'galleries',
            'policies', 'questions', 'nearbyPoints',
            'facilityItems', 'facilityTags', 'bedFacilities',
            'roomFurnitureGroups', 'details', 'cheapestUnitType'
        ])->findOrFail($id);

    return ApiResponse::success(
        (new HostAccommodationResource($property))->resolve(),
        'Detail properti berhasil dimuat.'
    );
}
```

### 8.3 Migration Steps Summary

1. **Flutter**: Ganti semua import `hotel_form_screen.dart` dan `property_form_payload.dart` menjadi `accommodation_form_screen.dart`.
2. **Flutter**: Update semua `Navigator.pushNamed` yang menuju `Routes.mitraHotelForm` atau `Routes.mitraPropertyForm` menjadi `Routes.mitraAccommodationForm` dengan argumen `booking_mode`.
3. **Flutter**: Hapus file `hotel_form_screen.dart` dan `property_form_payload.dart` setelah memastikan tidak ada dependency.
4. **Laravel**: Buat `HostAccommodationResource.php` jika belum ada.
5. **Laravel**: Update `HostHotelController::show()` dan `PropertyController::hostShow()` untuk menggunakan `HostAccommodationResource`.
6. **Laravel**: Pastikan eager loading contract terpenuhi di semua controller yang memuat data akomodasi untuk form.

### 8.4 Backward Compatibility

- Endpoint lama (`GET host/hotels/{id}`, `GET host/properties/{id}`) tetap berfungsi.
- Response lama (`AccommodationResource`) tetap dipertahankan untuk public consumer.
- `HostAccommodationResource` hanya digunakan untuk host-side form detail.
- Flutter `AccommodationFormScreen` dapat menerima data dari `AccommodationResource` atau `HostAccommodationResource` karena struktur JSON-nya seragam.

### 8.5 Testing Checklist

- [ ] Flutter `flutter analyze` tidak ada error.
- [ ] Mode selector menampilkan 2 pilihan (Hotel/Inn dan Properti/Unit).
- [ ] Tab 1-5 untuk room mode dapat di-navigate.
- [ ] Tab 1-4 untuk unit mode dapat di-navigate.
- [ ] Load data dari `accommodation_room_dummy.json` berhasil mengisi form hotel.
- [ ] Load data dari `accommodation_unit_dummy.json` berhasil mengisi form properti.
- [ ] Submit form hotel mengirim payload yang sesuai dengan `POST host/hotels`.
- [ ] Submit form properti mengirim payload yang sesuai dengan `POST host/properties`.
- [ ] Draft autosave bekerja untuk kedua mode.
- [ ] Validasi form menangkap field kosong sebelum submit.

---

## 9. Daftar File Terkait

| File | Kegunaan |
|---|---|
| `flutter/lib/screens/mitra/accommodation_form_screen.dart` | Layar form terpadu (room + unit) |
| `flutter/assets/json/accommodation_room_dummy.json` | Dummy data hotel untuk form |
| `flutter/assets/json/accommodation_unit_dummy.json` | Dummy data properti untuk form |
| `laravel/app/Http/Resources/HostAccommodationResource.php` | Resource host-side form detail |
| `laravel/app/Http/Resources/AccommodationResource.php` | Resource public detail (auto-branching) |
| `laravel/app/Http/Controllers/Api/HostHotelController.php` | Controller host hotel CRUD |
| `laravel/app/Http/Controllers/Api/PropertyController.php` | Controller host property CRUD |
| `docs/domain/accommodations.md` | Master arsitektur akomodasi (21 tabel, 3NF) |
| `docs/ai/02-frontend-map.md` | Peta Flutter (47 route, 45 screen) |
| `docs/ai/03-api-contract.md` | Kontrak API 84 endpoint |

## workflows/run_api_sequence.xml

```
<laravel_cli_workflow>
  <metadata>
    <description>Automated terminal workflow: Start server, authenticate, and test endpoints.</description>
    <base_url>http://127.0.0.1:8000</base_url>
    <login_endpoint>/api/v1/auth/login</login_endpoint>
    <credentials>
      <email>andi@example.com</email>
      <password>password</password>
    </credentials>
  </metadata>

  <execution_steps>
    Step 1: Start Laravel built-in development server in the background (php artisan serve).
    Step 2: Send a POST request to <login_endpoint> using curl with the provided credentials.
    Step 3: Capture the returned authentication token/cookie from the login response dynamically into working memory.
    Step 4: Use that captured token to test a protected endpoint (e.g., GET /api/v1/orders) to verify if the response structure
matches expected parameters.
    Step 5: If any parameters are mismatched or broken, output the code fixes for the Controller/Resource.
  </execution_steps>
</laravel_cli_workflow>
```